

《神经网络与深度学习》

循环神经网络

<https://nndl.github.io/>

参考资料

- ▶ 《神经网络与深度学习》 第6章

- ▶ <https://nndl.github.io/>

- ▶ 网络资料

- ▶ An Introduction to Recurrent Neural Networks

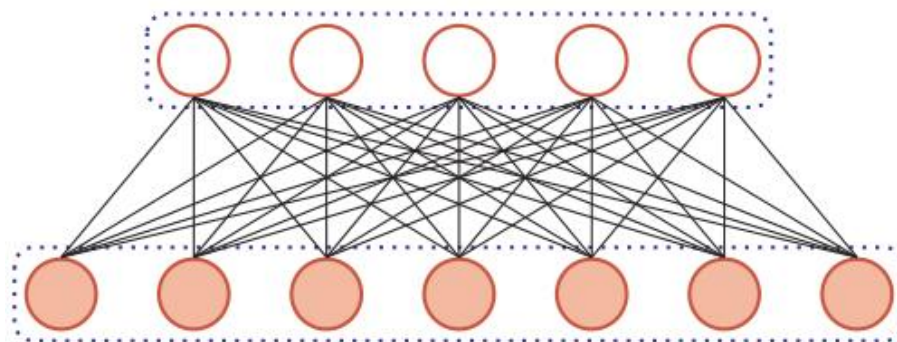
- ▶ <https://medium.com/explore-artificial-intelligence/an-introduction-to-recurrent-neural-networks-72c97bf0912>

- ▶ Recurrent Neural Networks

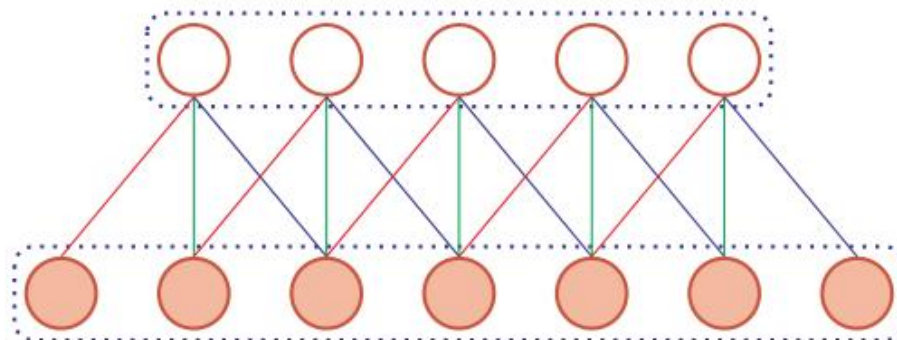
- ▶ <https://towardsdatascience.com/recurrent-neural-networks-d4642c9bc7ce>

前馈网络

- ▶ 连接存在层与层之间，每层的节点之间是无连接的。（无循环）
 - ▶ 输入和输出的维数都是固定的，不能任意改变。无法处理变长的序列数据。



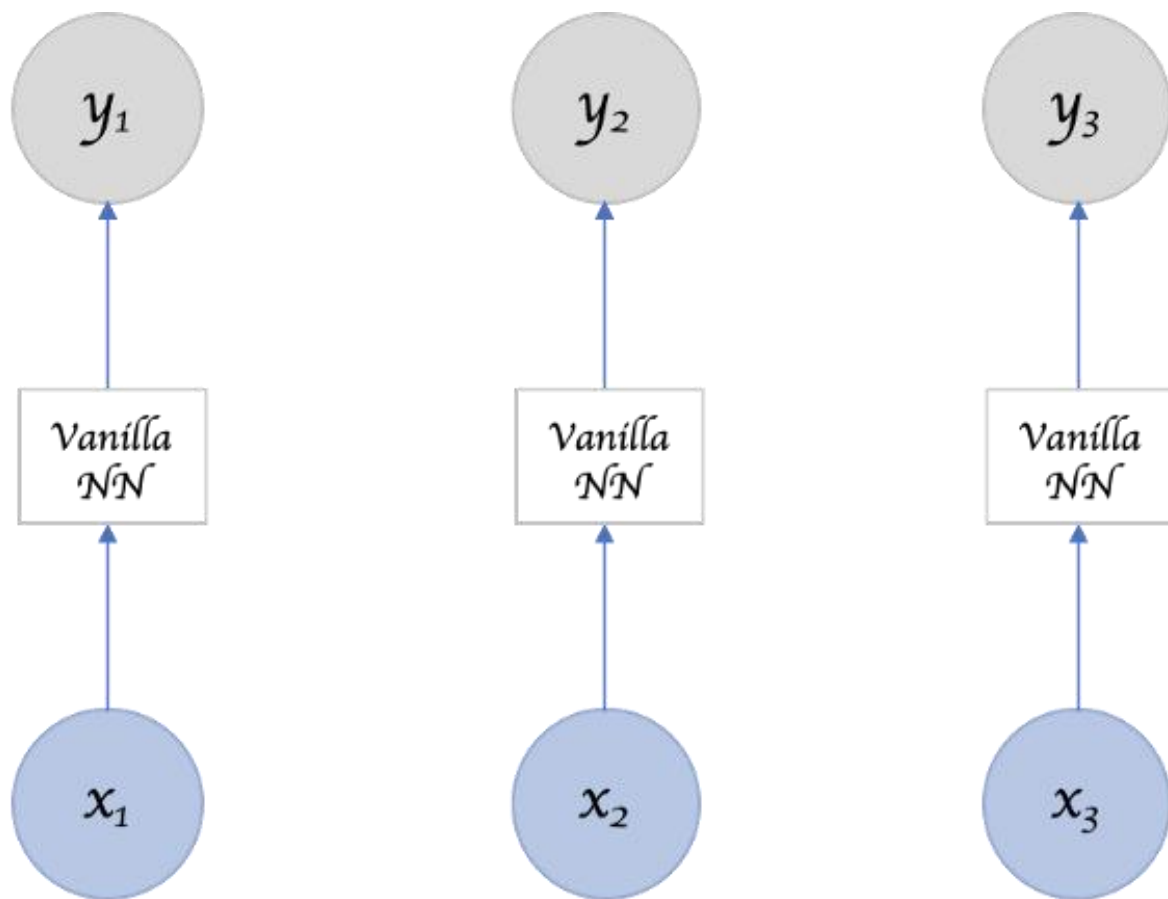
(a) 全连接层



(b) 卷积层

前馈网络

- 假设每次输入都是独立的，也就是说每次网络的输出只依赖于当前的输入。



有限状态自动机 (Finite Automata)

- ▶ 有限状态自动机是一种具有离散输入/输出系统的数学模型，简称有限自动机。
- ▶ 状态: 一个标识，能区分自动机在不同时刻的状况。有限状态系统具有任意有限数目的内部“状态”
- ▶ 自动机接受一定的输入，执行一定的动作，产生一定的结果。
- ▶ 自动机的本质: 根据状态、输入和规则决定下一个状态
 - ▶ 状态 + 输入(激励) + 规则 $\rightarrow$ 状态迁移
 - ▶ 可能的状态、运行的规则都是事先确定的。一旦开始运行，就按照实现确定的规则工作，因此叫“自动机”。使用状态迁移描述整个工作过程

有限状态自动机 (Finite Automata)

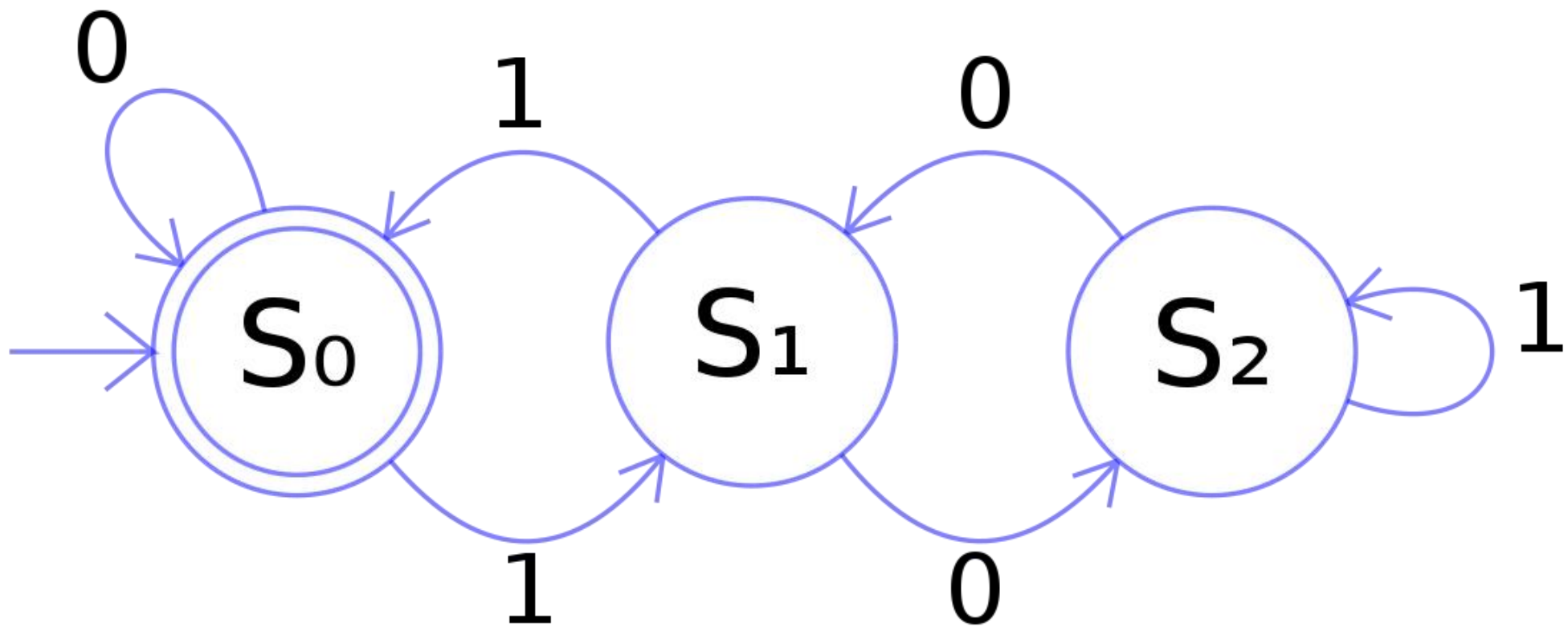
A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of **states**,
- Σ is a finite **alphabet**,
- $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ are the **accept states**.

Here δ is a **total** function, that is, δ must be defined for all possible inputs.

CSDN @暖仔会飞

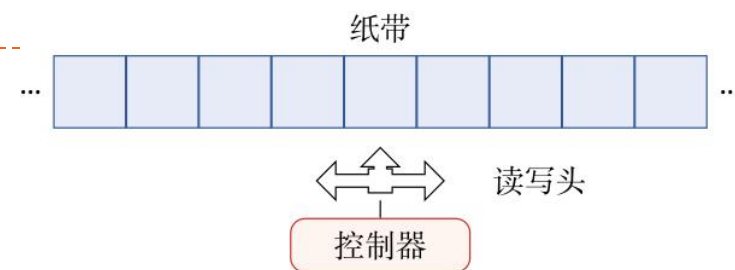
有限状态自动机 (Finite Automata)



如何用FNN去模拟一个有限状态自动机?

图灵机

► 一种抽象数学模型，可以用来模拟任何可计算问题。



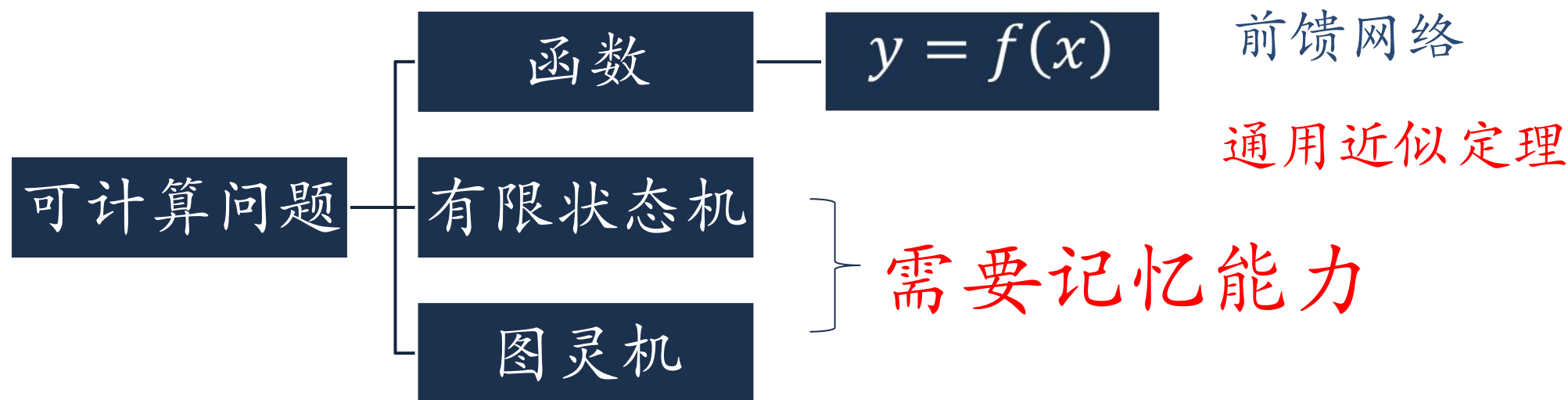
工作原理:读头在输入带上从左向右移动，每当读头从带上读到一个字符时，便引起控制器状态的改变，同时读头右移一个符号的位置。控制器:控制器包括有限个状态，状态与状态之间存在着某种转换关系。每当在某一状态下读入一个字符时，便使状态发生改变（称为状态转换）。状态转换包括以下几种情况:

转换到自身-保持当前状态不变

转换的后继状态只有一个

转换的后继状态有若干个

可计算问题



如何给网络增加记忆能力？

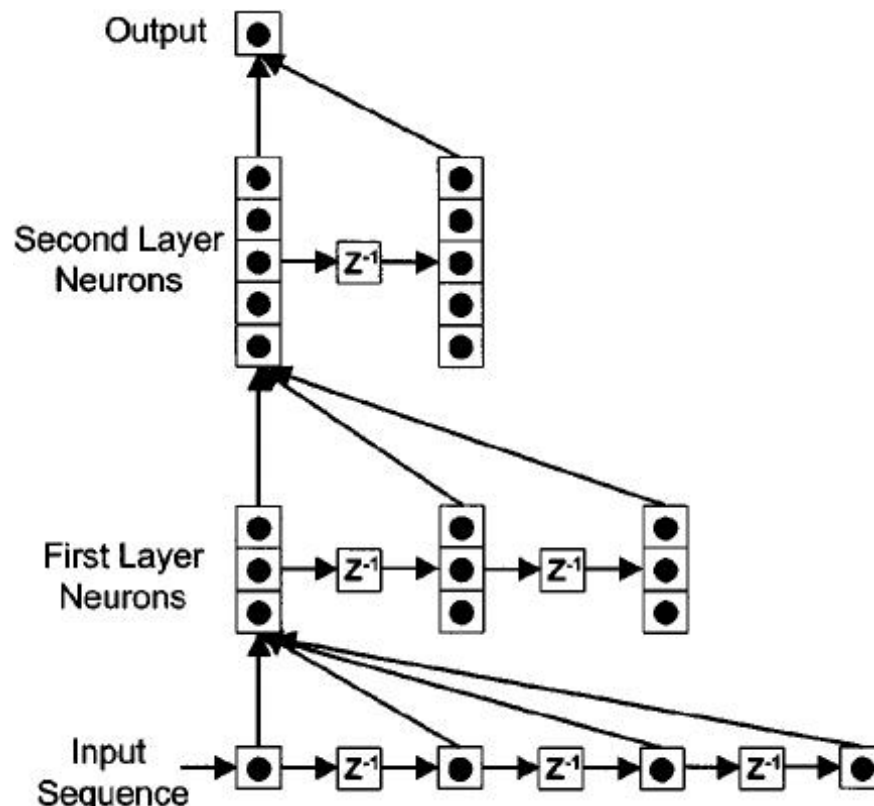
如何给网络增加记忆能力？

▶ 延时神经网络 (Time Delay Neural Network, TDNN)

- ▶ 建立一个额外的延时单元，用来存储网络的历史信息（可以包括输入、输出、隐状态等）

$$\mathbf{h}_t^{(l)} = f(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l-1)}, \dots, \mathbf{h}_{t-K}^{(l-1)})$$

- ▶ 这样，前馈网络就具有了短期记忆的能力。



<https://www.researchgate.net/publication/12314435> Neural system identification model of human sound localization

如何给网络增加记忆能力?

▶ 自回归模型 (Autoregressive Model, AR)

- ▶ 一类时间序列模型，用变量 y_t 的历史信息来预测自己

$$y_t = w_0 + \sum_{k=1}^K w_k y_{t-k} + \epsilon_t$$

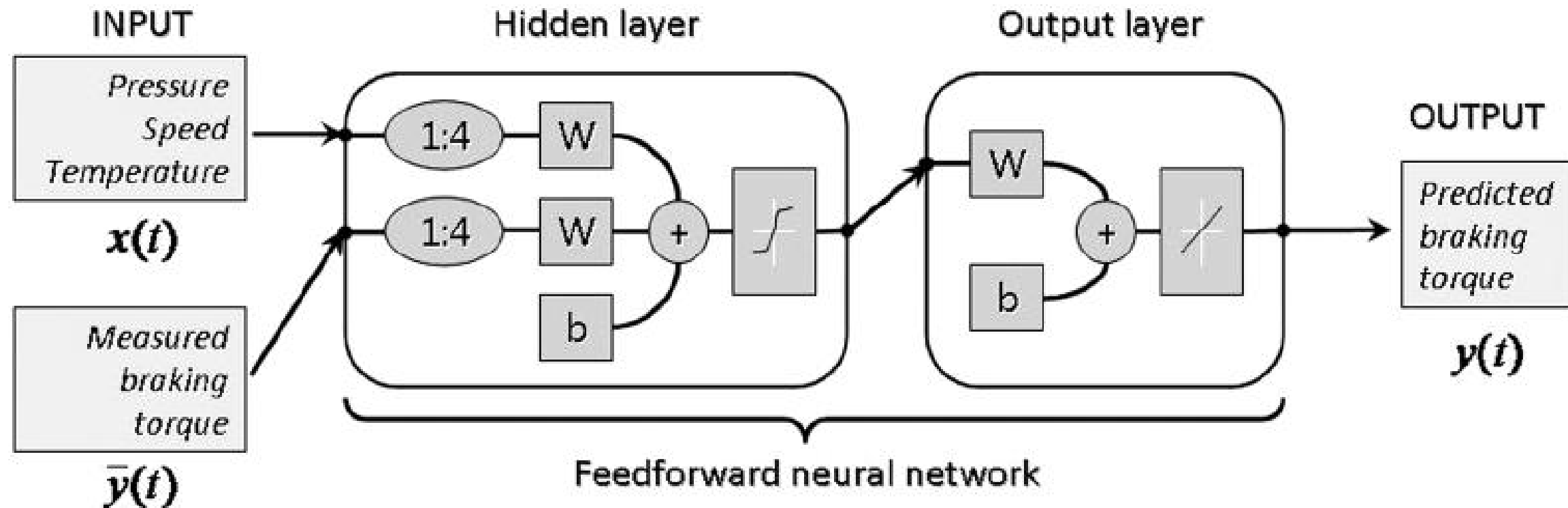
- ▶ $\epsilon_t \sim N(0, \sigma^2)$ 为第 t 个时刻的噪声

▶ 有外部输入的非线性自回归模型 (Nonlinear Autoregressive with Exogenous Inputs Model, NARX)

$$y_t = f(\mathbf{x}_t, \mathbf{x}_{t-1}, \dots, \mathbf{x}_{t-K_x}, y_{t-1}, y_{t-2}, \dots, y_{t-K_y})$$

- ▶ 其中 $f(\cdot)$ 表示非线性函数，可以是一个前馈网络， K_x 和 K_y 为超参数。

非线性自回归模型



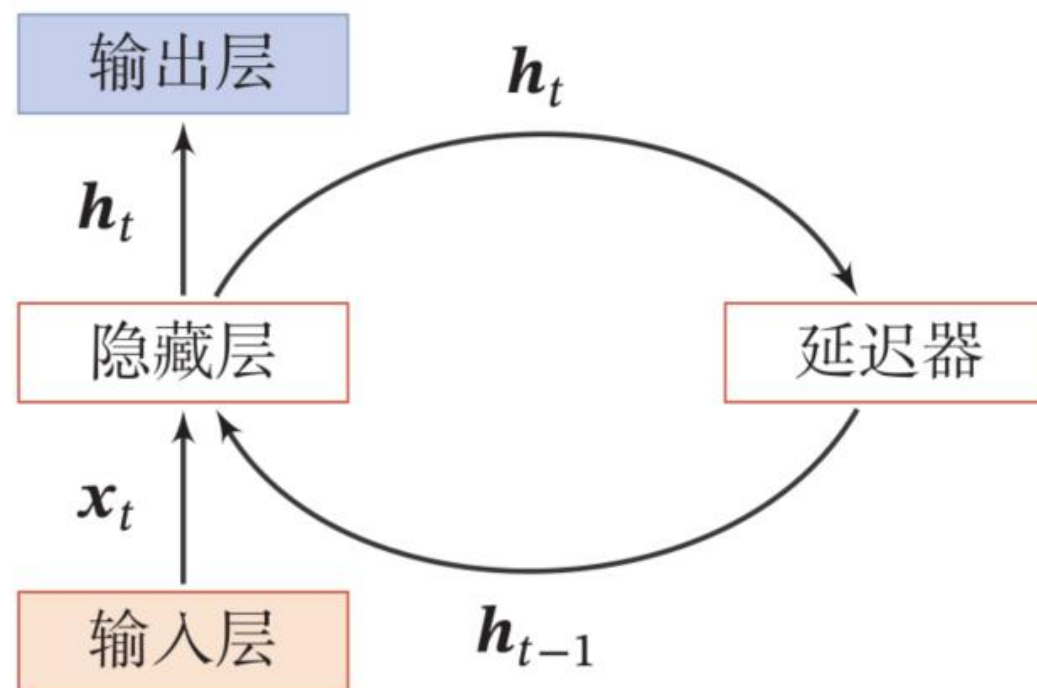
https://www.researchgate.net/publication/234052442_Braking_torque_control_using_reccurent_neural_networks

循环神经网络（Recurrent Neural Network，RNN）

- ▶ 循环神经网络通过使用带自反馈的神经元，能够处理任意长度的时序数据。

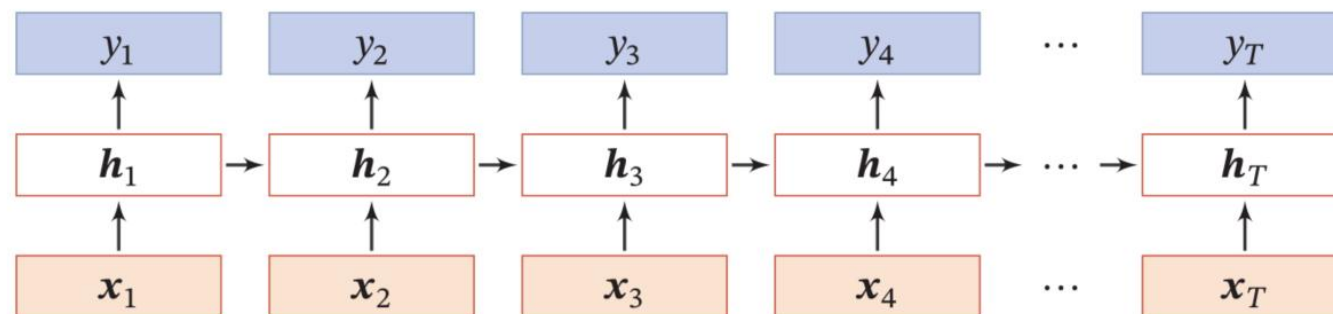
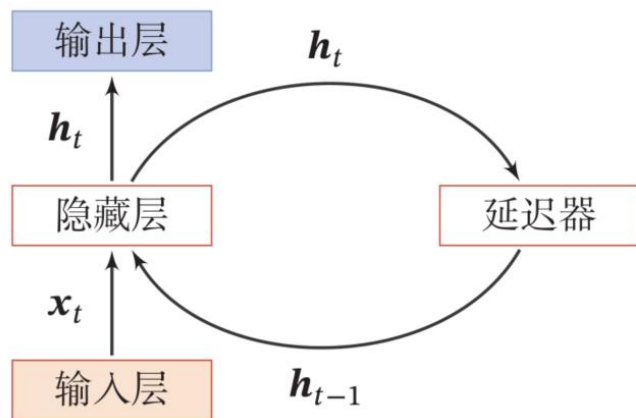
$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

活性值
状态



- ▶ 循环神经网络比前馈神经网络更加符合生物神经网络的结构。
- ▶ 循环神经网络已经被广泛应用在语音识别、语言模型以及自然语言生成等任务上

按时间展开



简单循环网络 (Simple Recurrent Network , SRN)

▶ 状态更新:

$$\mathbf{h}_t = f(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t + \mathbf{b})$$

▶ 一个完全连接的循环网络是任何非线性动力系统的近似器。

定理 6.1 – 循环神经网络的通用近似定理 [Haykin, 2009]: 如果一个完全连接的循环神经网络有足够数量的 sigmoid 型隐藏神经元, 它可以以任意的准确率去近似任何一个非线性动力系统

$$\mathbf{s}_t = g(\mathbf{s}_{t-1}, \mathbf{x}_t), \quad (6.10)$$

$$\mathbf{y}_t = o(\mathbf{s}_t), \quad (6.11)$$

其中 $\mathbf{s}_t$ 为每个时刻的隐状态, $\mathbf{x}_t$ 是外部输入, $g(\cdot)$ 是可测的状态转换函数, $o(\cdot)$ 是连续输出函数, 并且对状态空间的紧致性没有限制.

图灵完备

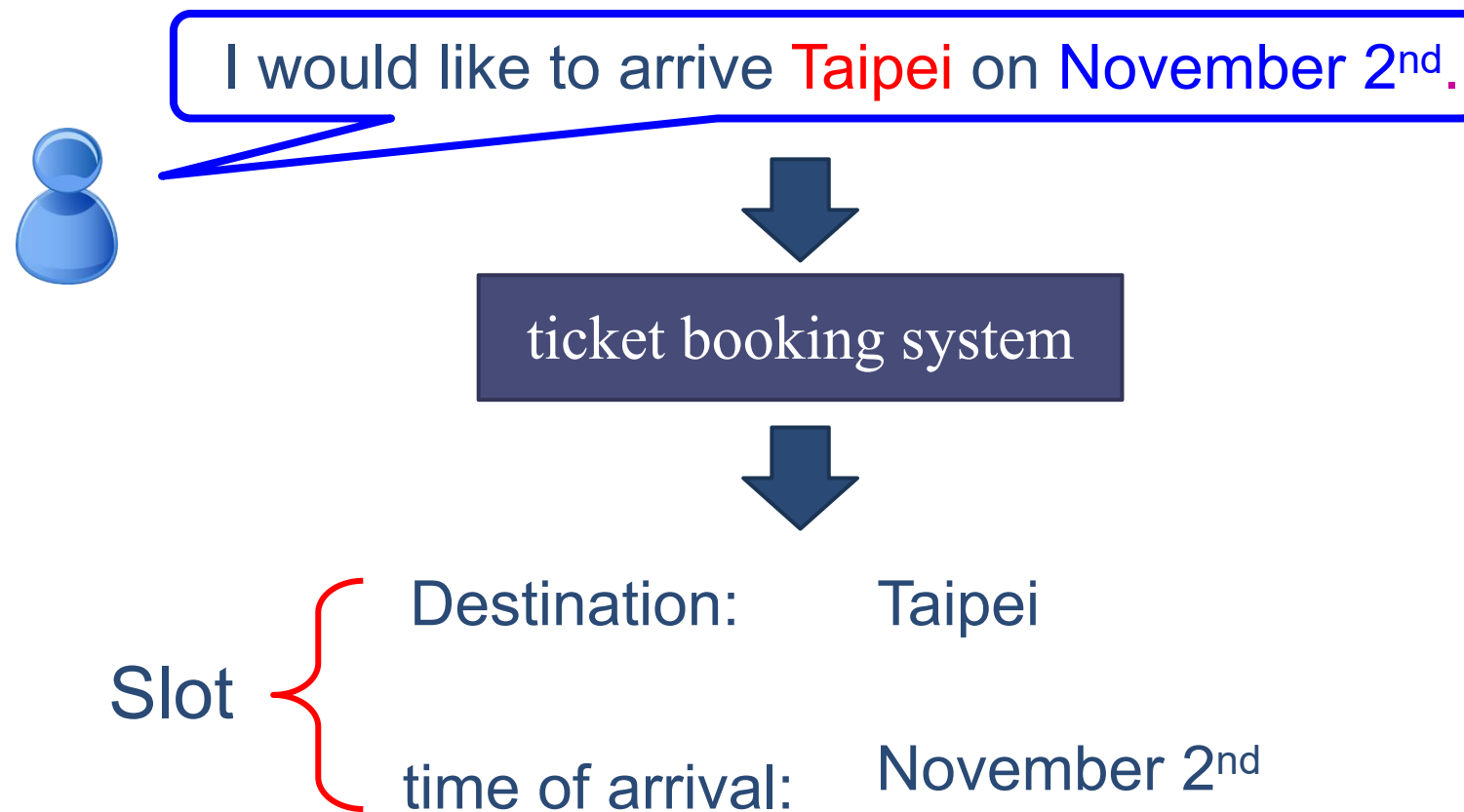
- ▶ 图灵完备 (Turing Completeness) 是指一种数据操作规则，比如一种计算机编程语言，可以实现图灵机的所有功能，解决所有的可计算问题。

定理 6.2 – 图灵完备 [Siegelmann et al., 1991]: 所有的图灵机都可以被一个由使用 Sigmoid 型激活函数的神经元构成的全连接循环网络来进行模拟。

- ▶ 一个完全连接的循环神经网络可以近似解决所有的可计算问题。

Example Application

► Slot Filling (填词)

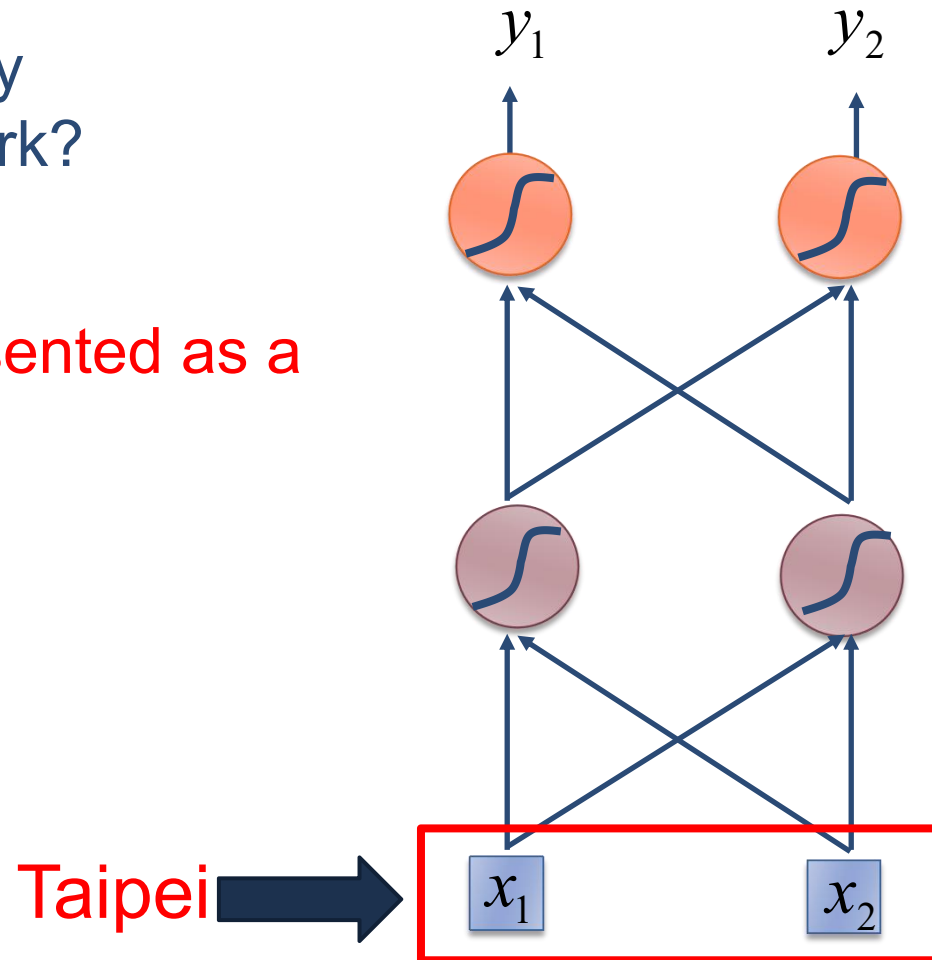


Example Application

Solving slot filling by
Feedforward network?

Input: a word

(Each word is represented as a
vector)



1-of-N encoding

How to represent each word as a vector?

将可能出现的词都写入一个字典，如 {apple, bag, cat, dog, elephant}，每个词对应一个维度，该词位置为 1，其余为 0，如 apple = [1,0,0,0,0]，bag = [0,1,0,0,0]，简单直观！

1-of-N Encoding lexicon = {apple, bag, cat, dog, elephant}

The vector is lexicon size.

Each dimension
corresponds to a word in
the lexicon

The dimension for the word
is 1, and others are 0

apple = [1 0 0 0 0]

bag = [0 1 0 0 0]

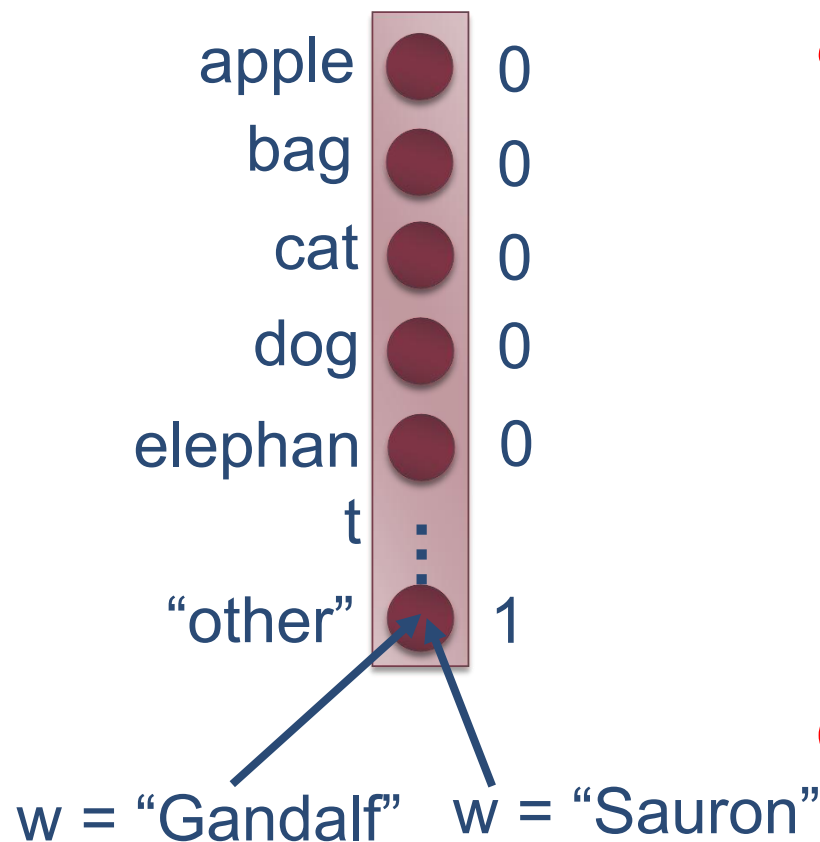
cat = [0 0 1 0 0]

dog = [0 0 0 1 0]

elephant = [0 0 0 0 1]

Beyond 1-of-N encoding

Dimension for “Other”

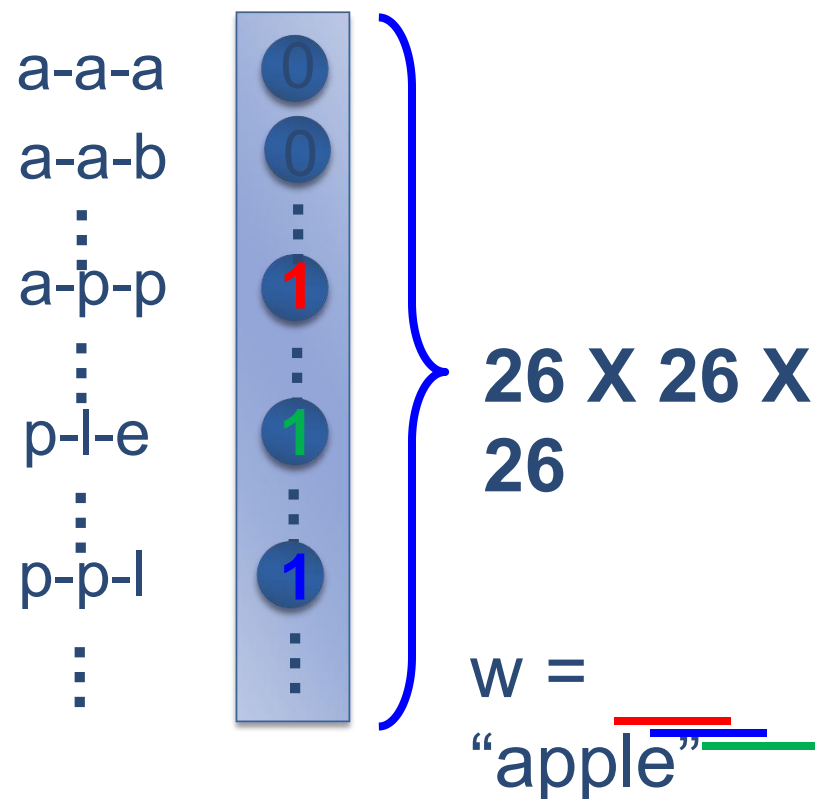


*One-Hot*编码虽然直观，但是词典大时向量极长，不适合作为模型的输入。通常用作损失函数的标签。

Dimension for “Other”: 在One-Hot基础上的改进，因为One-Hot 编码只能处理词典里见过的词，遇到词典里没有的词（未登录词，OOV）One-Hot 就没法编码了。所以给One-Hot增加一个维度，遇到词典外的词直接把 Other 维度设为 1，其余为 0，如 $\text{Gandalf} = [0,0,0,0,0,1]$

Word Hashing (词哈希)

Word hashing



One-Hot 编码词典越大，向量维度越高（比如词典有 10 万个词，One-Hot 就是 10 万维），计算量爆炸。**Word Hashing** 不是精确识别是哪个词，而是用字符片的特征来表征词，用字符级的小片段来描述单词，既解决维度爆炸，又能处理未登录词。通常作为模型输入使用。

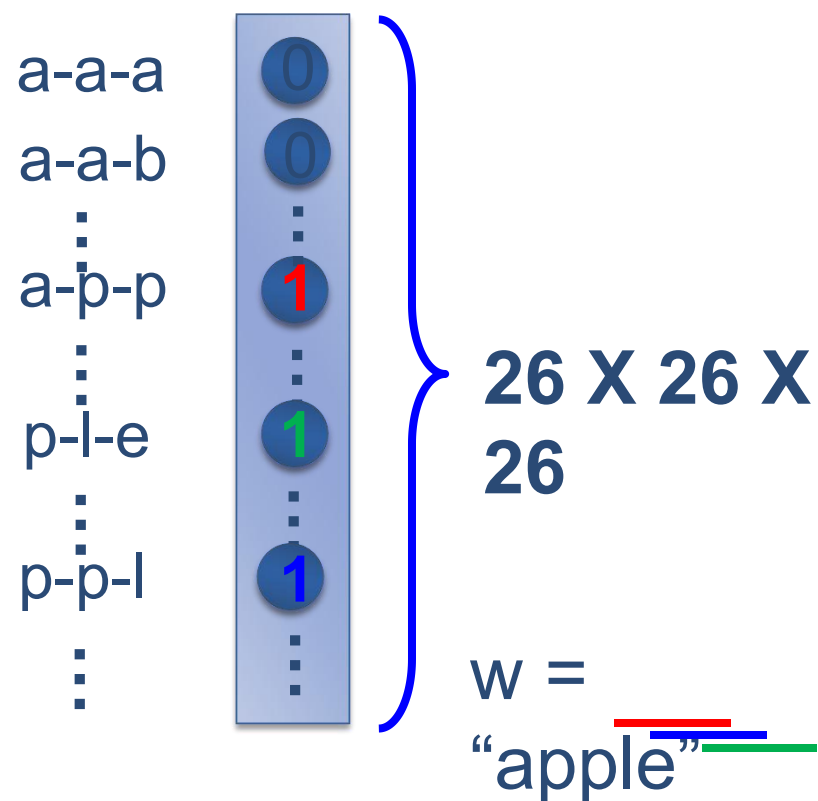
第一步，给单词加边界符，标记单词的开头和结尾。比如 `apple` → `<apple>`

第二步，把单词拆成字符级 n-gram（通常是 3-gram，即连续 3 个字符），例如 `<apple>` → `<ap, app, ppl, ple, le>`，`<Taipei>` → `<Ta, Tai, aip, ipe, pei, ei>`（边界符 `<` 和 `>` 也属于一个字符）

第三步，把这些 n-gram 映射到固定低维向量。Word Hashing 的 3-gram 词典是语料里实际见过的片段集合，而非所有字母组合的全集，所以实际上 Word Hashing 维度通常在几千维，完全可控。

Word Hashing (词哈希)

Word hashing



假设 3-gram 词典只有 8 个维度：[<ap, app, <Ta, Tai, <Ga, Gan, <ar, arr]，则：

<apple> 的向量：[1,1,0,0,0,0,0,0]（只包含 <ap, app>）；

<Taipei> 的向量：[0,0,1,1,0,0,0,0]（只包含 <Ta, Tai>）<arrive> 的向量：[0,0,0,0,0,0,1,1]（只包含 <ar, arr>）

所以模型不是认Taipei这个词，而是认字符片段，只要片段相似，模型就会认为单词的角色相似。

Example Application

Solving slot filling by
Feedforward network?

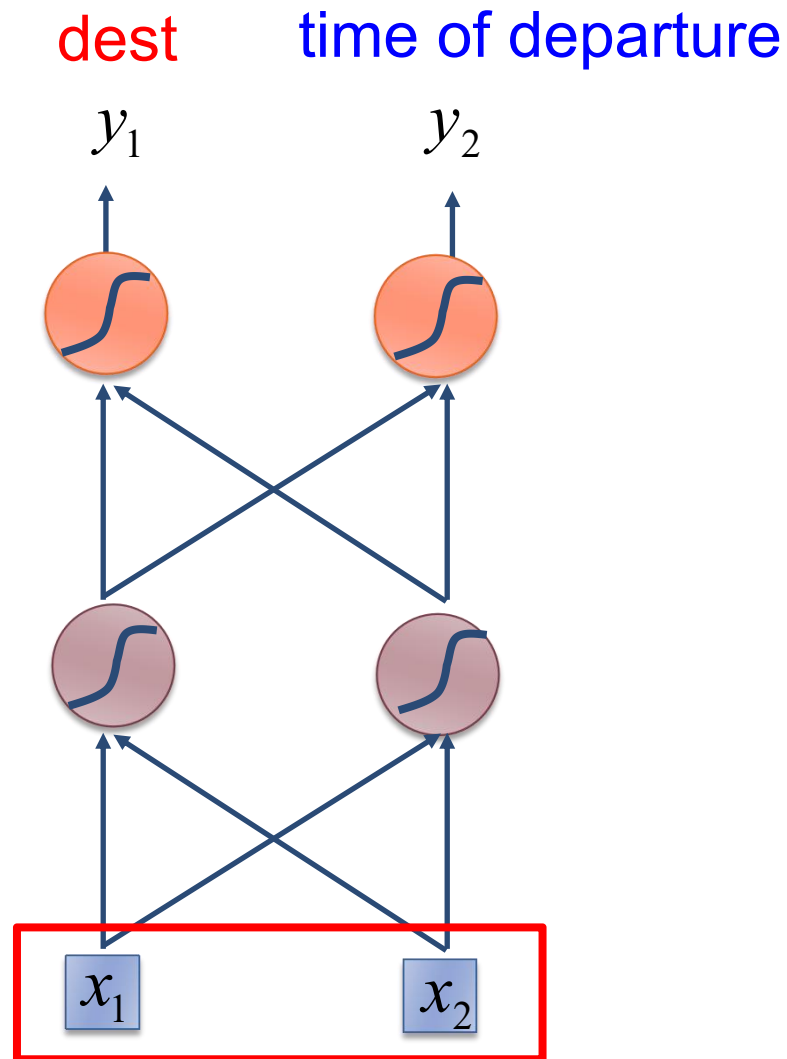
Input: a word

(Each word is represented as a
vector)

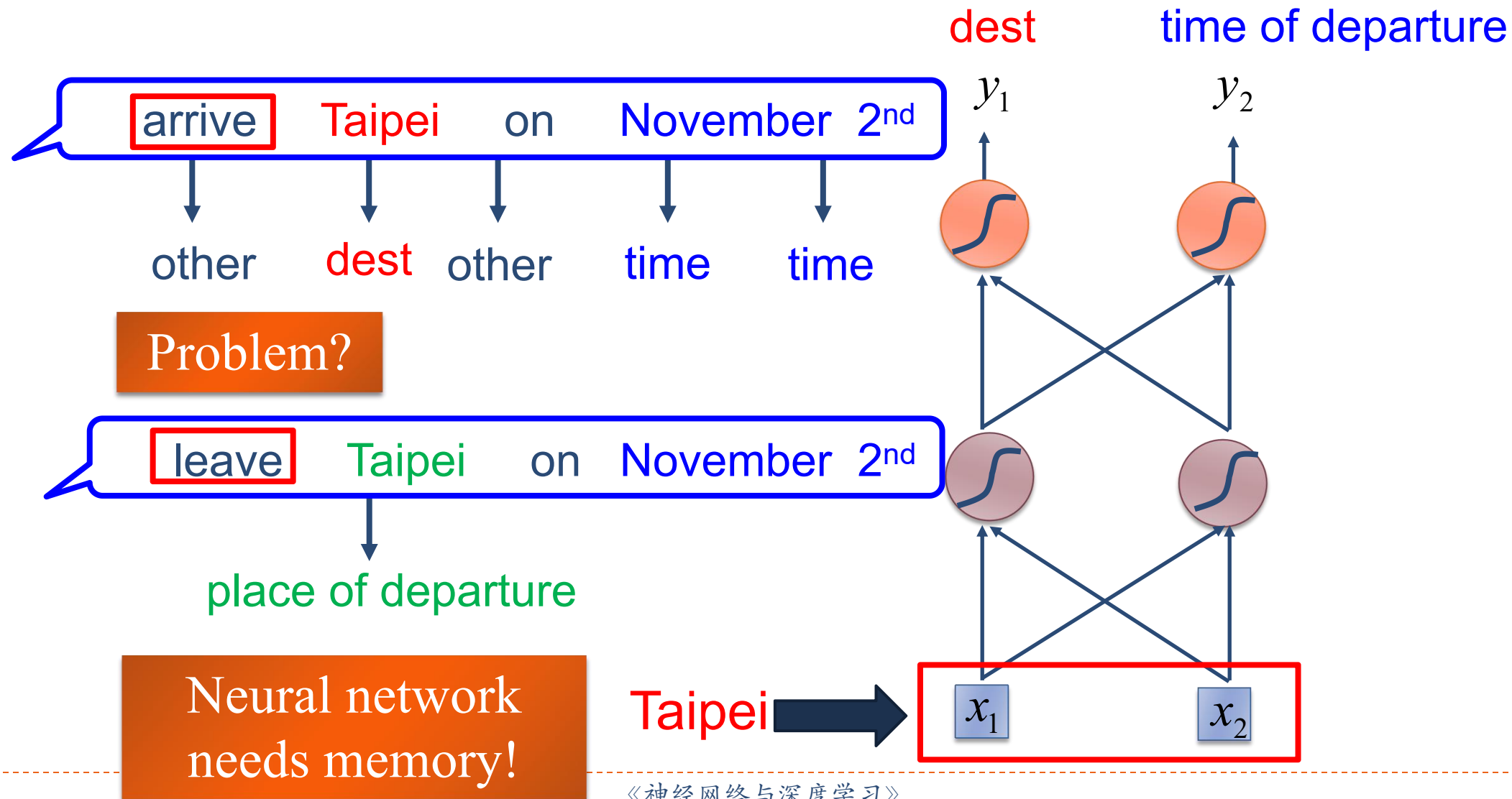
Output:

Probability distribution that
the input word belonging to
the slots

Taipei



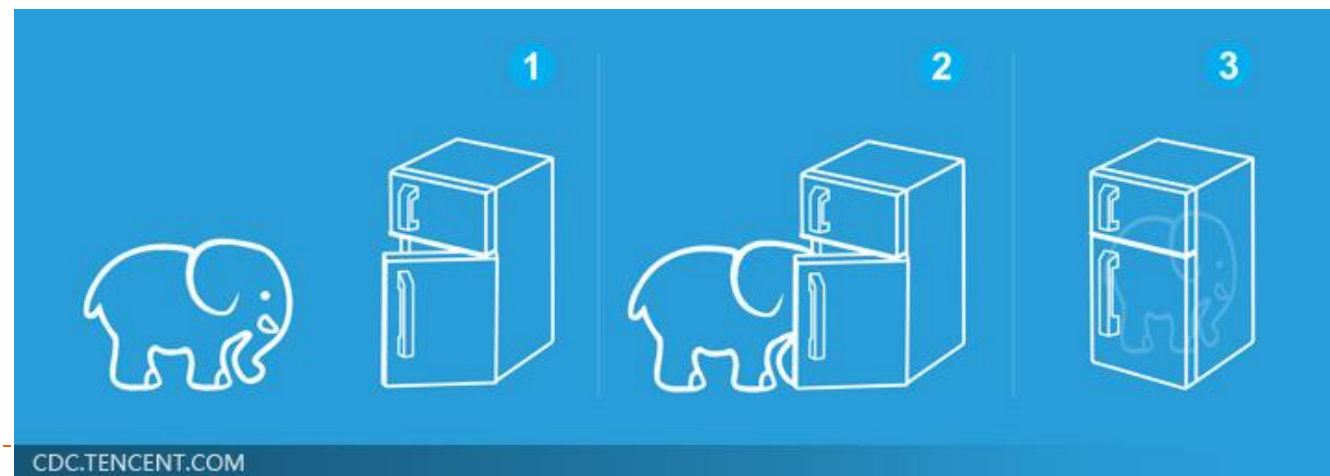
Example Application



Three Steps for Deep Learning

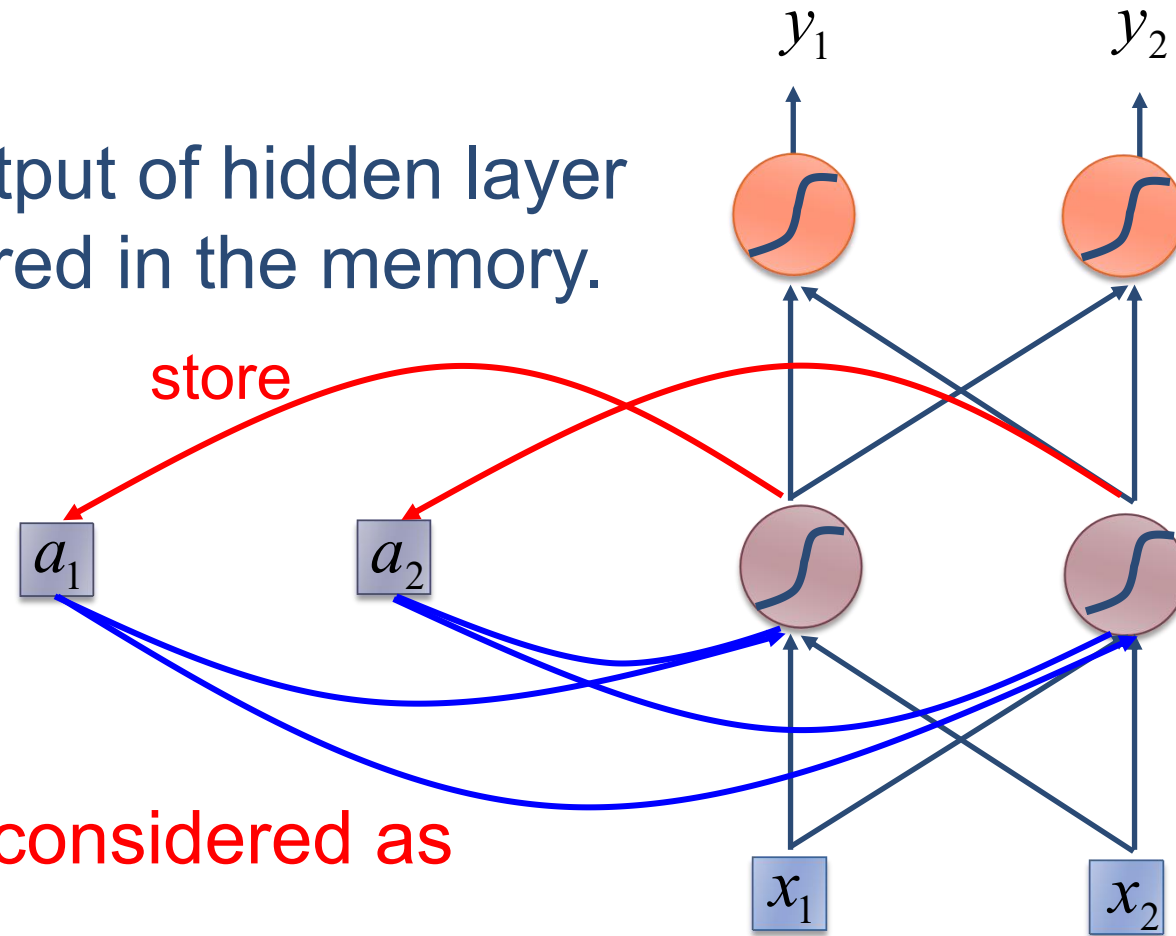


Deep Learning is so simple



Recurrent Neural Network (RNN)

The output of hidden layer are stored in the memory.



Memory can be considered as another input.

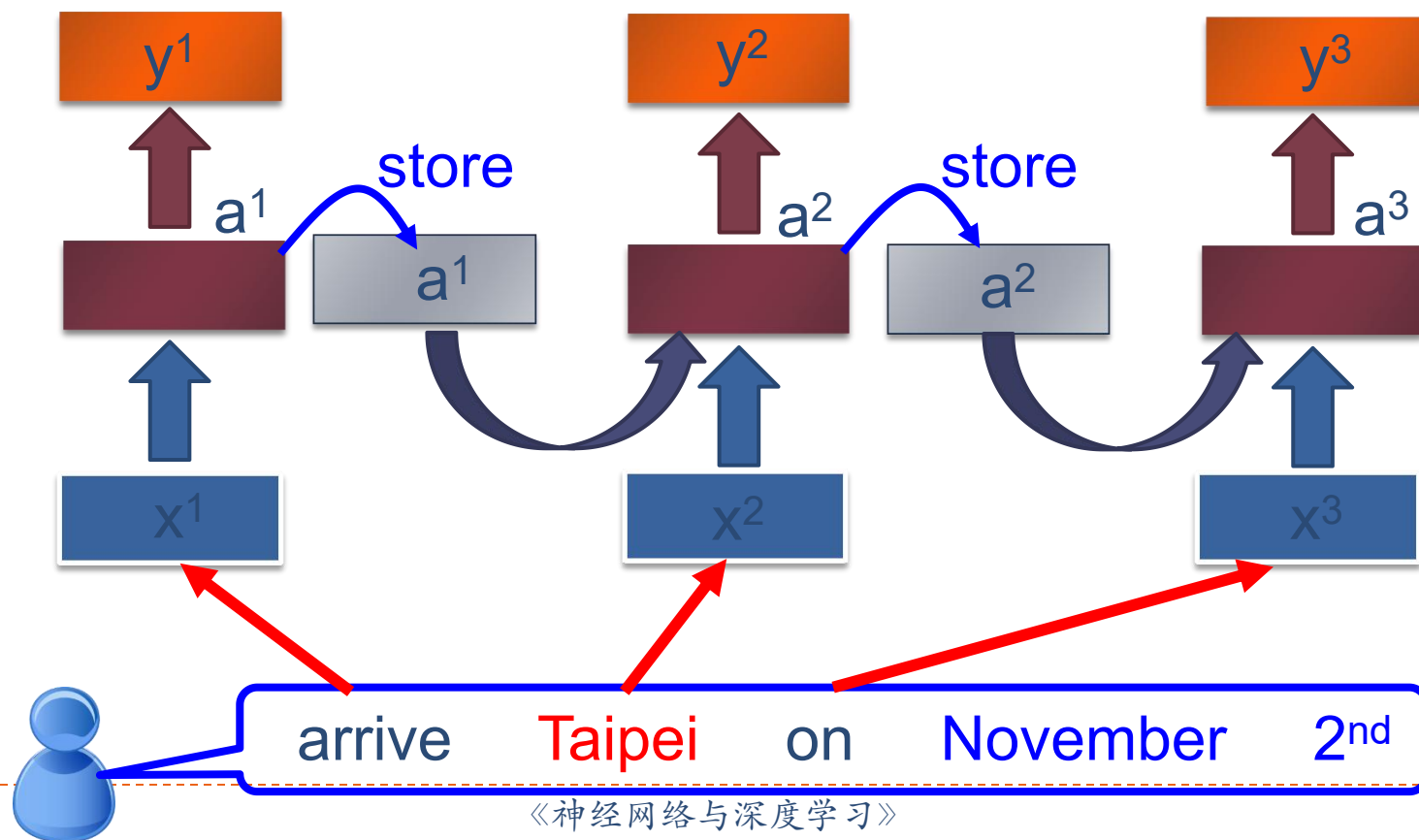
RNN

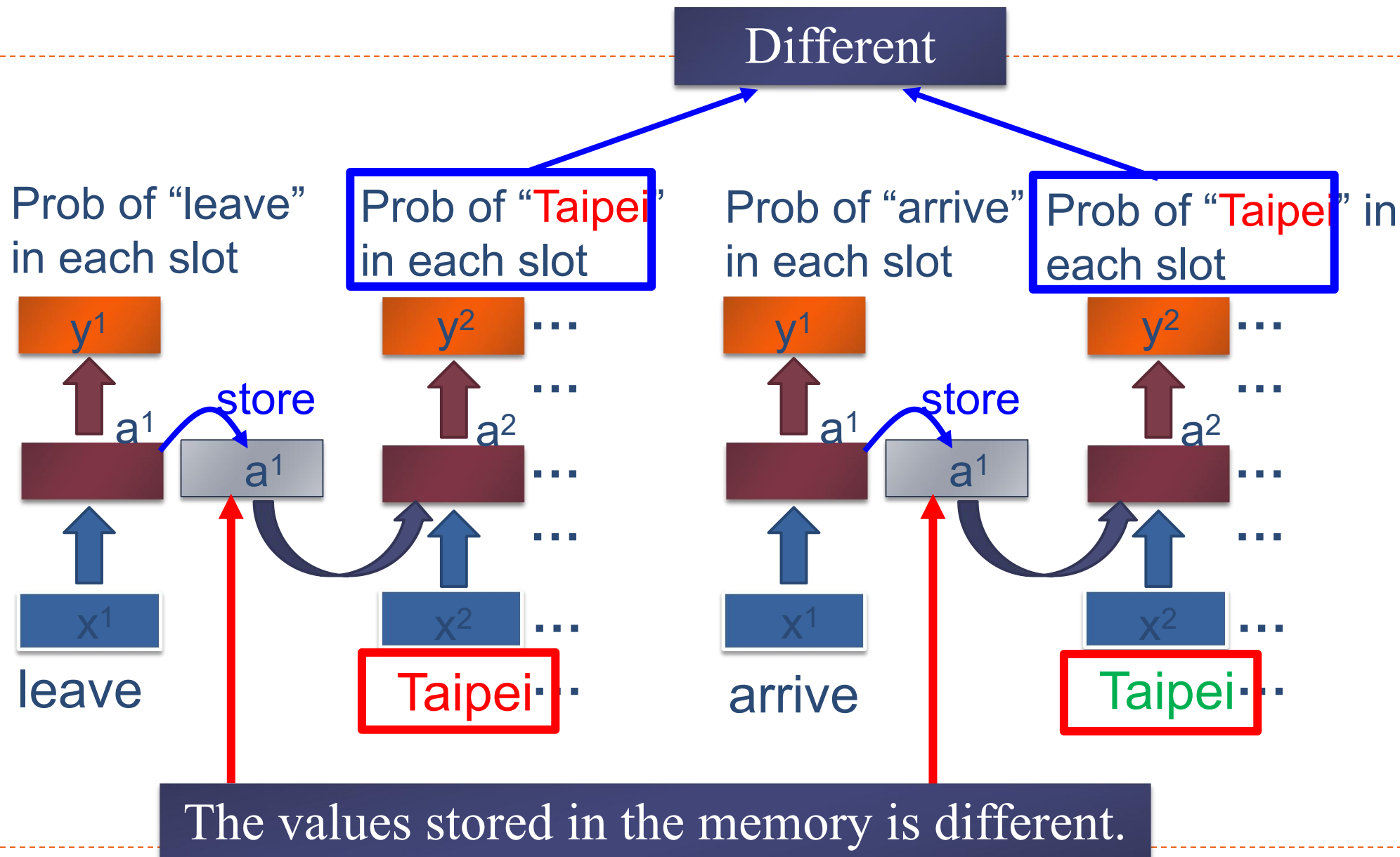
The same network is used again and again.

Probability of “arrive”
in each slot

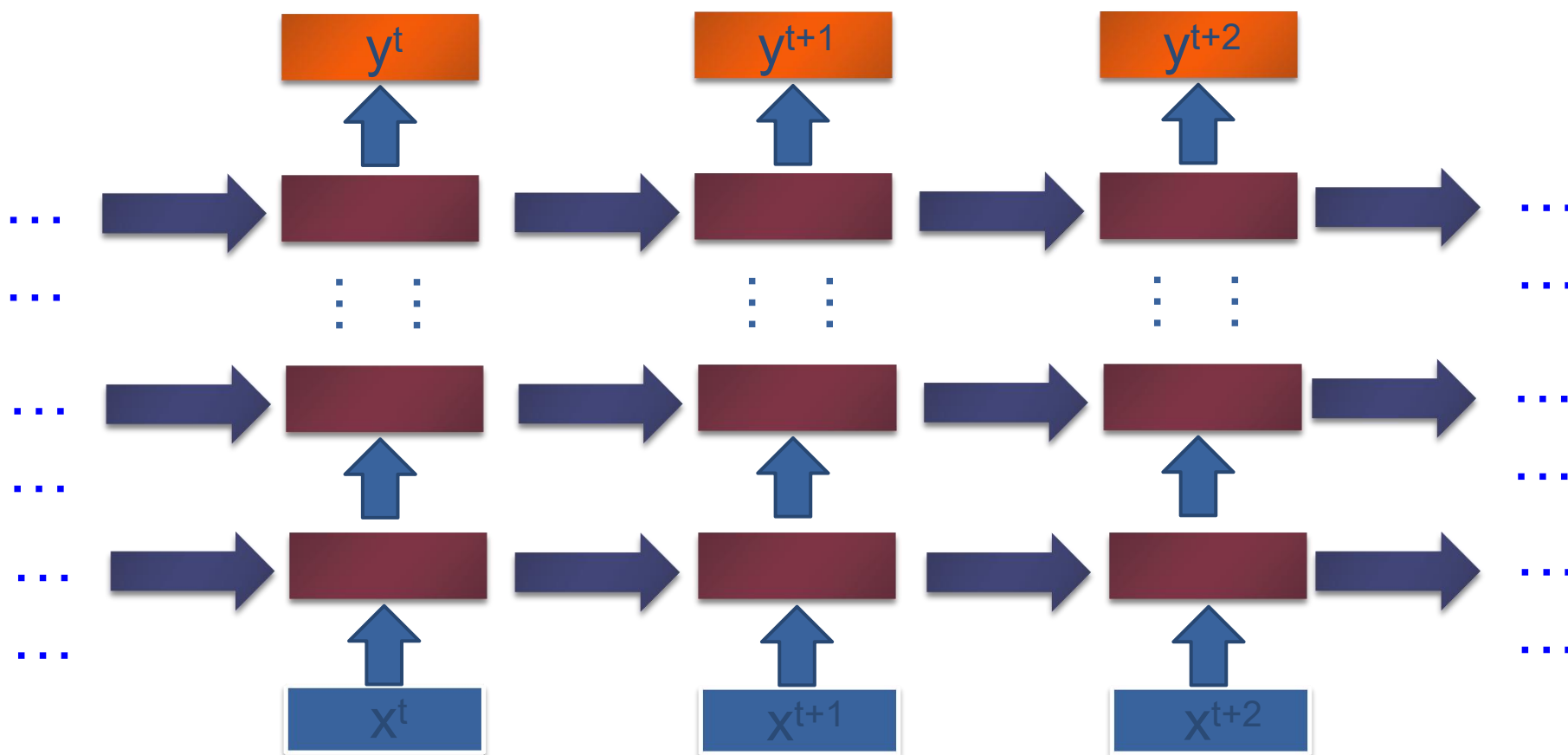
Probability of
“**Taipei**” in each slot

Probability of
“on” in each slot

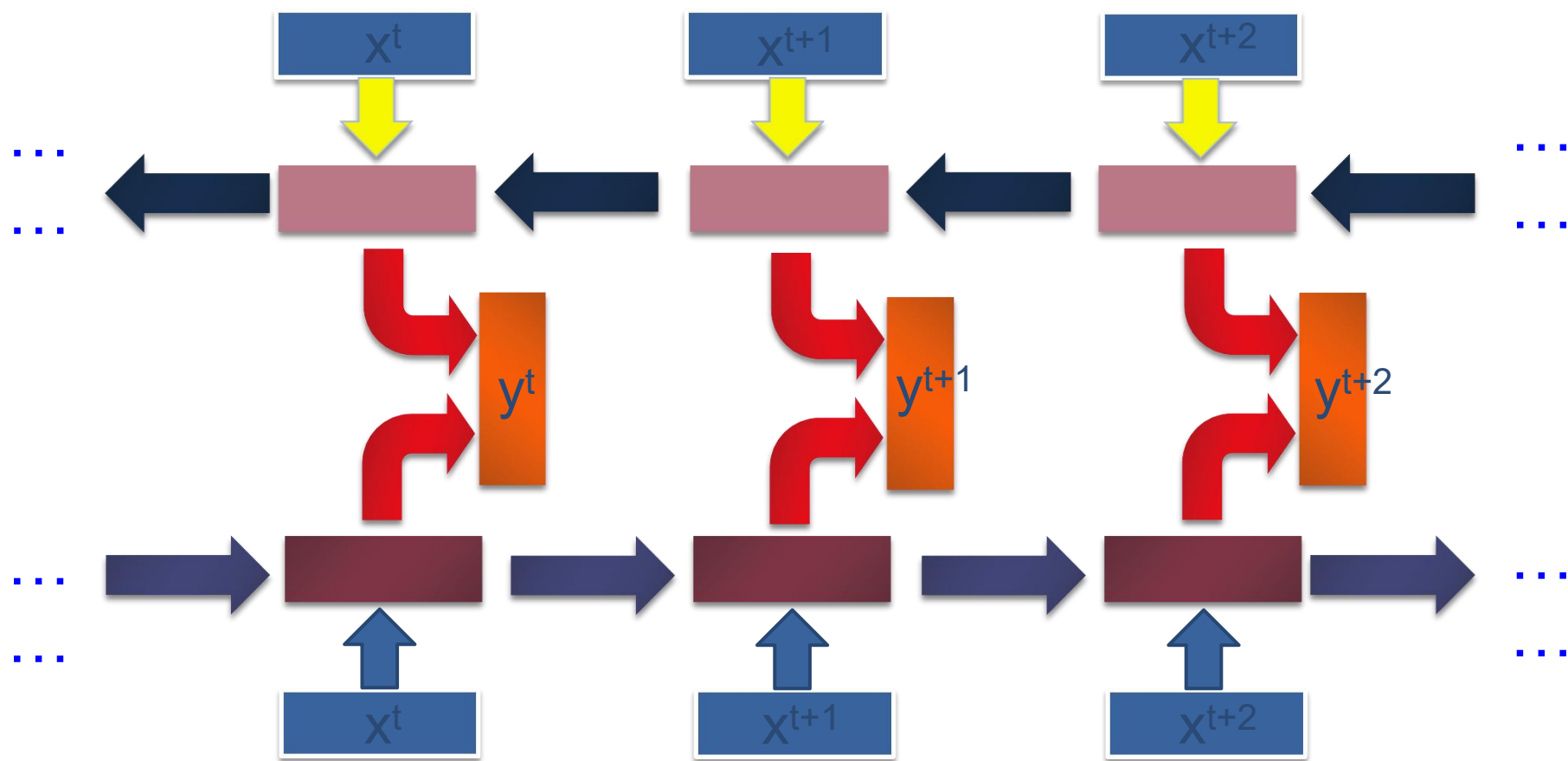




Of course it can be deep ...

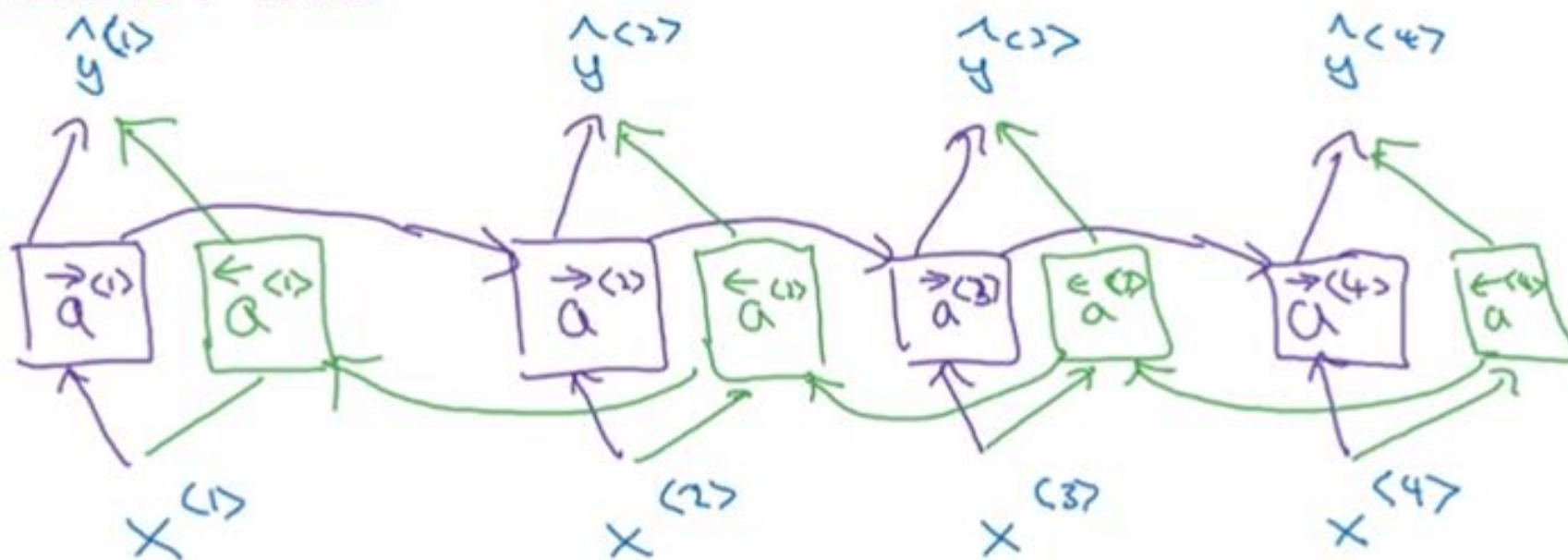


Bidirectional RNN



Bidirectional RNN (BRNN)

紫色部分, 原始RNN



绿色: 反向链接

<https://blog.csdn.net/ybdesire>

参数学习

▶ 机器学习

▶ 给定一个训练样本 $(\mathbf{x}, \mathbf{y})$, 其中

- ▶ $\mathbf{x} = (x_1, \dots, x_T)$ 为长度是 T 的输入序列,
- ▶ $\mathbf{y} = (y_1, \dots, y_T)$ 是长度为 T 的标签序列。

▶ 时刻 t 的瞬时损失函数为

$$\mathcal{L}_t = \mathcal{L}(\mathbf{y}_t, g(\mathbf{h}_t)),$$

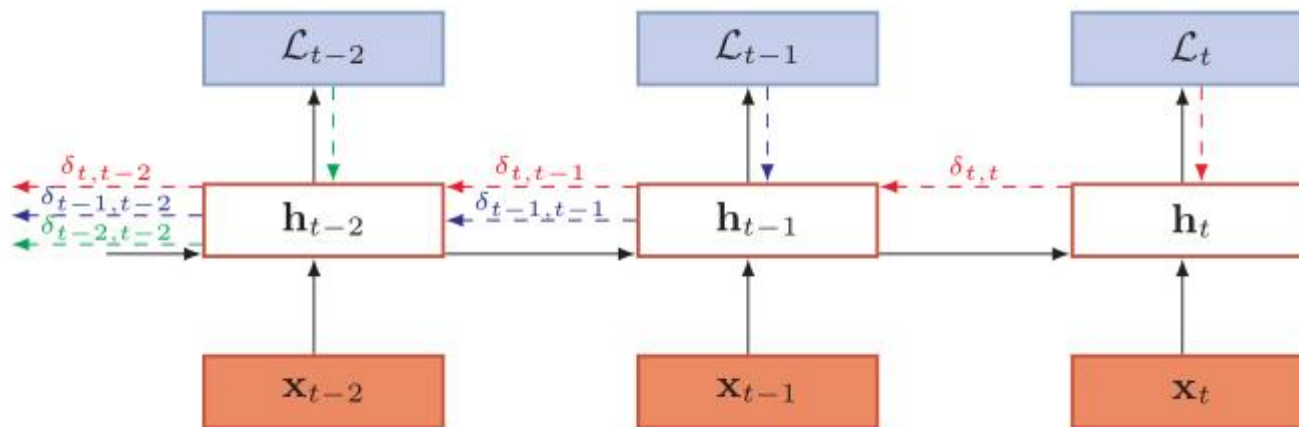
▶ 总损失函数

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t.$$

梯度

▶ 随时间反向传播算法

$$\mathbf{h}_{t+1} = f(\mathbf{z}_{t+1}) = f(U\mathbf{h}_t + W\mathbf{x}_{t+1} + \mathbf{b})$$



$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\text{diag}(f'(\mathbf{z}_{\tau})) U^T \right) \delta_{t,t}$$

$\delta_{t,k}$ 为第 t 时刻的损失对第 k 步隐藏神经元的净输入 $\mathbf{z}_k$ 的导数

梯度消失/爆炸

► 梯度

$$\frac{\partial \mathcal{L}}{\partial U} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{h}_{k-1}^T$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k} \mathbf{x}_k^T,$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_{t=1}^T \sum_{k=1}^t \delta_{t,k}.$$

► 其中

$$\delta_{t,k} = \prod_{\tau=k}^{t-1} \left(\frac{\text{diag}(f'(\mathbf{z}_\tau)) U^T}{\lambda} \right) \delta_{t,t}$$

由于梯度爆炸或消失问题，实际上只能学习到短周期的依赖关系。这就是所谓的长程依赖问题。

长程依赖问题

- ▶ 循环神经网络在时间维度上非常深!

- ▶ 梯度消失或梯度爆炸

- ▶ 如何改进?

- ▶ 梯度爆炸问题

- ▶ 权重衰减

- ▶ 梯度截断

- ▶ 梯度消失问题

- ▶ 改进模型

长程依赖问题

▶ 改进方法

▶ 循环边改为线性依赖关系

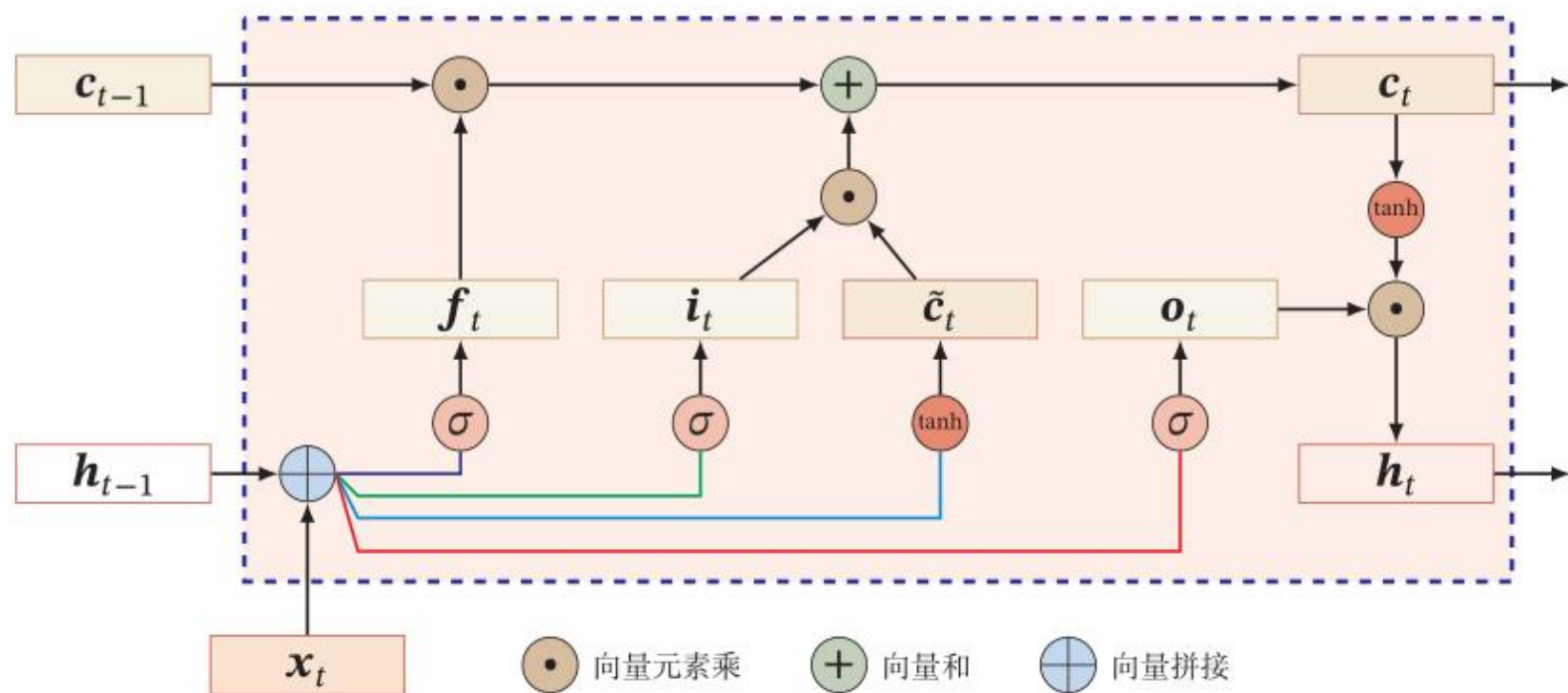
$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta),$$

▶ 增加非线性

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta),$$

残差网络?

长短期记忆神经网络 (Long Short-Term Memory, LSTM)



输入门

$$\mathbf{i}_t = \sigma(W_i \mathbf{x}_t + U_i \mathbf{h}_{t-1} + \mathbf{b}_i),$$

$$\tilde{\mathbf{c}}_t = \tanh(W_c \mathbf{x}_t + U_c \mathbf{h}_{t-1} + \mathbf{b}_c) \quad \text{后选状态}$$

遗忘门

$$\mathbf{f}_t = \sigma(W_f \mathbf{x}_t + U_f \mathbf{h}_{t-1} + \mathbf{b}_f),$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad \text{当前时刻内部状态}$$

输出门

$$\mathbf{o}_t = \sigma(W_o \mathbf{x}_t + U_o \mathbf{h}_{t-1} + \mathbf{b}_o),$$

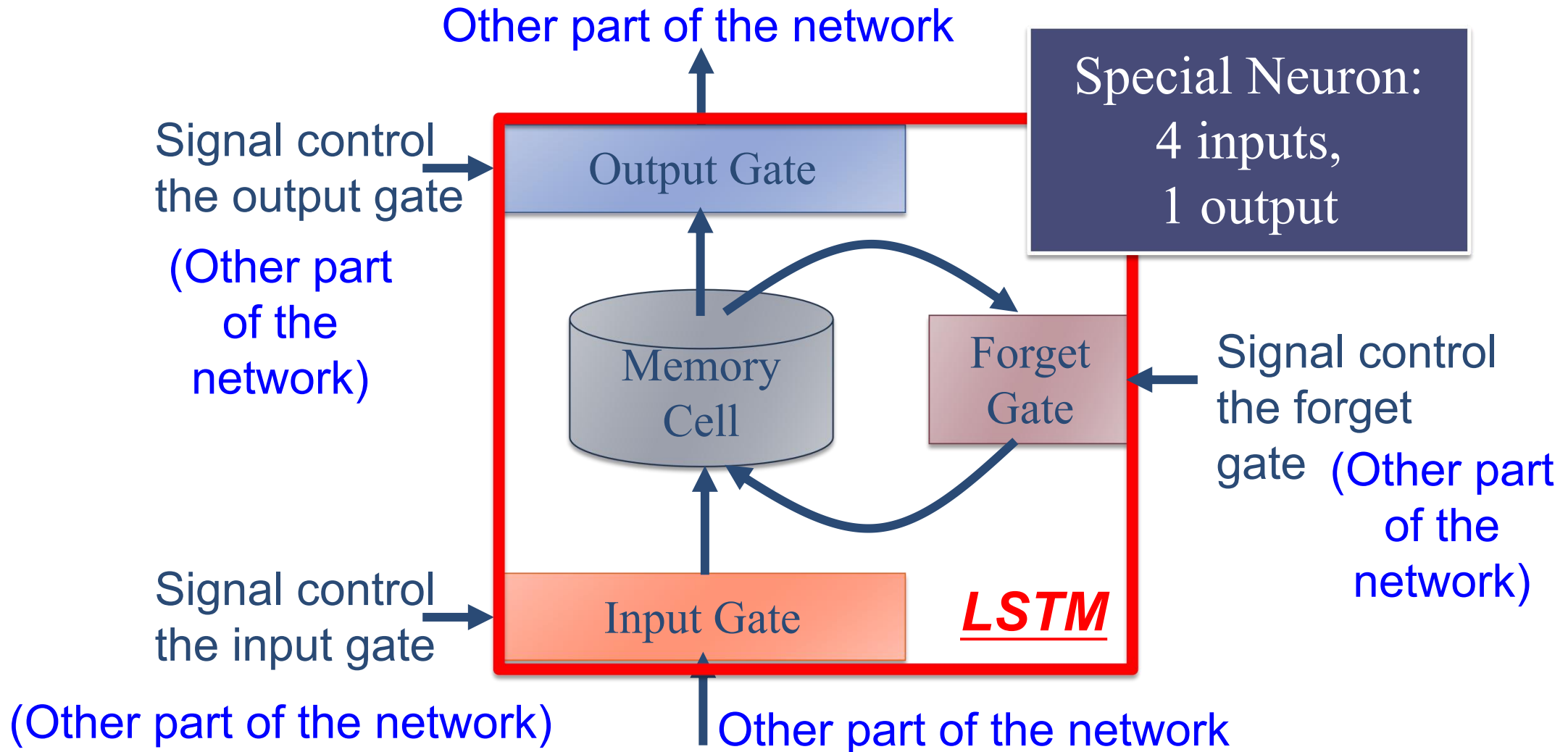
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad \text{当前时刻外部状态}$$

长短期记忆神经网络 (Long Short-Term Memory, LSTM)

LSTM 网络引入门控机制 (Gating Mechanism) 来控制信息传递的路径. 公式(6.51)和公式(6.52)中三个“门”分别为输入门 $\mathbf{i}_t$ 、遗忘门 $\mathbf{f}_t$ 和输出门 $\mathbf{o}_t$. 这三个门的作用为

- (1) 遗忘门 $\mathbf{f}_t$ 控制上一个时刻的内部状态 $\mathbf{c}_{t-1}$ 需要遗忘多少信息.
- (2) 输入门 $\mathbf{i}_t$ 控制当前时刻的候选状态 $\tilde{\mathbf{c}}_t$ 有多少信息需要保存.
- (3) 输出门 $\mathbf{o}_t$ 控制当前时刻的内部状态 $\mathbf{c}_t$ 有多少信息需要输出给外部状态 $\mathbf{h}_t$.

Long Short-term Memory (LSTM)



Long Short-term Memory (LSTM)

Special Neuron:
4 inputs,
1 output

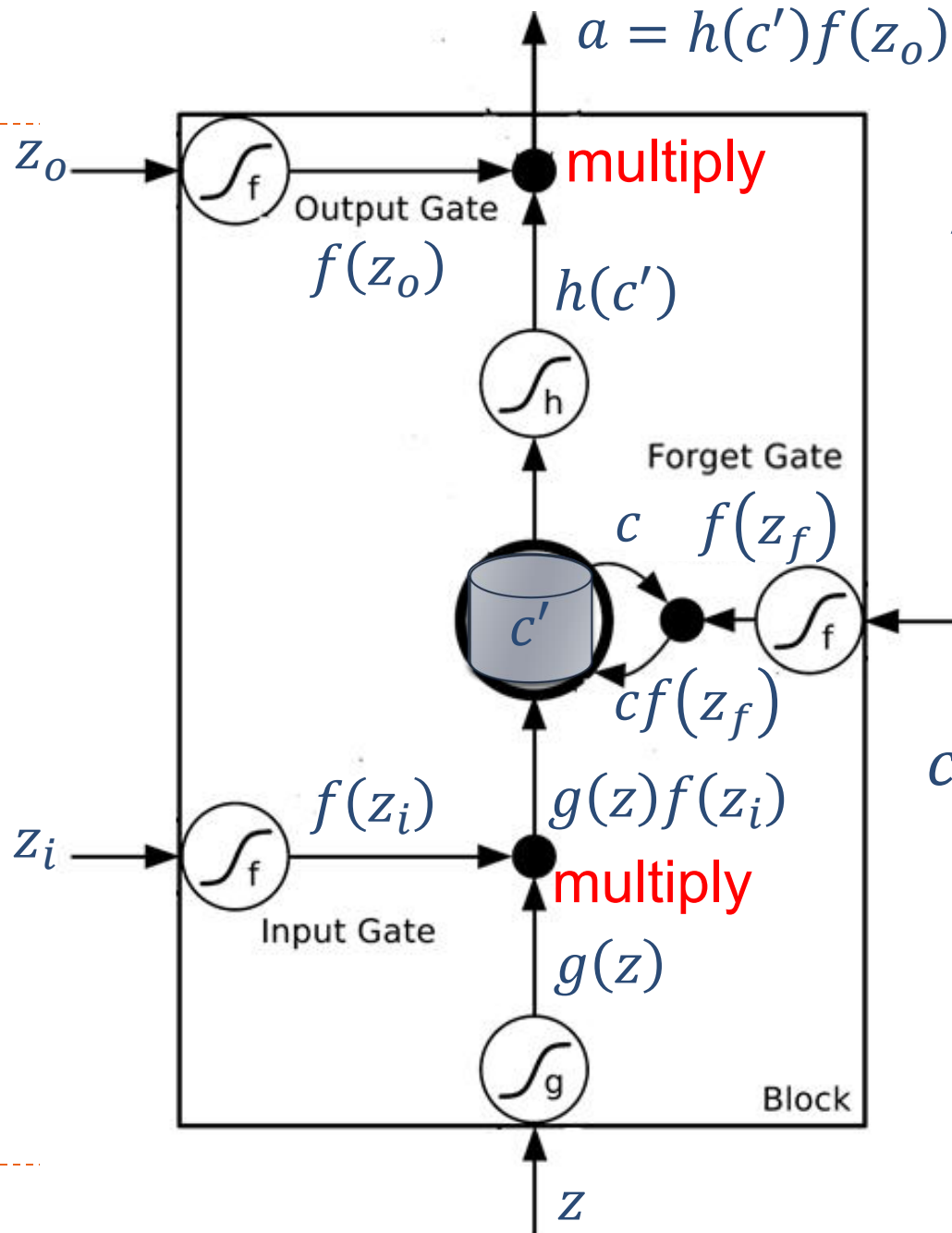
遗忘门控制信号

输入门控制信号

输出门控制信号

候选状态控制信号

外部状态

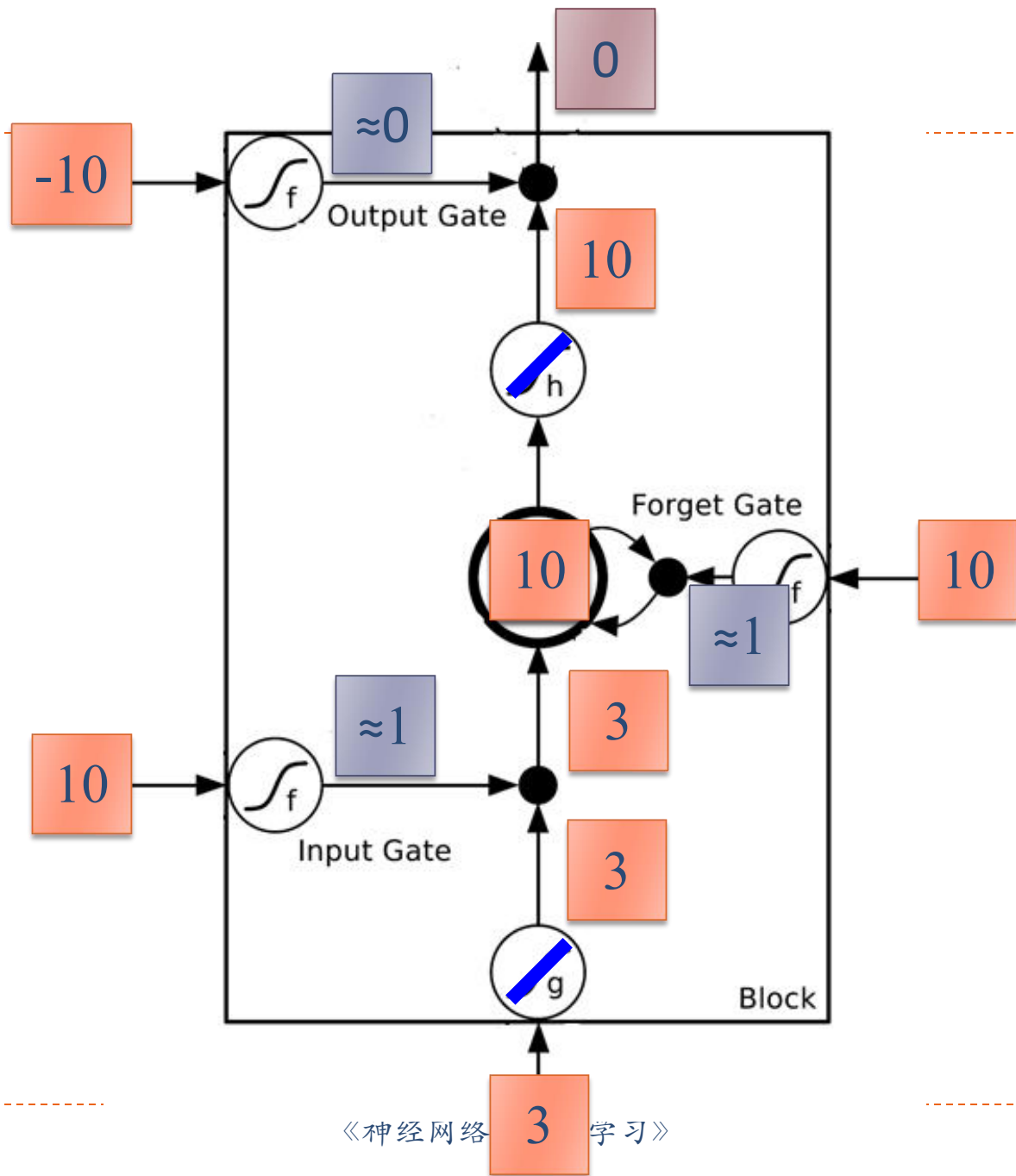


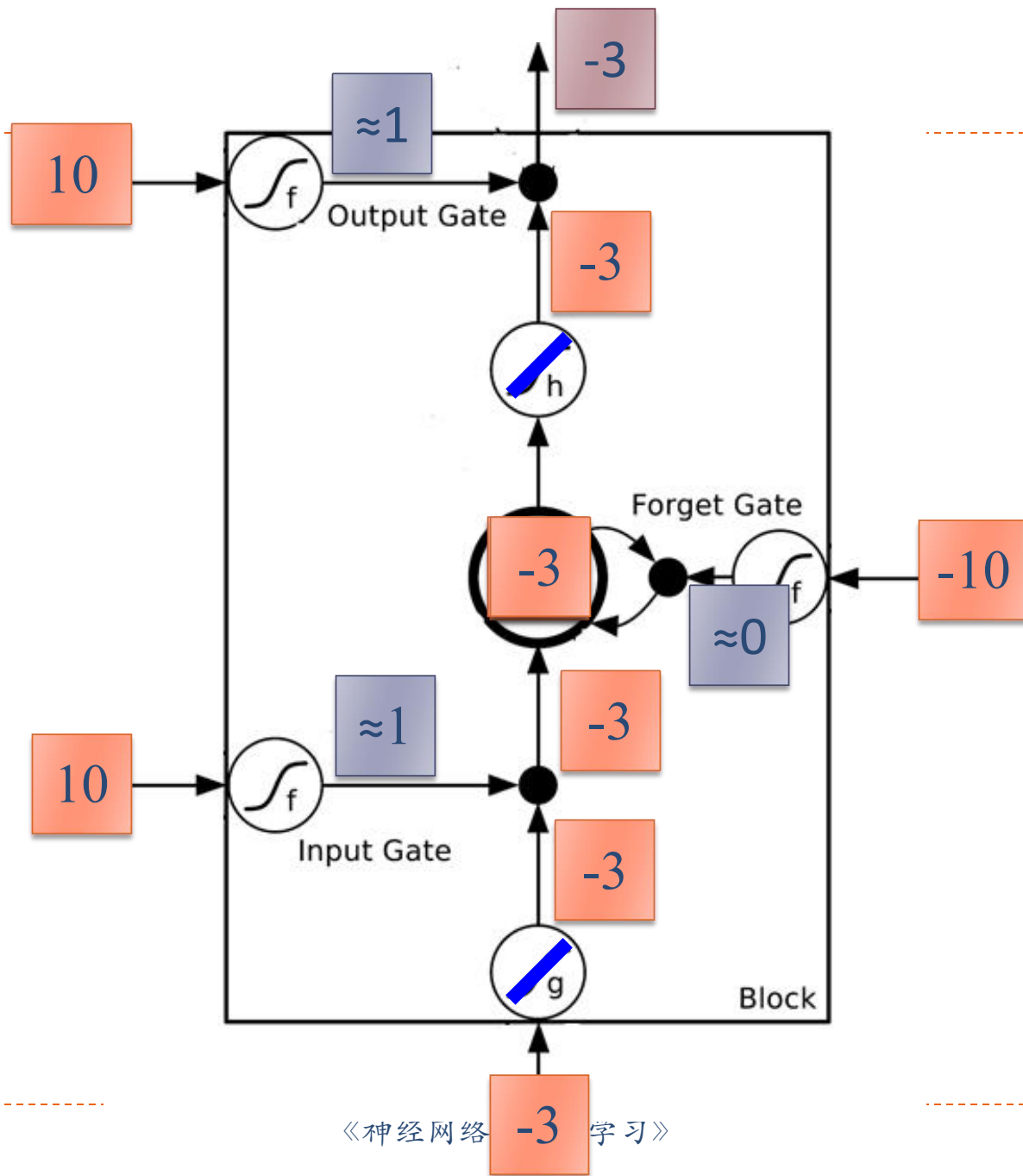
Activation function f is usually a sigmoid function

Between 0 and 1

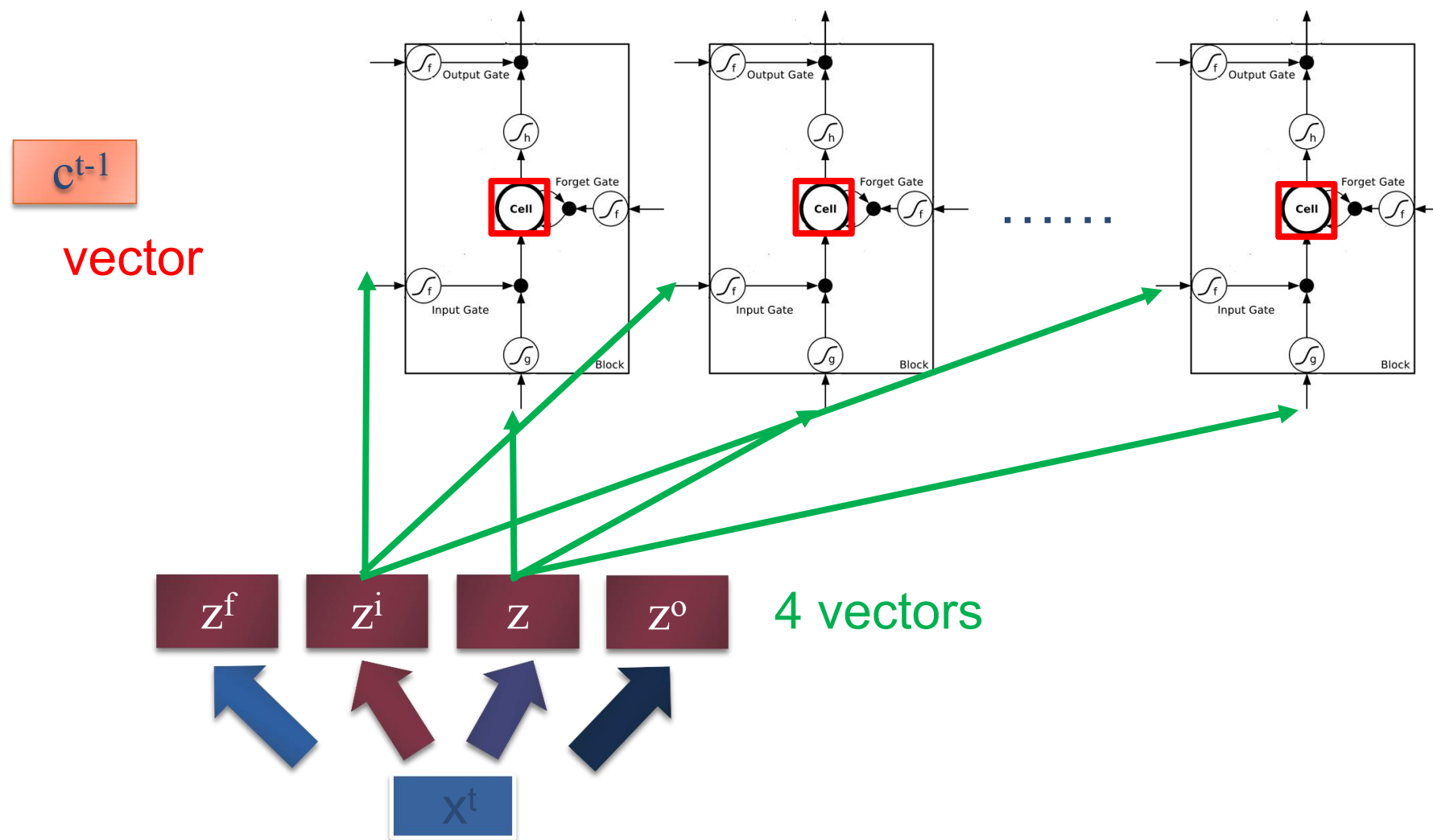
Mimic open and close gate

$$c' = g(z)f(z_i) + cf(z_f)$$

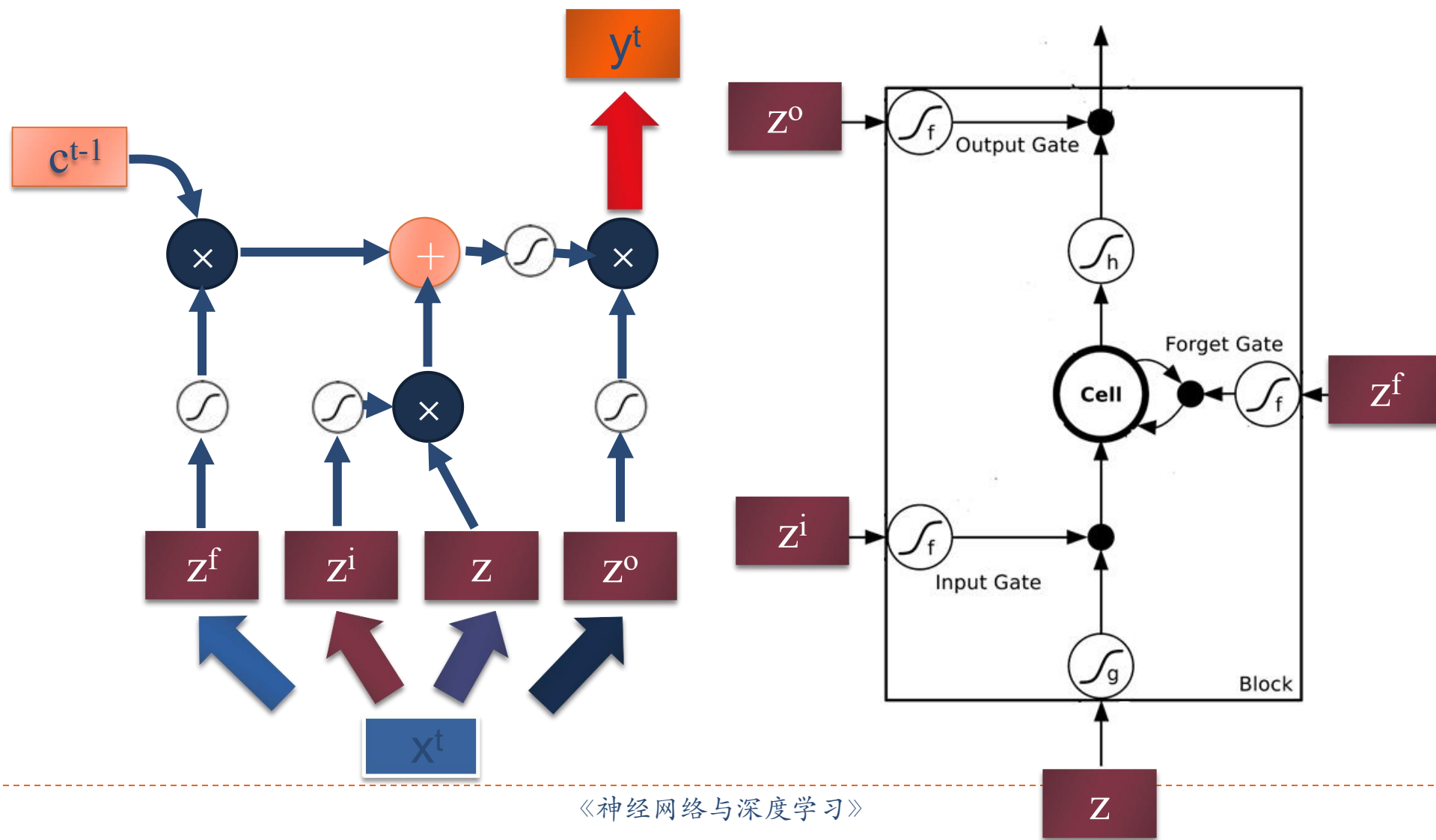




LSTM

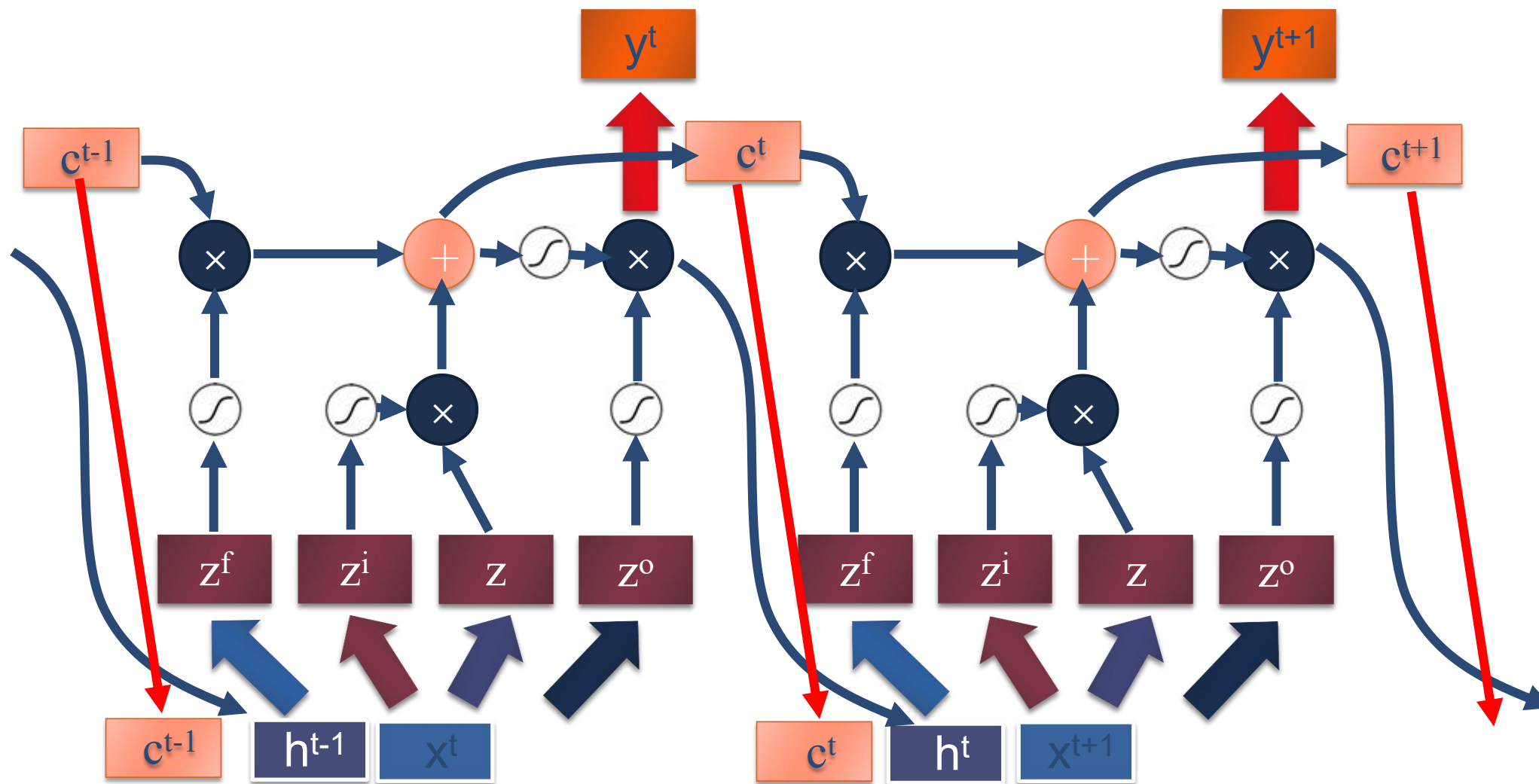


LSTM



LSTM

Extension: “peephole”



Multiple-layer LSTM

Don't worry if you cannot understand this.
Keras can handle it.

Keras supports
“LSTM”, “GRU”, “SimpleRNN” layers

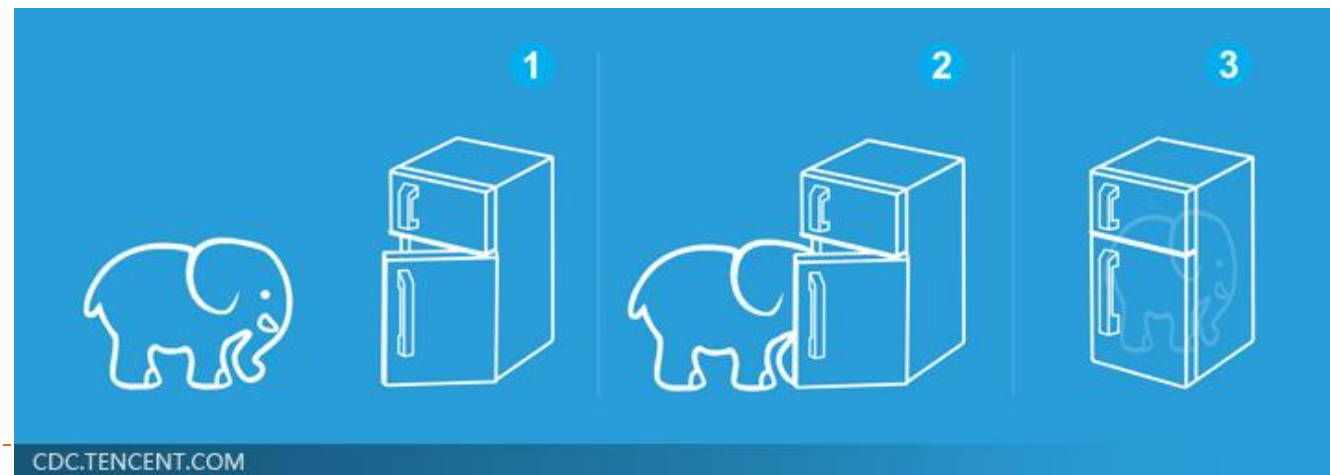
This is quite standard now.

我到底看了什麼？

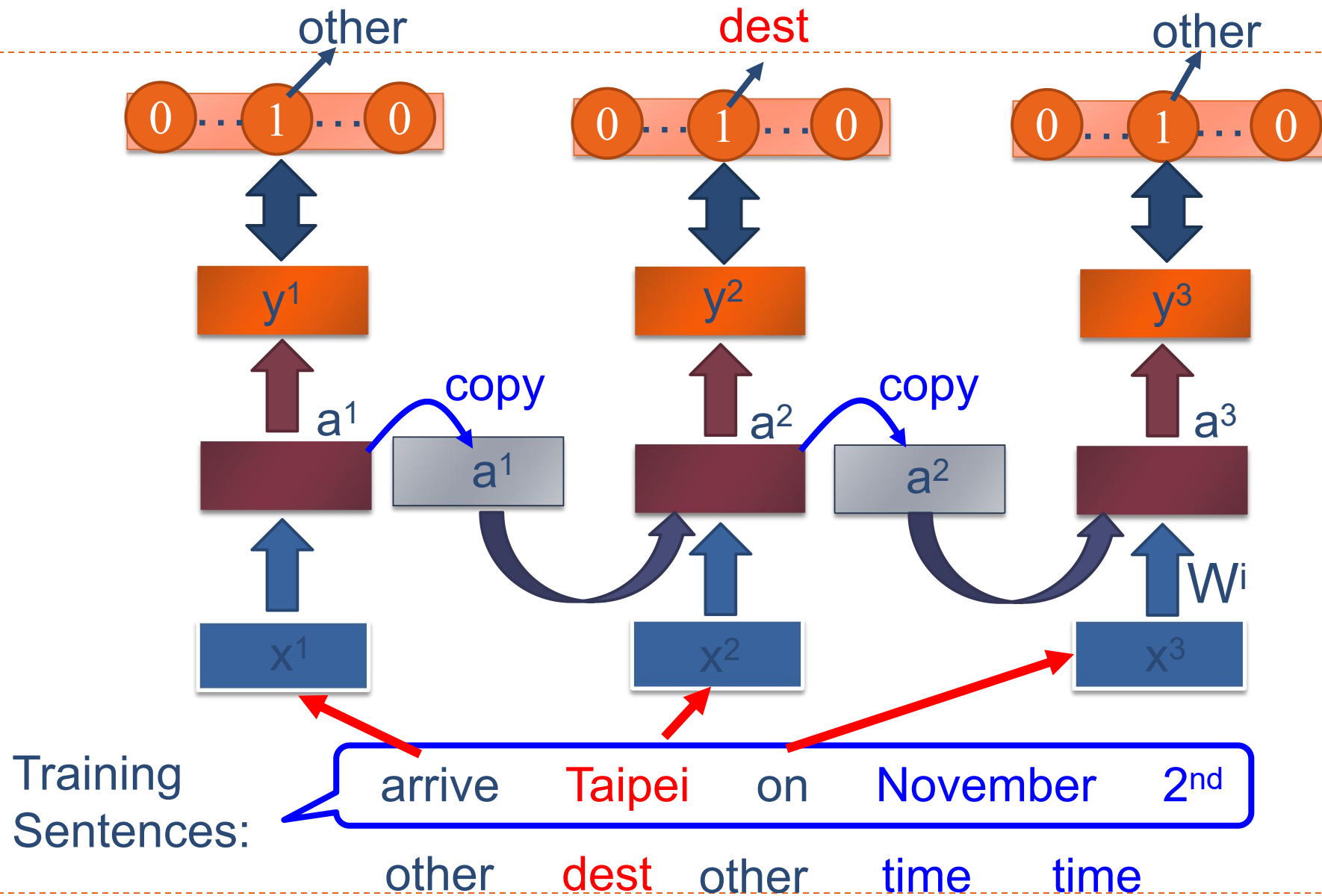
Three Steps for Deep Learning



Deep Learning is so simple



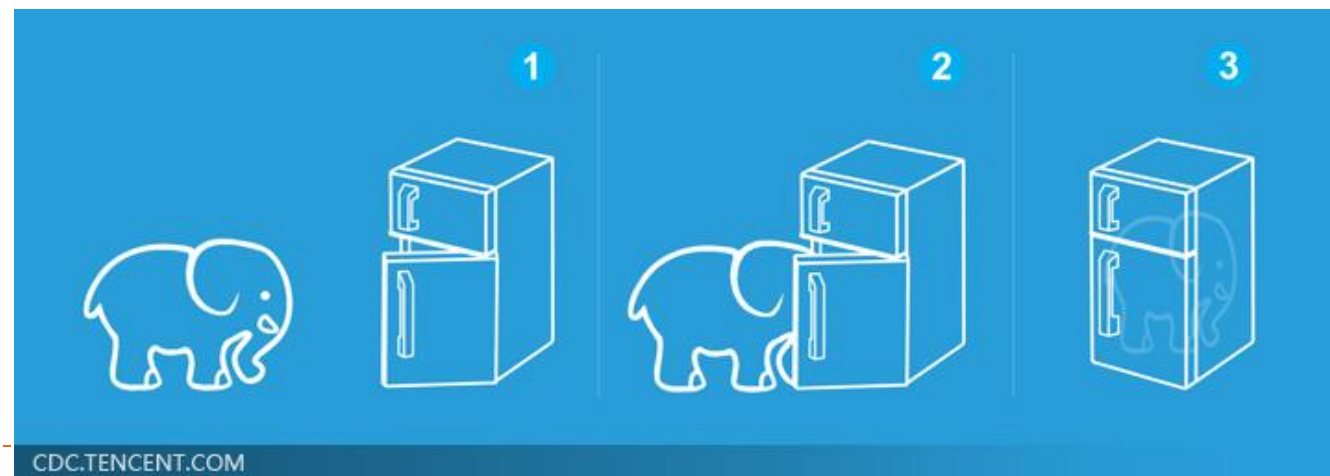
Learning Target



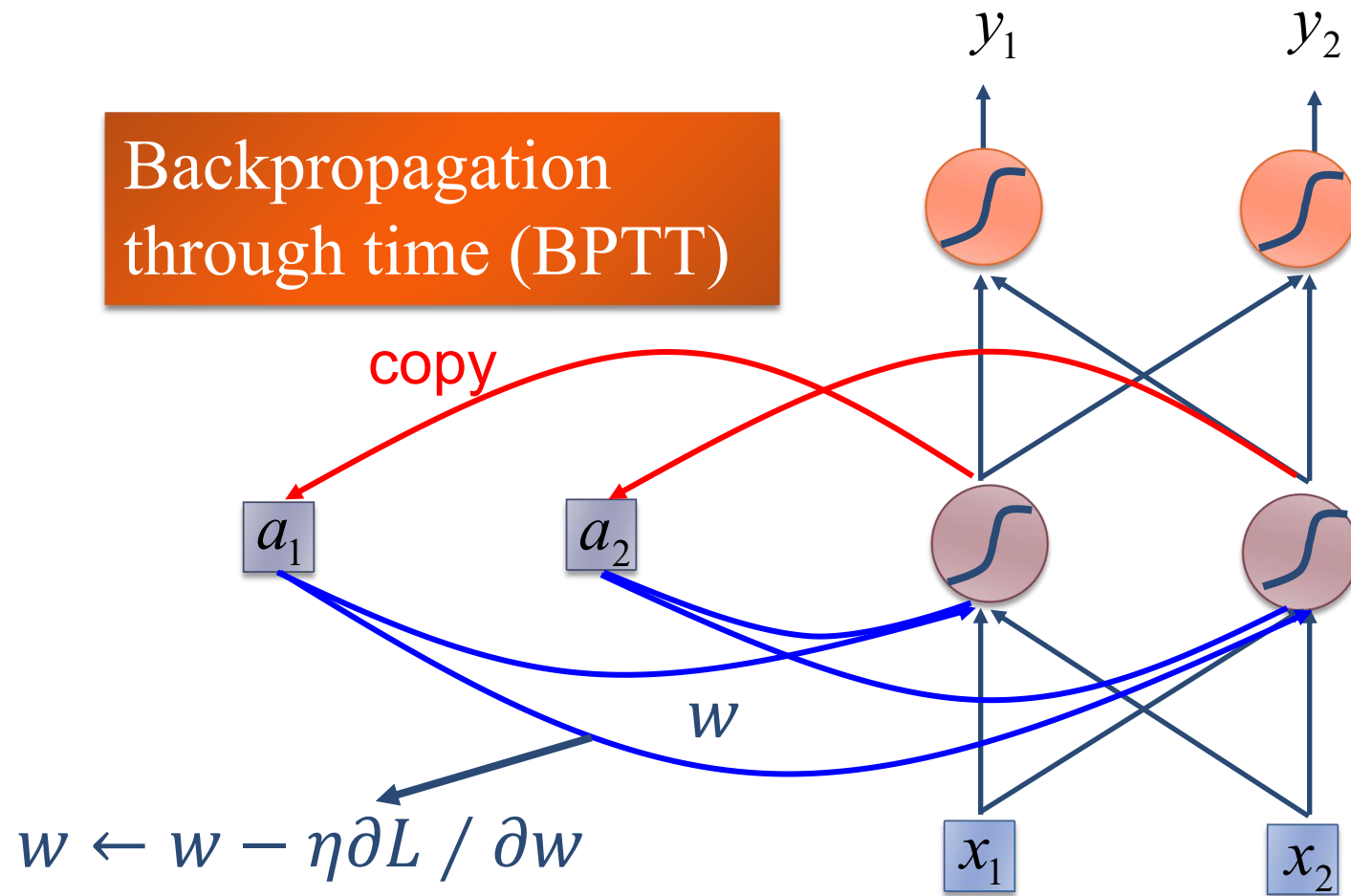
Three Steps for Deep Learning



Deep Learning is so simple



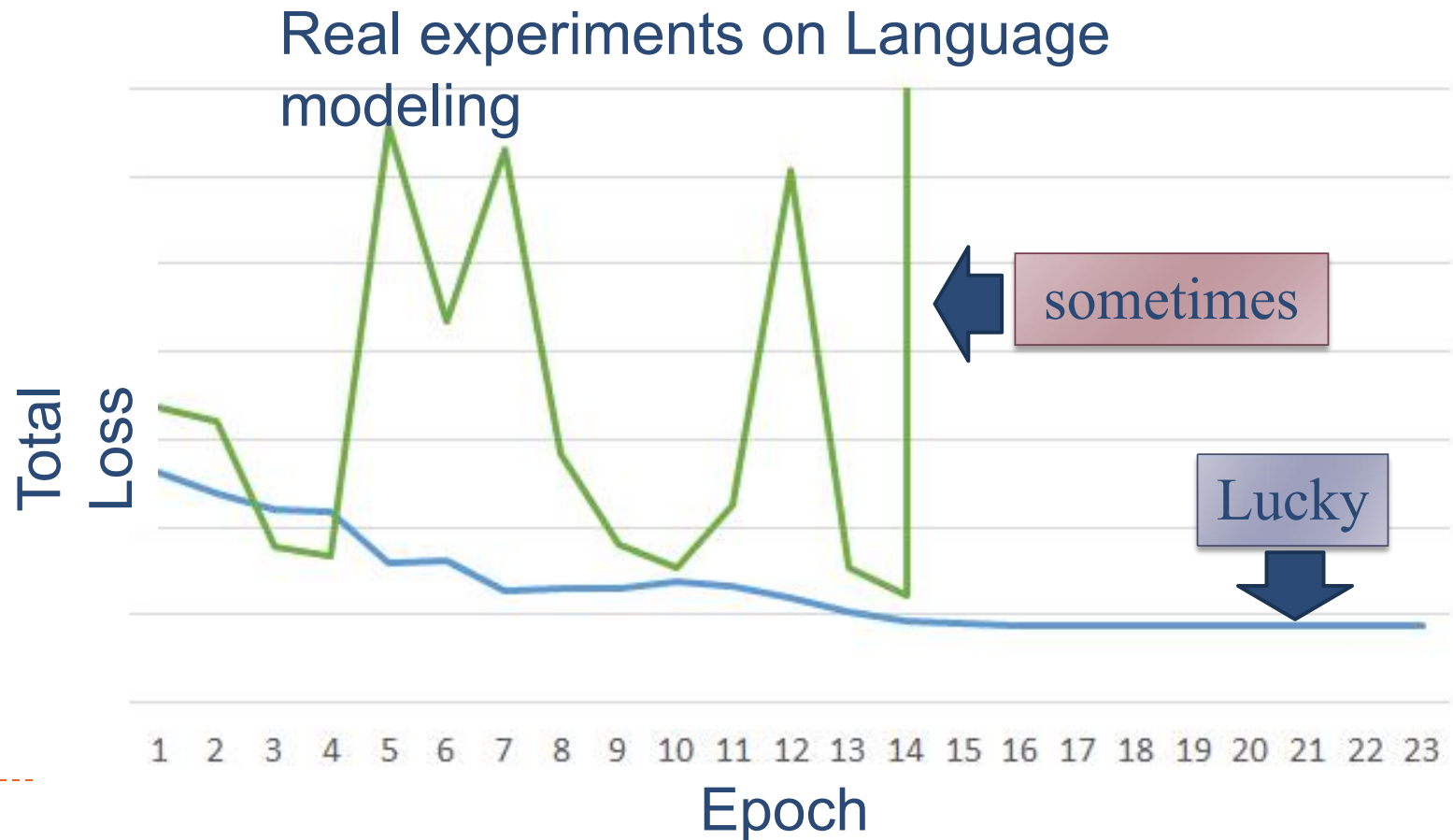
Learning



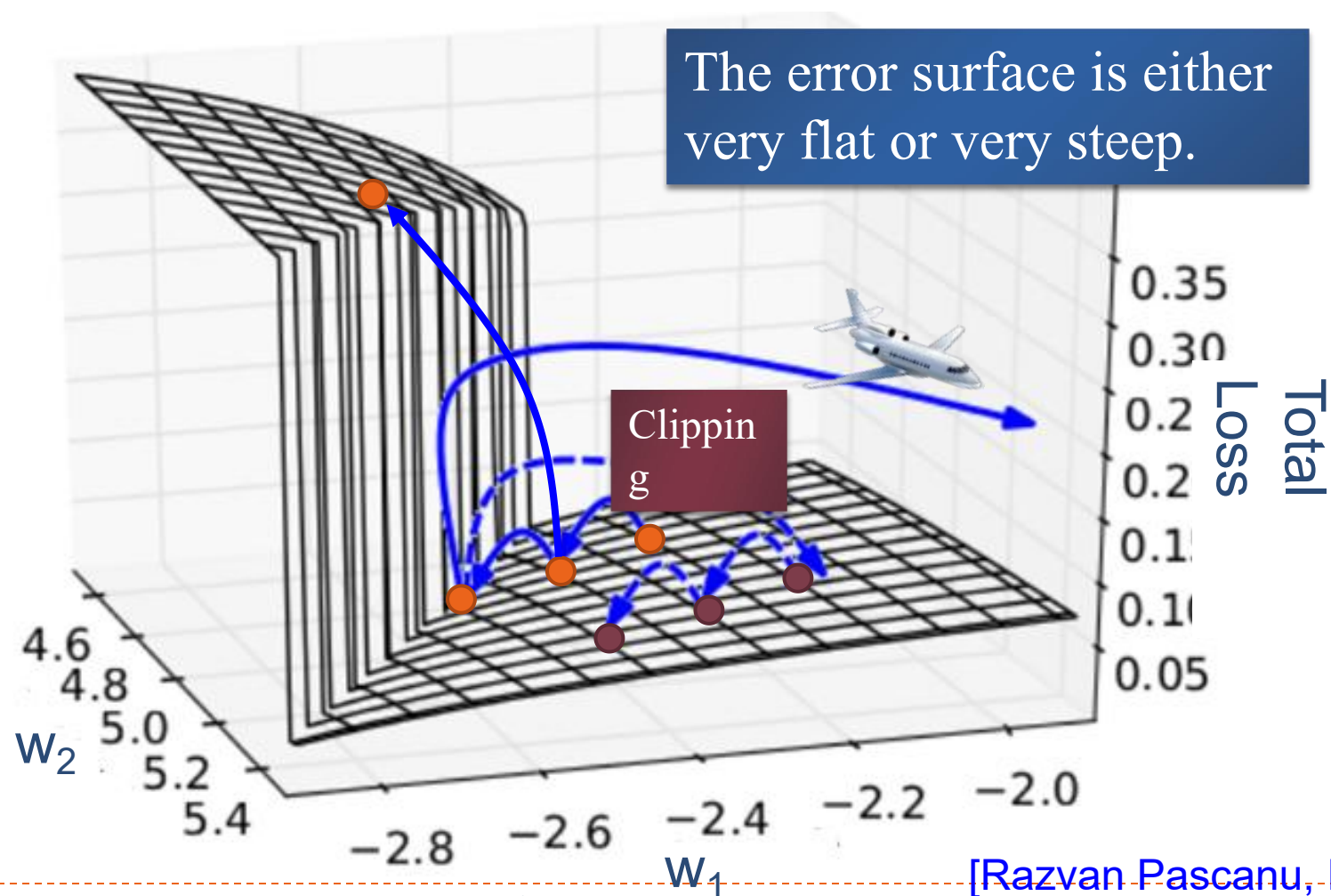
RNN Learning is very difficult in practice.

Unfortunately

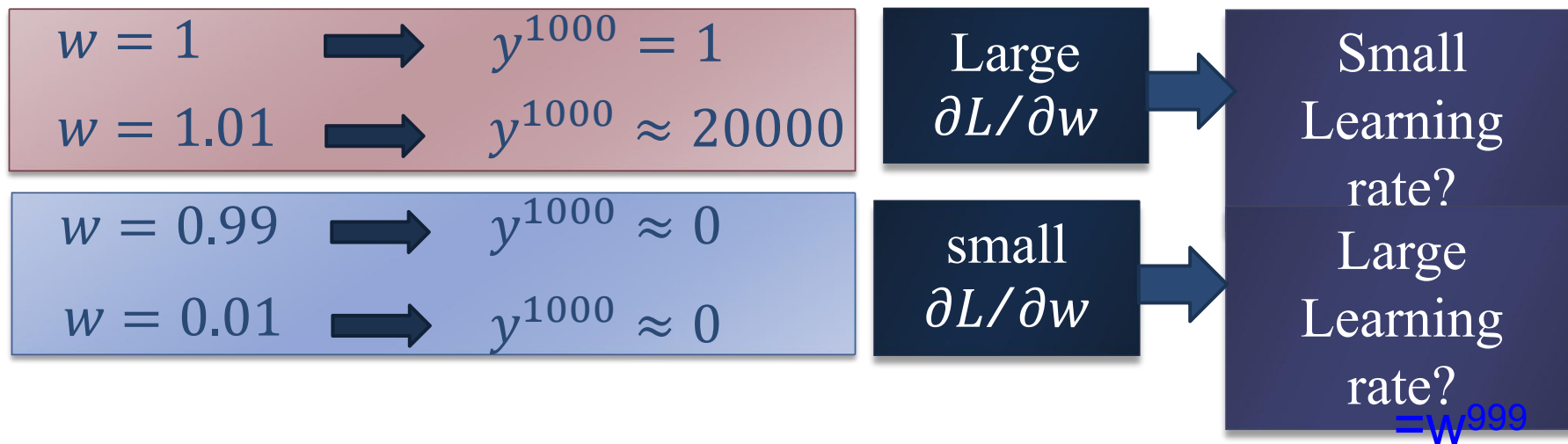
- RNN-based network is not always easy to learn



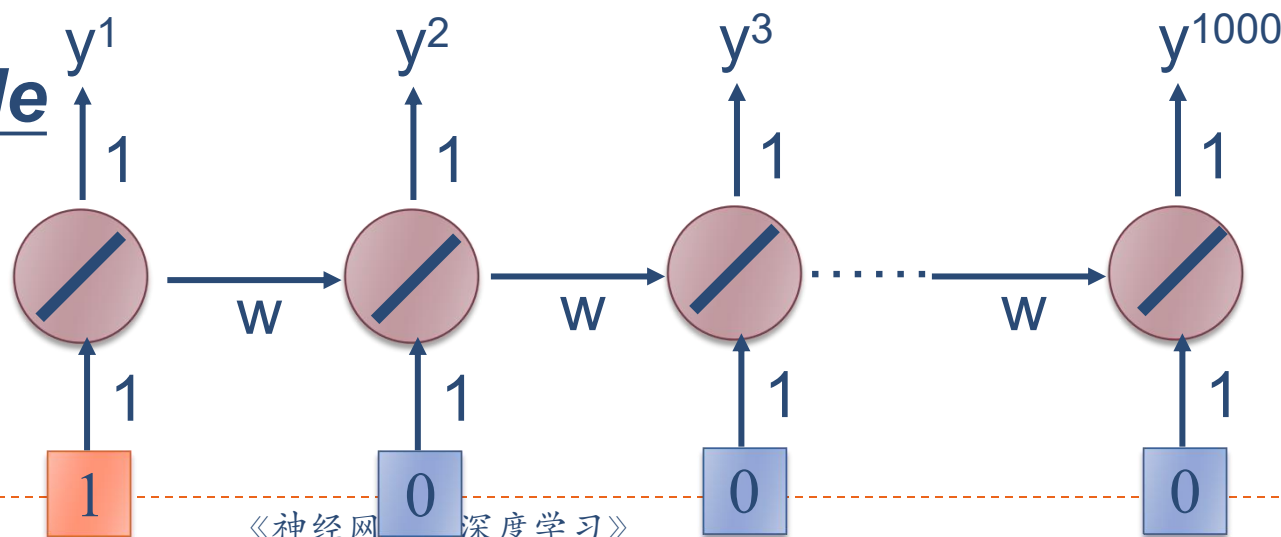
The error surface is rough.



Why?



Toy Example



Helpful Techniques

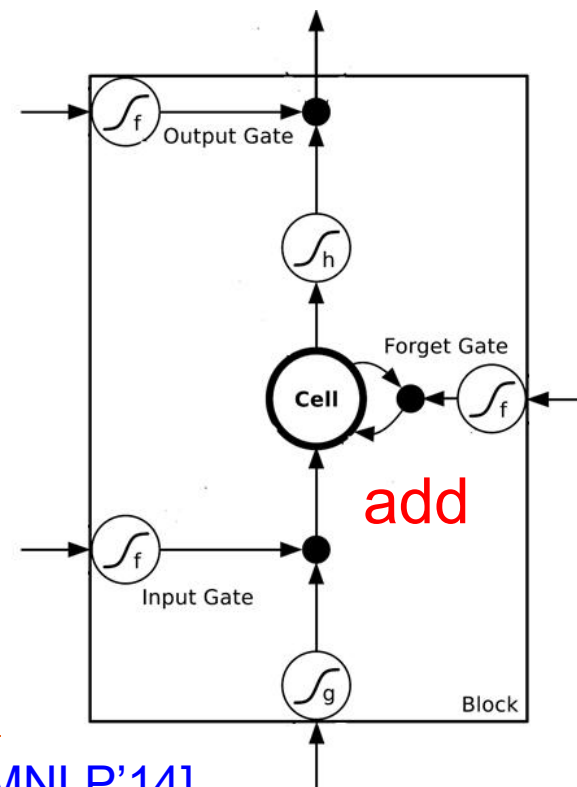
► Long Short-term Memory (LSTM)

- Can deal with gradient vanishing (not gradient explode)

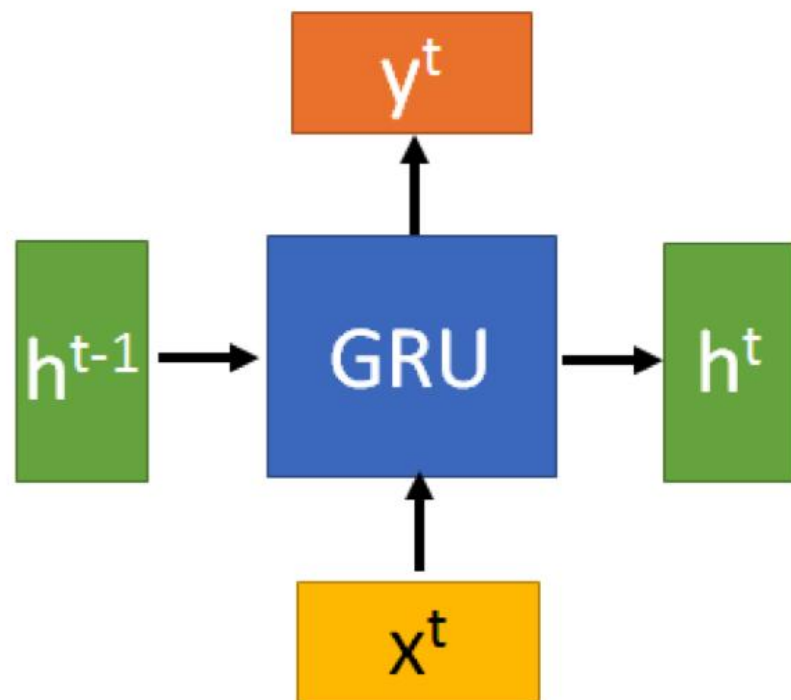
- Memory and input are added
- The influence never disappears unless forget gate is closed

➡ No Gradient vanishing
(if forget gate is closed)

Gated Recurrent Unit
(GRU):
simpler than LSTM



GRU (门控循环单元)



https://blog.csdn.net/liusisi_

$$\mathbf{h}_t = \mathbf{h}_{t-1} + g(\mathbf{x}_t; \theta),$$

GRU 网络引入门控机制来控制信息更新的方式. 和 LSTM 不同, GRU 不引入额外的记忆单元 (Memory Cell), GRU 网络也是在公式(6.50)的基础上引入两个更新门: 更新门 (Update Gate) 和重置门 (Reset Gate)

宏观结构

通过上一个传输下来的状态 $\mathbf{H}_{t-1}$ 和当前节点的输入 $\mathbf{x}_t$ 来获取两个门控状态。

GRU

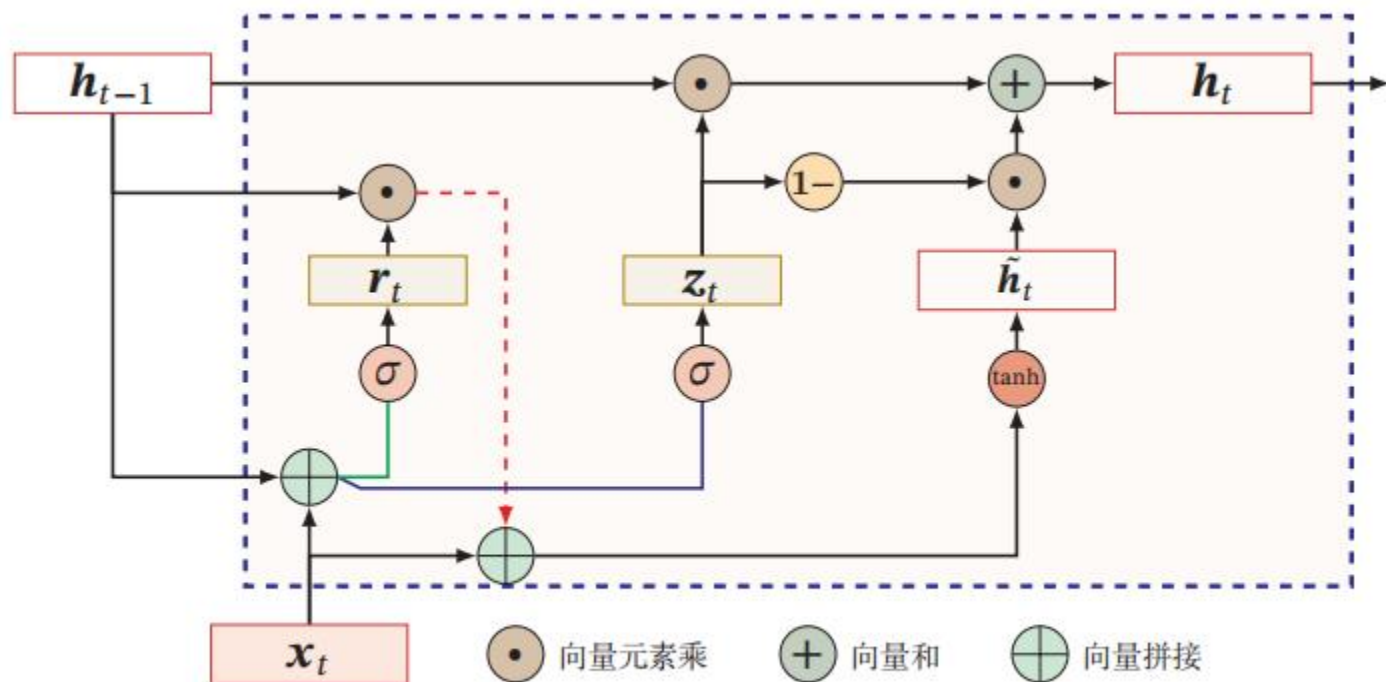


图 6.8 GRU 网络的循环单元结构

更新门 z_t

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z).$$

作用：控制当前状态需要从历史状态中保留多少信息（不经过非线性变换），以及需要从候选状态中接受多少新信息

微观（内部）结构

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot g(x_t, h_{t-1}; \theta),$$

GRU

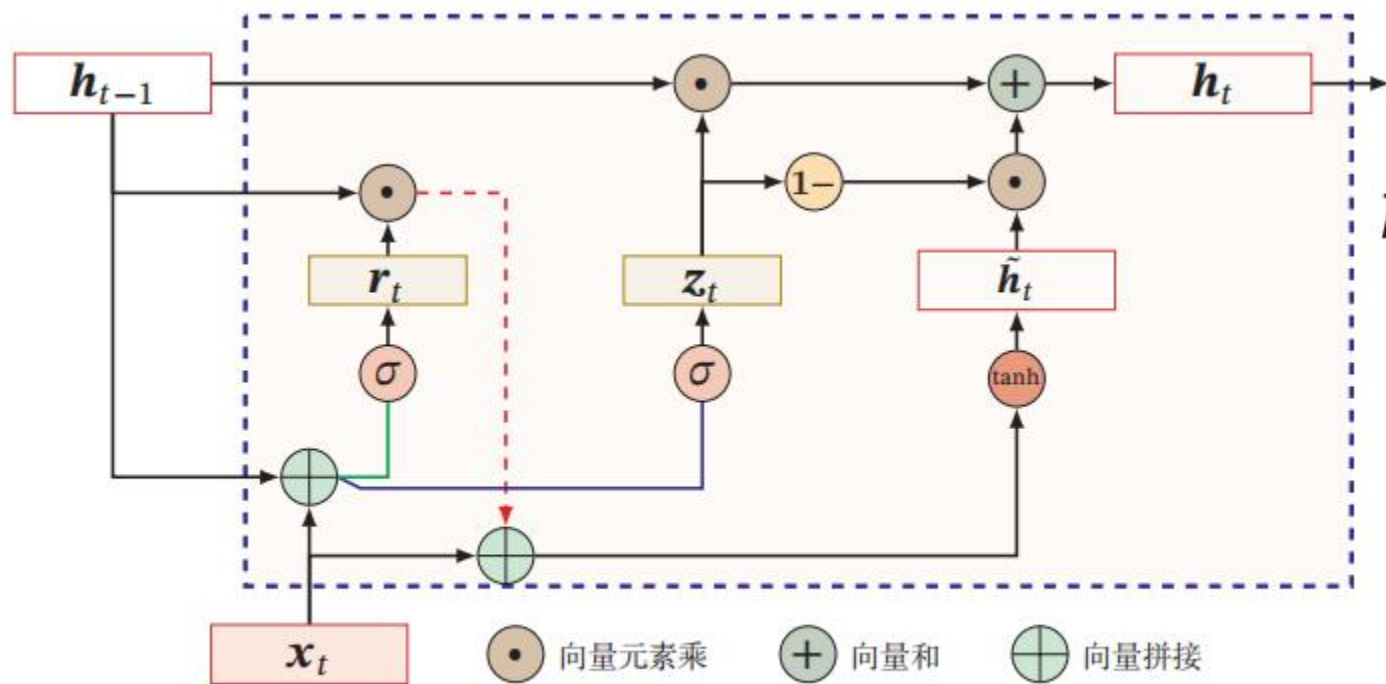
$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot g(\mathbf{x}_t, \mathbf{h}_{t-1}; \theta),$$

在LSTM网络中,输入门和遗忘门是互补关系,具有一定的冗余性. GRU网络直接使用一个门来控制输入和遗忘之间的平衡. 当 $\mathbf{z}_t = \mathbf{0}$ 时,当前状态 $\mathbf{h}_t$ 和前一时刻的状态 $\mathbf{h}_{t-1}$ 之间为非线性函数关系;当 $\mathbf{z}_t = \mathbf{1}$ 时, $\mathbf{h}_t$ 和 $\mathbf{h}_{t-1}$ 之间为线性函数关系.

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h),$$

当前时刻的候选状态

GRU

重置门 r_t

$$r_t = \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r),$$

作用：用来控制候选状态的计算是否依赖上一时刻的状态 $\mathbf{h}_{t-1}$ 。

微观（内部）结构

GRU

当 $\mathbf{r}_t = \mathbf{0}$ 时, 候选状态 $\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{b})$ 只和当前输入 $\mathbf{x}_t$ 相关, 和历史状态无关. 当 $\mathbf{r}_t = \mathbf{1}$ 时, 候选状态 $\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$ 和当前输入 $\mathbf{x}_t$ 以及历史状态 $\mathbf{h}_{t-1}$ 相关, 和简单循环网络一致.

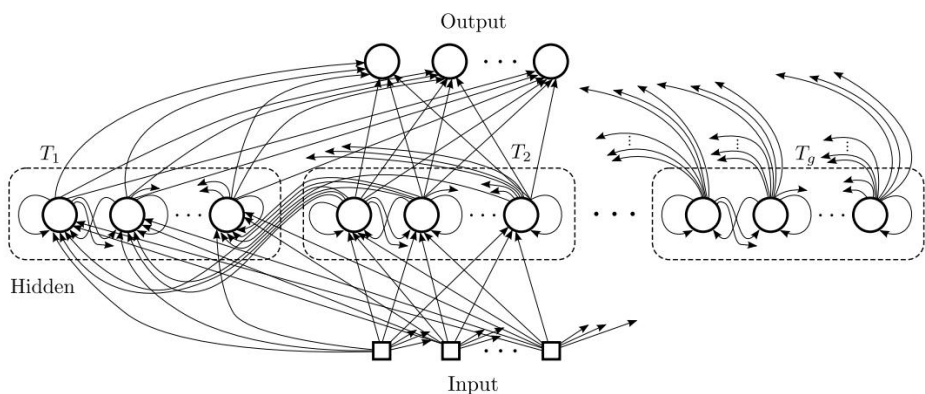
综上, GRU 网络的状态更新方式为

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t. \quad (6.69)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h (\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h),$$

RNN改进措施

Clockwise RNN



[Jan Koutnik, JMLR'14]

Clockwise RNN (CW-RNN) 是一种改进的循环神经网络 (RNN) 架构，旨在解决传统RNN在处理长距离依赖时出现的梯度消失和信息衰减问题。

核心理念

► 分模块时钟机制：将隐含层划分为多个模块，每个模块具有不同的更新周期（即“时钟频率”）。

● 高频模块（如周期为1）负责处理短期依赖，每次时间步都更新。

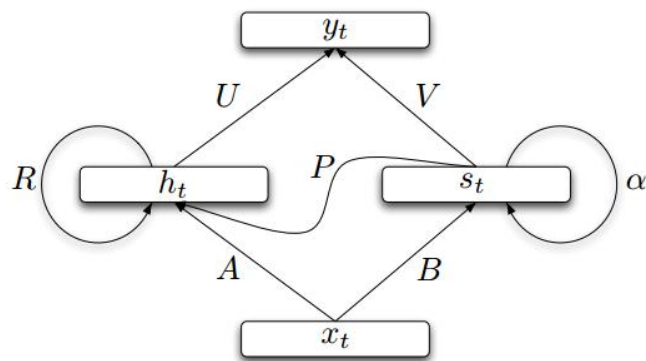
● 低频模块（如周期为2、4、8...）负责捕捉长期依赖，仅在特定时间步更新。

► 单向信息流：模块间连接具有方向性——从高频模块指向低频模块，确保短期信息可逐步传递至长期记忆。

► 参数结构约束：通过将权重矩阵设为上三角矩阵，强制实现上述单向信息流，避免低频模块干扰高频模块的快速更新。

Helpful Techniques

Structurally Constrained Recurrent Network (SCRN)



[Tomas Mikolov, ICLR'15]

Tomas Mikolov 在 ICLR 2015 的论文《Learning Longer Memory in Recurrent Neural Networks》中提出。GitHub 上有开源实现。

SCRN 是一种对标准循环神经网络 (RNN) 结构进行约束的改进模型，旨在缓解 RNN 训练中的梯度消失/爆炸问题，并提升对长序列依赖的建模能力。

结构设计：通过限制 RNN 的权重矩阵结构（如使用对角矩阵或低秩分解），减少参数数量并简化动态行为。隐状态更新被约束为更平滑的变换，从而改善梯度传播。

优势：在某些任务上性能可与 LSTM 相当甚至更好。训练稳定性高于 vanilla RNN，但实现更简单、参数更少。

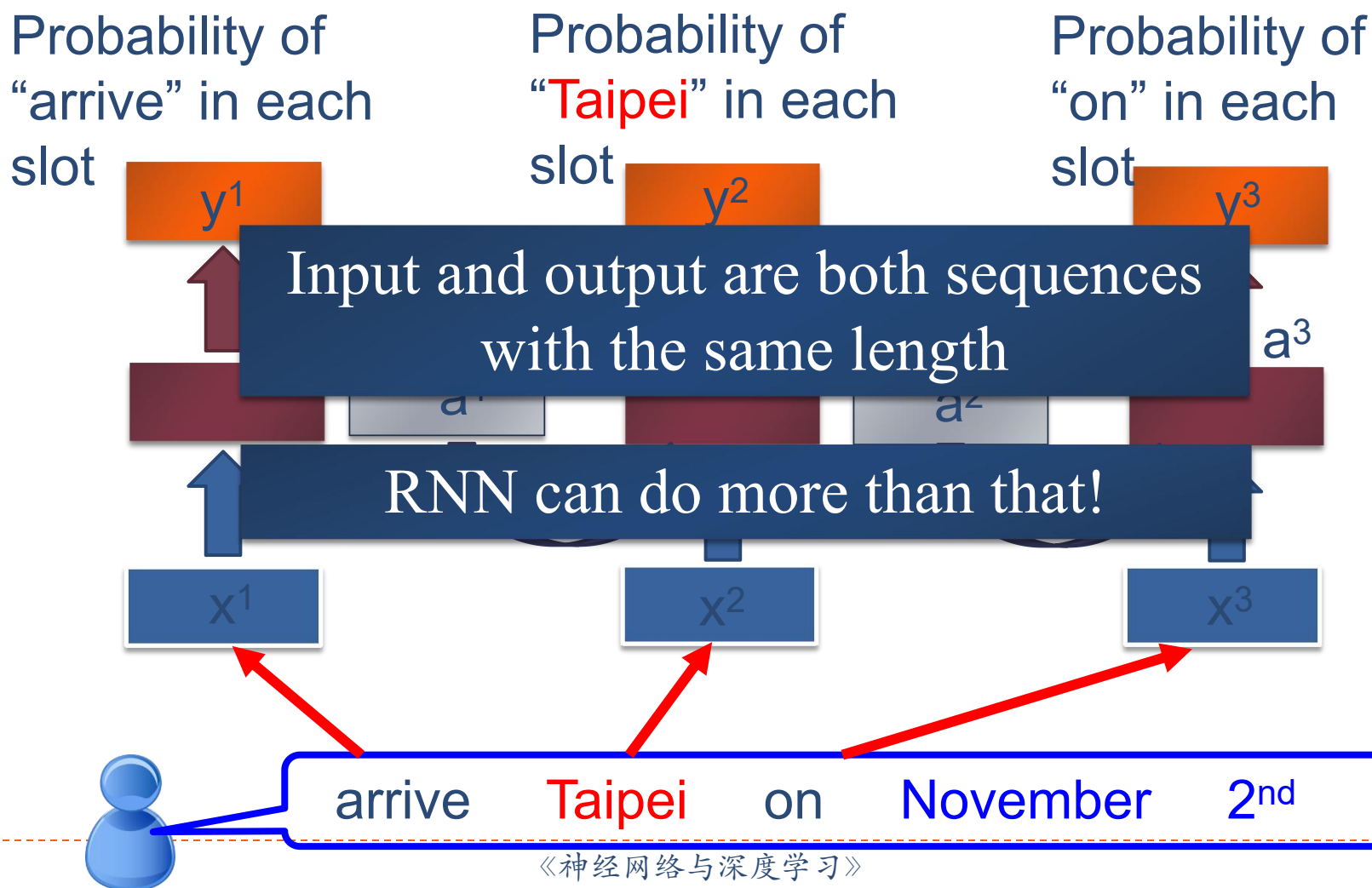
应用场景：

序列建模任务，如语言建模、槽位填充 (Slot Filling)、情感分析等。

Structurally Constrained Recurrent Network (SCRN)

模型	是否门控	参数复杂度	梯度稳定性	与SCRN关系
Vanilla RNN	否	低	差	SCRN 是其结构约束版本
LSTM	是 (3门)	高 (4倍参数)	高	SCRN 更轻量, 但性能接近
GRU	是 (2门)	中	高	SCRN 不使用门控机制
SCRN	否 (结构约束)	低~中	中~高	通过结构而非门控改善训练

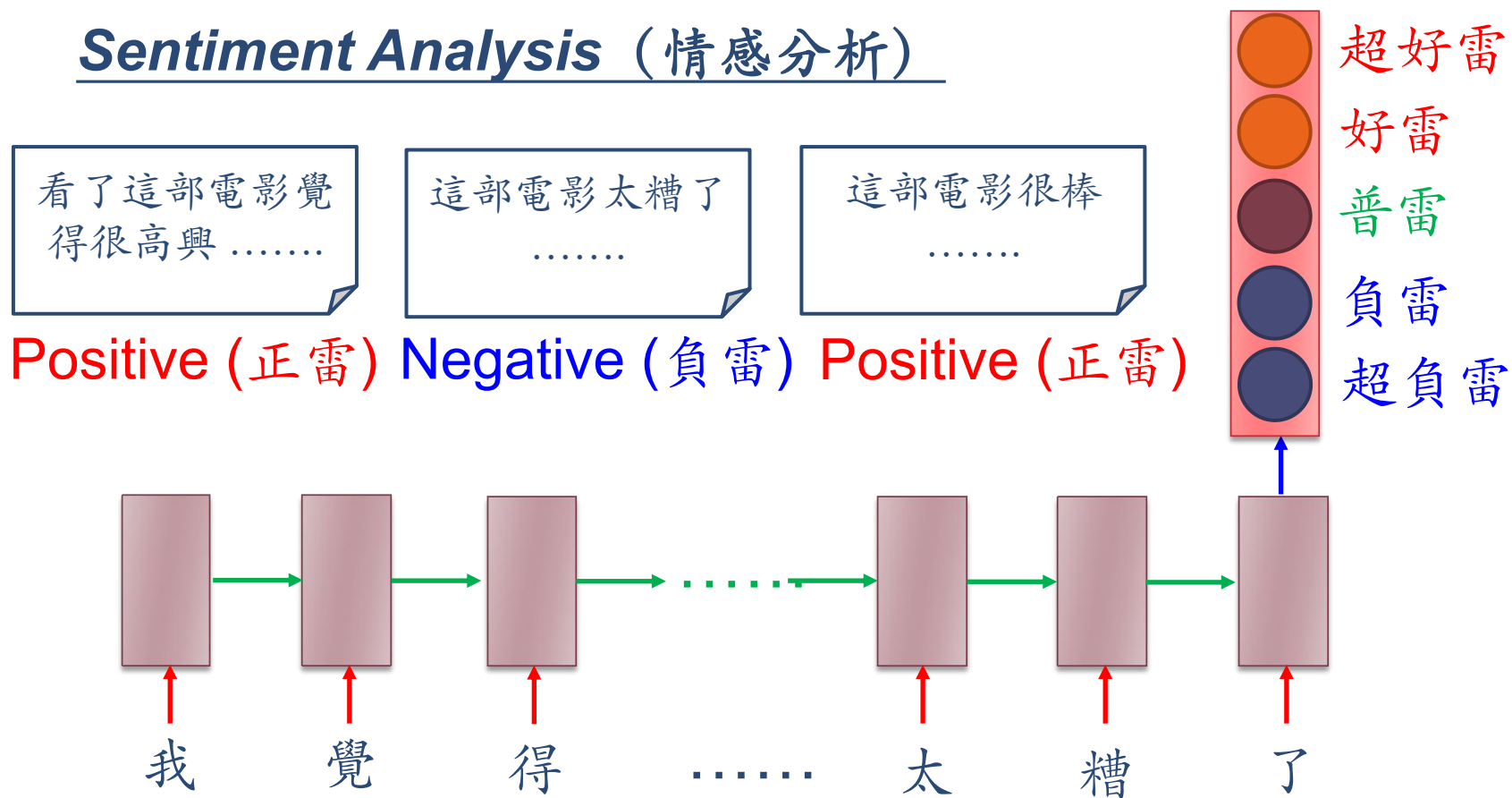
填词 (One-to-One)



Many to one

- Input is a vector sequence, but output is only one vector

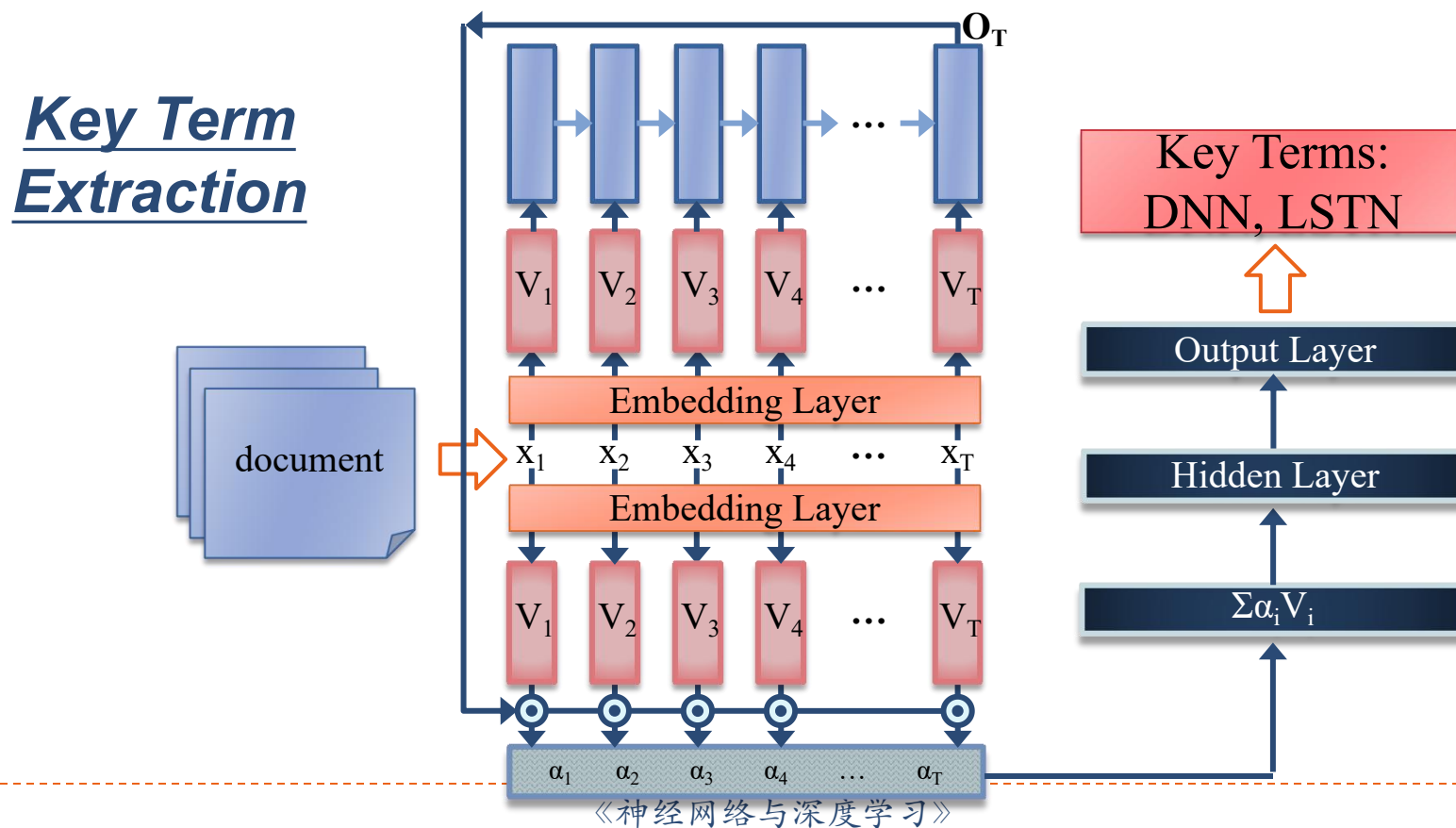
Sentiment Analysis (情感分析)



Many to one

[Shen & Lee, Interspeech 16]

- Input is a vector sequence, but output is only one vector



Many to Many (Output is shorter)

► Both input and output are both sequences, but the output is shorter.

► E.g. Speech Recognition

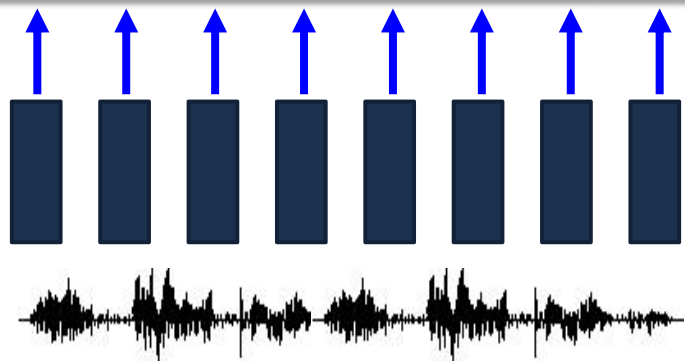
Output: “好棒” (character sequence)



Trimming

好 好 好 棒 棒 棒 棒 棒

Input:



(vector sequence)

RNN在语音识别等序列任务中如何处理重复字符与输出对齐问题

• RNN处理序列时，输入（如语音帧）通常比输出（如文字“好棒”）长得多：例如，说“好棒”可能对应10个语音特征向量（时间步），而“好棒棒”对应12个——若直接用标准RNN逐帧预测，可能输出“好好好好棒棒棒棒”，无法区分意图。

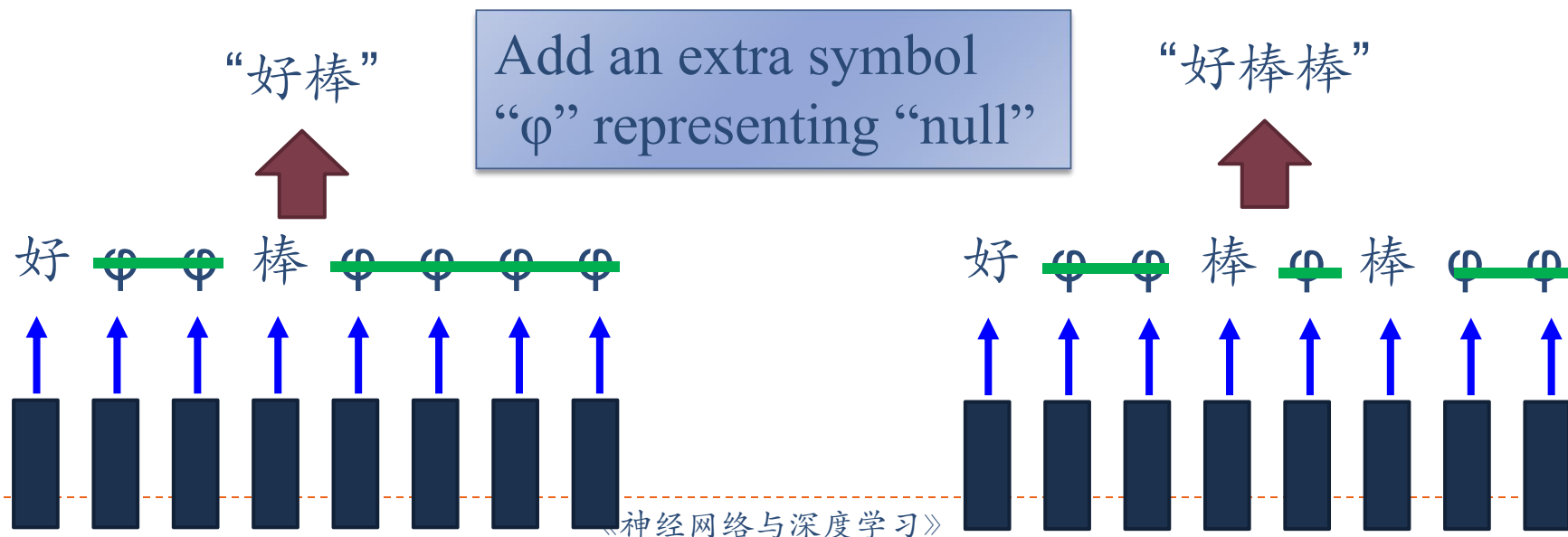
• 传统Trimming（去重）失效：简单删除连续重复字符会把“好棒棒”误判为“好棒”，因两者都变成“好棒”。

Many to Many (Output is shorter)

RNN在语音识别等序列任务中
如何处理重复字符与输出对齐问题

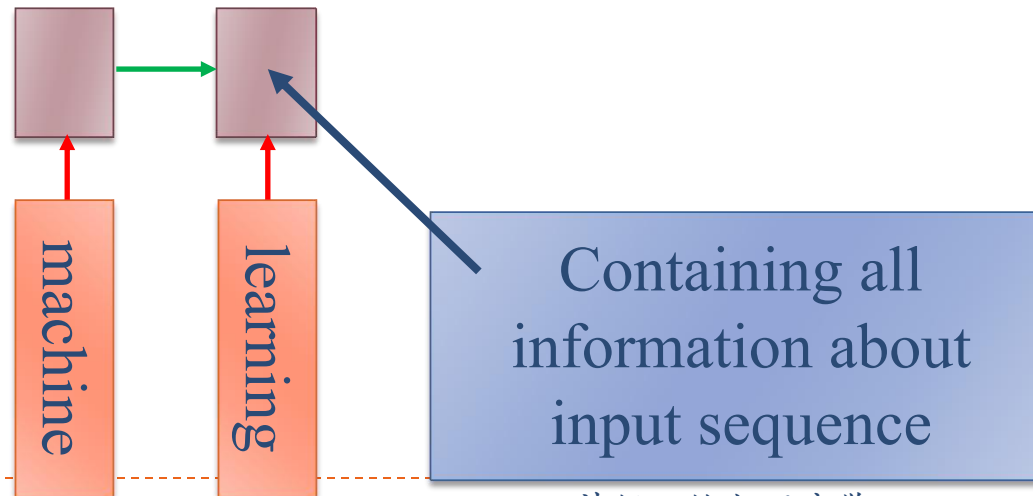
- ▶ Connectionist Temporal Classification (CTC) [Alex Graves, ICML'06][Alex Graves, ICML'14][Haşim Sak, Interspeech'15][Jie Li, Interspeech'15][Andrew Senior, ASRU'15]

- CTC引入空符号 (*blank/φ*) 解决对齐歧义：允许RNN在输出序列中插入“blank”，训练时枚举所有可能的对齐路径（如“好-好-棒-φ-棒” vs “好-φ-棒-棒-φ”），最终删除blank后得到目标标签（如“好棒”或“好棒棒”），无需人工指定每个字对应哪些帧。
- RNN本身不自动区分“好棒”与“好棒棒”：其隐藏状态虽有记忆，但原始RNN输出层仍需配合CTC或Seq2Seq（如加结束符）才能实现变长输出；仅靠Elman RNN无法自动解决此问题，必须借助CTC或注意力机制。



Many to Many (No Limitation)

- ▶ Both input and output are both sequences with different lengths. → Sequence to sequence learning
- ▶ E.g. Machine Translation (machine learning → 機器學習)



Many to Many (No Limitation)

- ▶ Both input and output are both sequences with different lengths. → Sequence to sequence learning
- ▶ E.g. Machine Translation (machine learning → 機器學習)

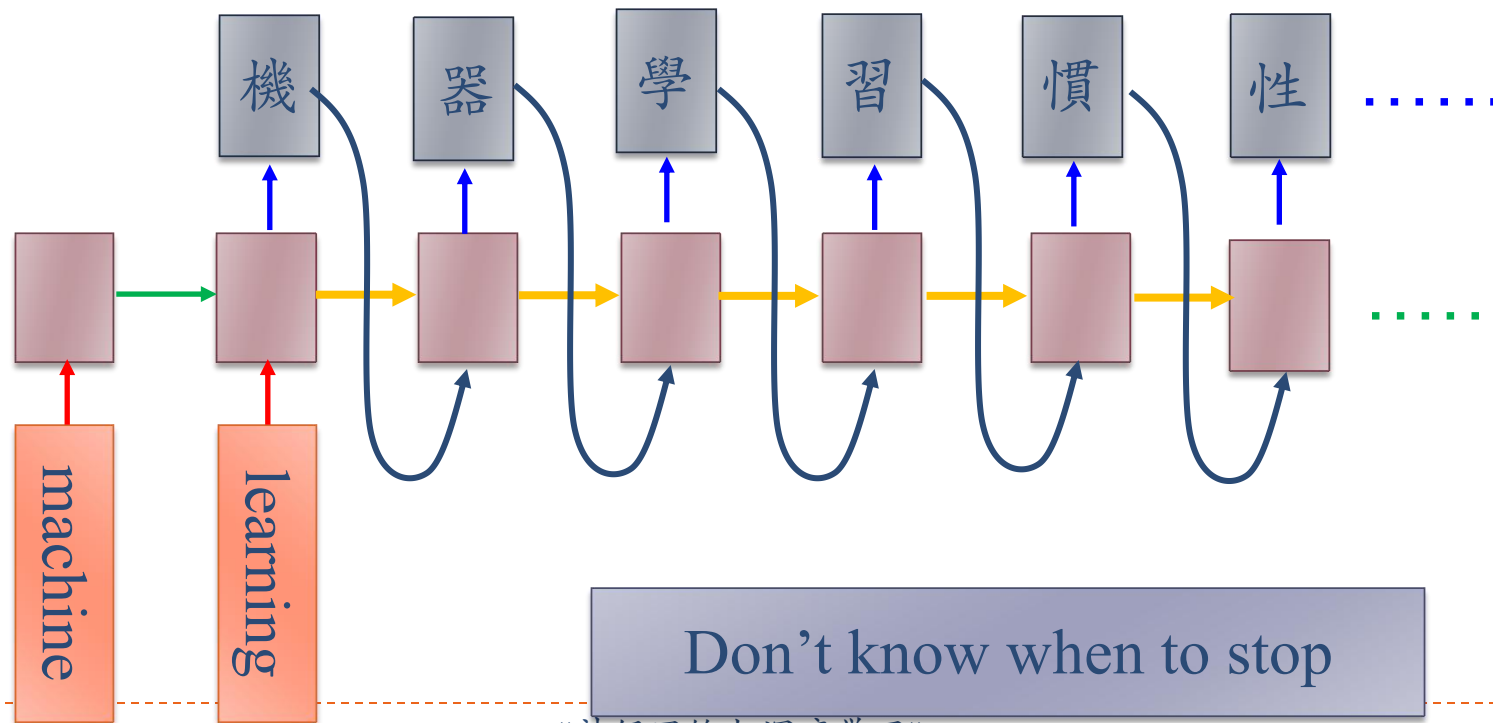
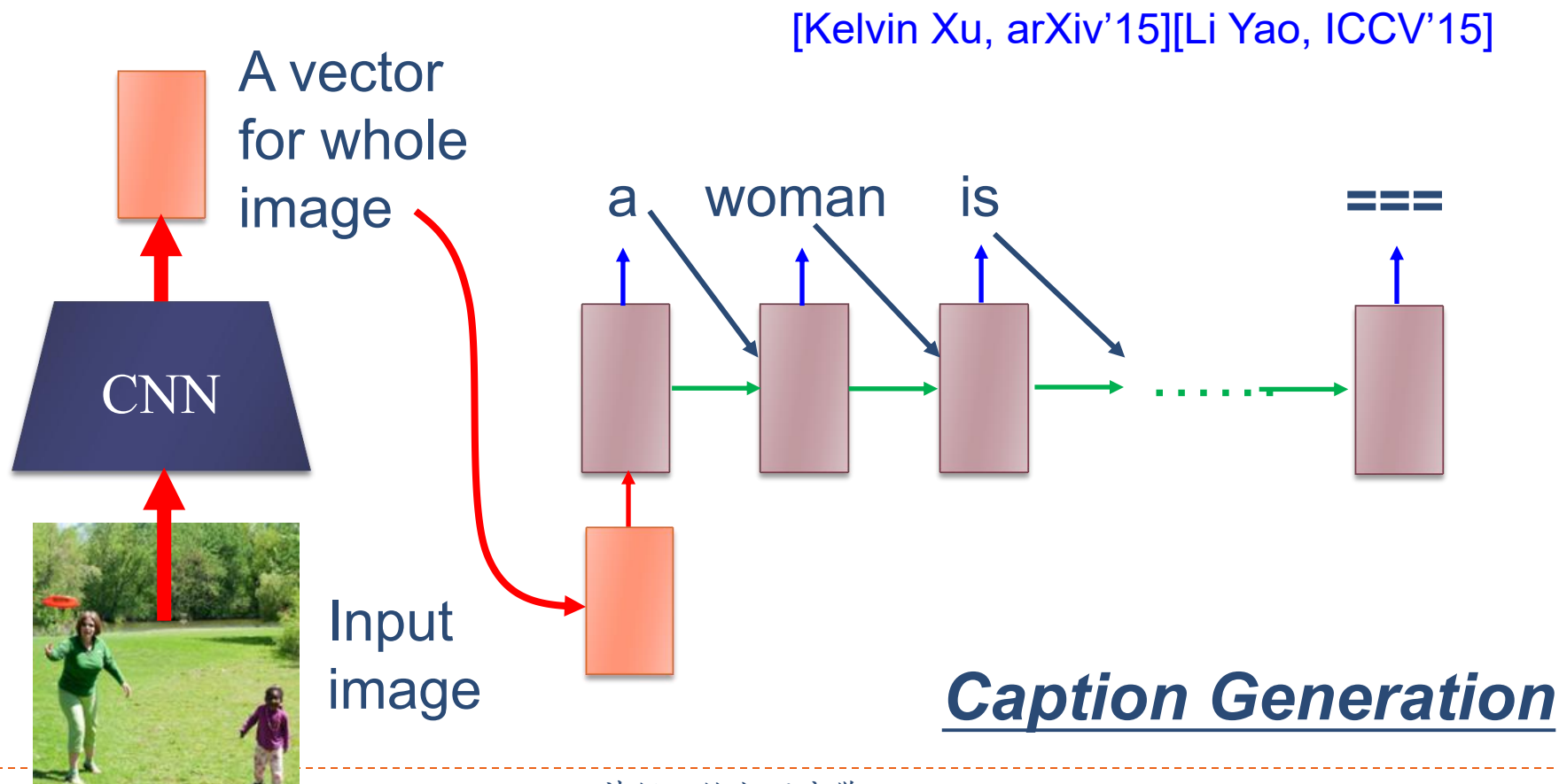
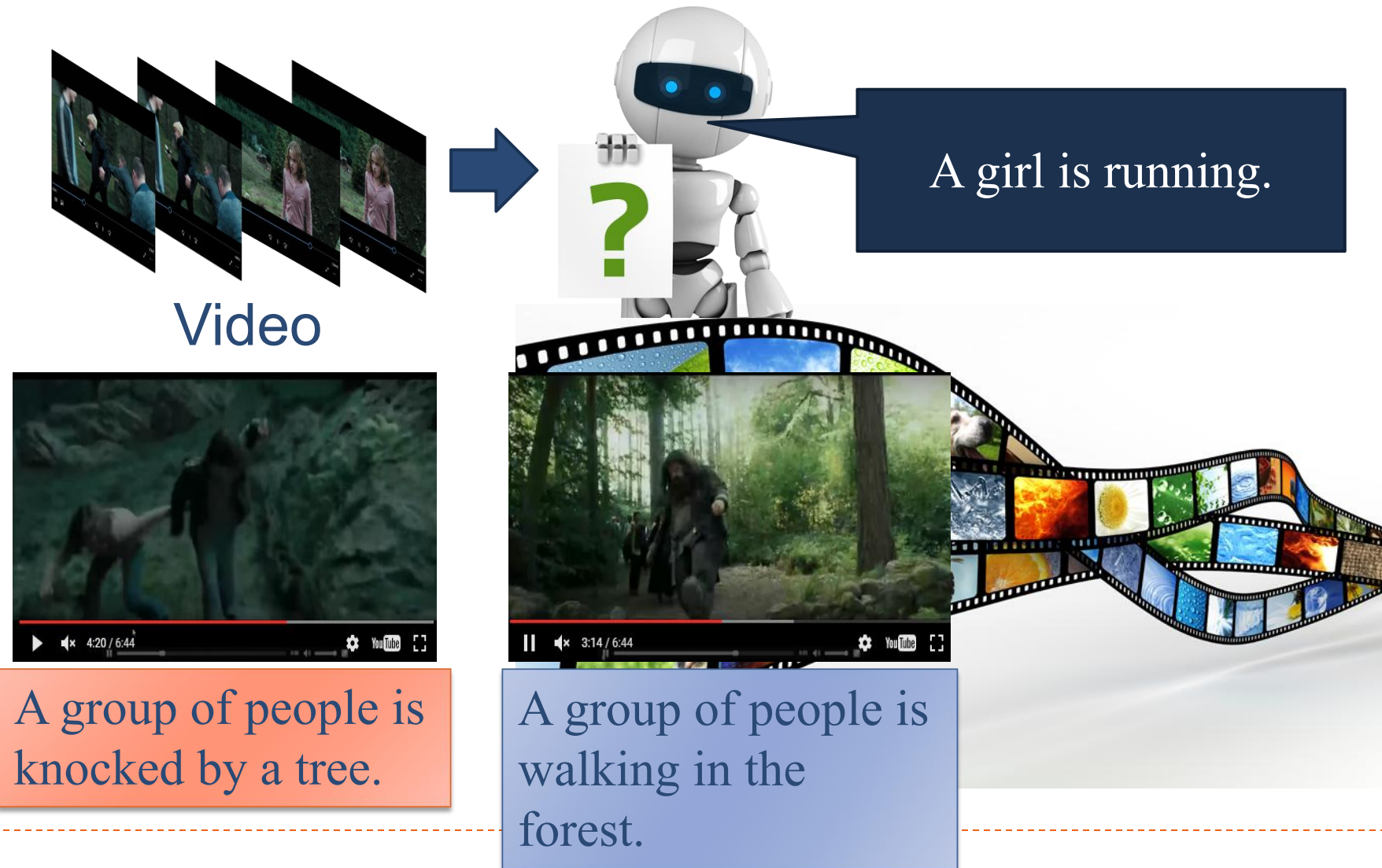


Image Caption Generation

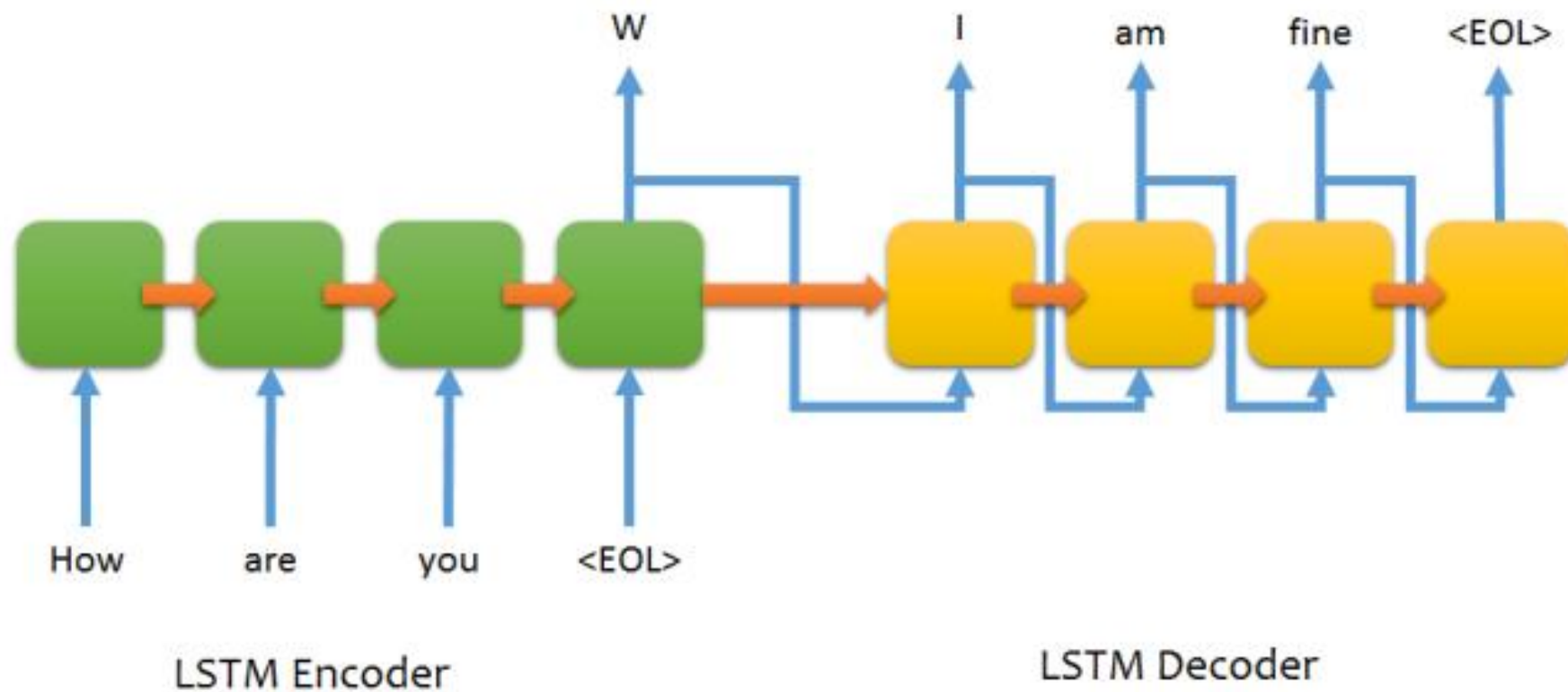
- Input an image, but output a sequence of words



Video Caption Generation

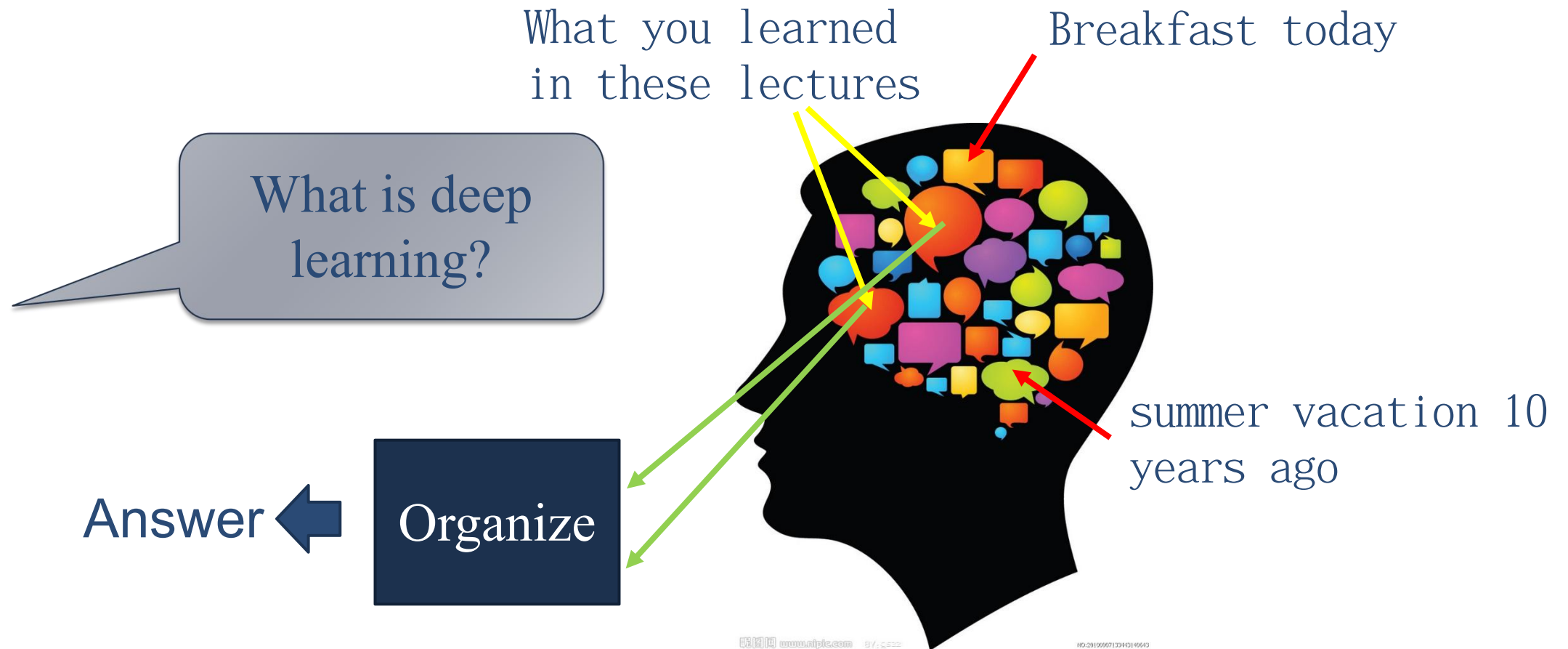


Chat-bot

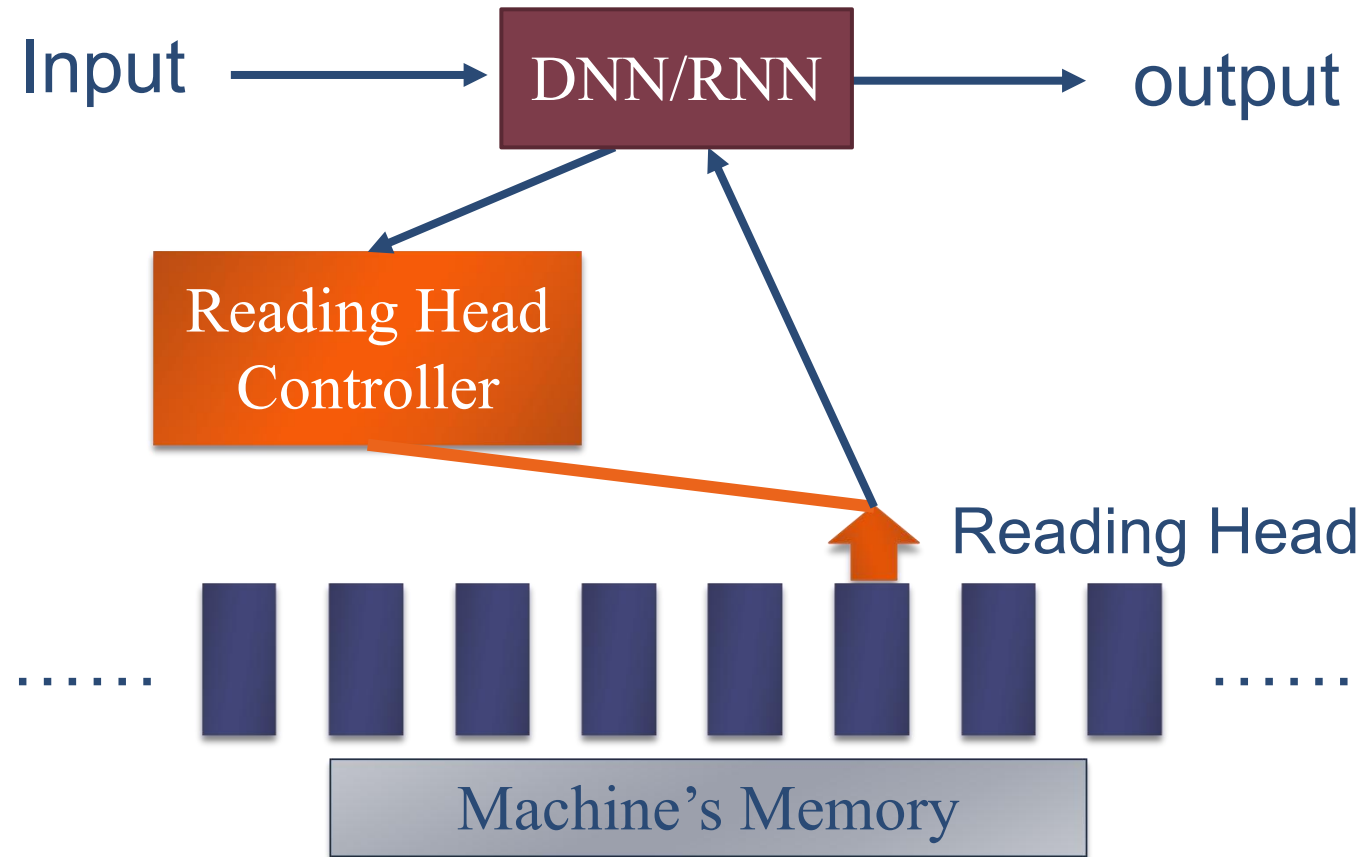


電視影集 (~40,000 sentences)、美國總統大選辯論

Attention-based Model

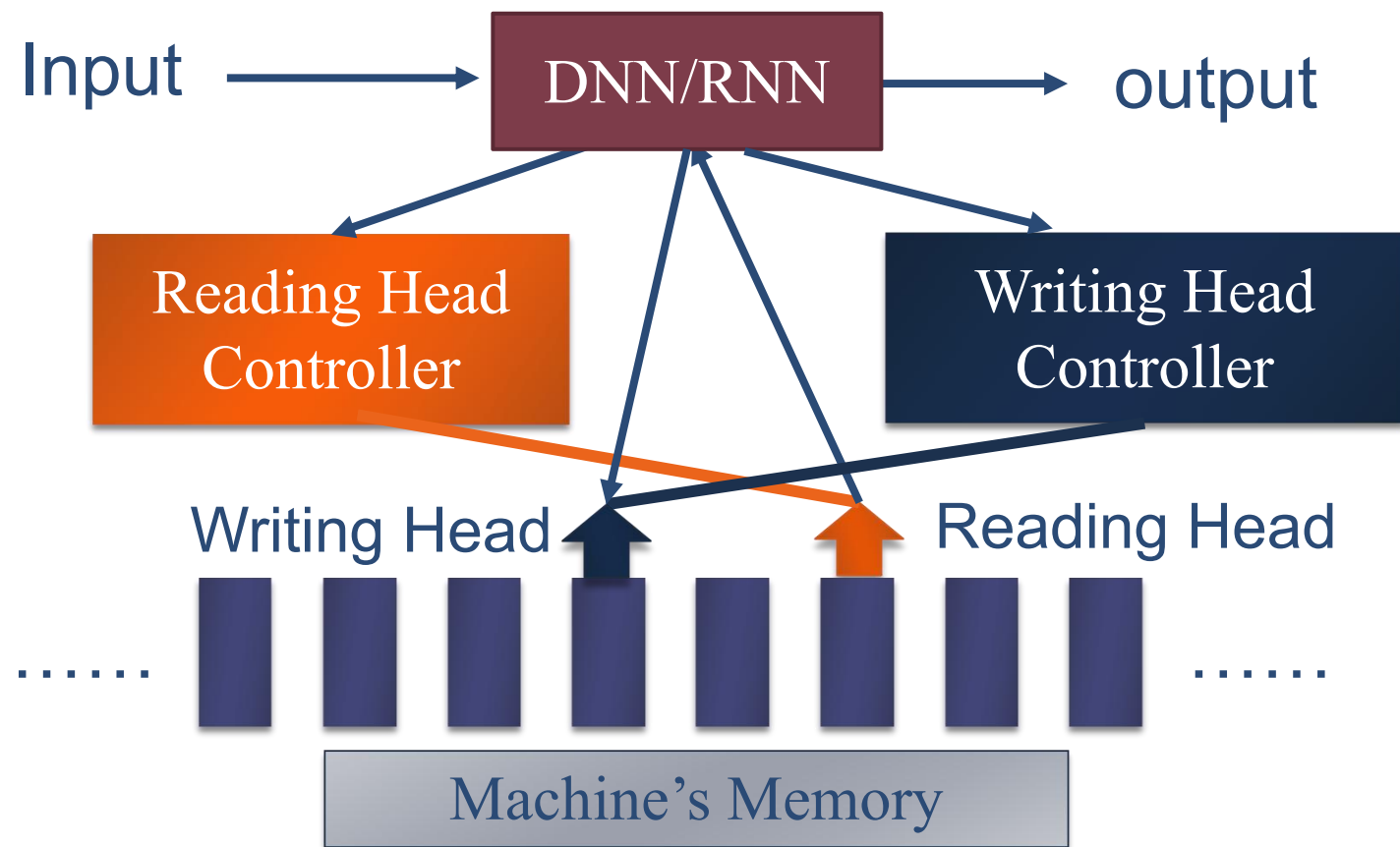


Attention-based Model

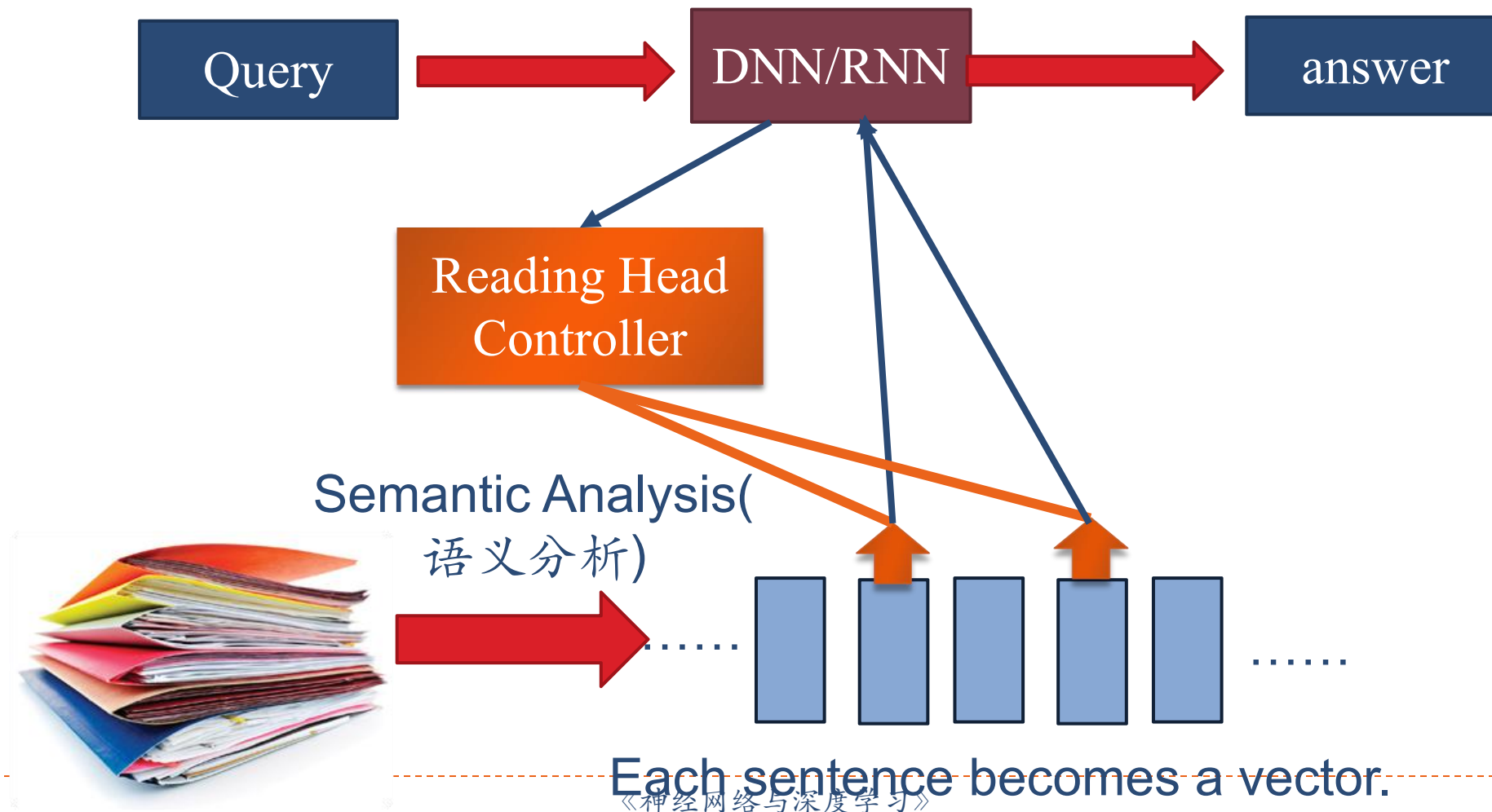


Ref: [http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/Attain%20\(v3\).ecm.mp4/index.html](http://speech.ee.ntu.edu.tw/~tlkagk/courses/MLDS_2015_2/Lecture/Attain%20(v3).ecm.mp4/index.html)

Attention-based Model v2



Reading Comprehension



Reading Comprehension

- End-To-End Memory Networks. S. Sukhbaatar, A. Szlam, J. Weston, R. Fergus. NIPS, 2015.

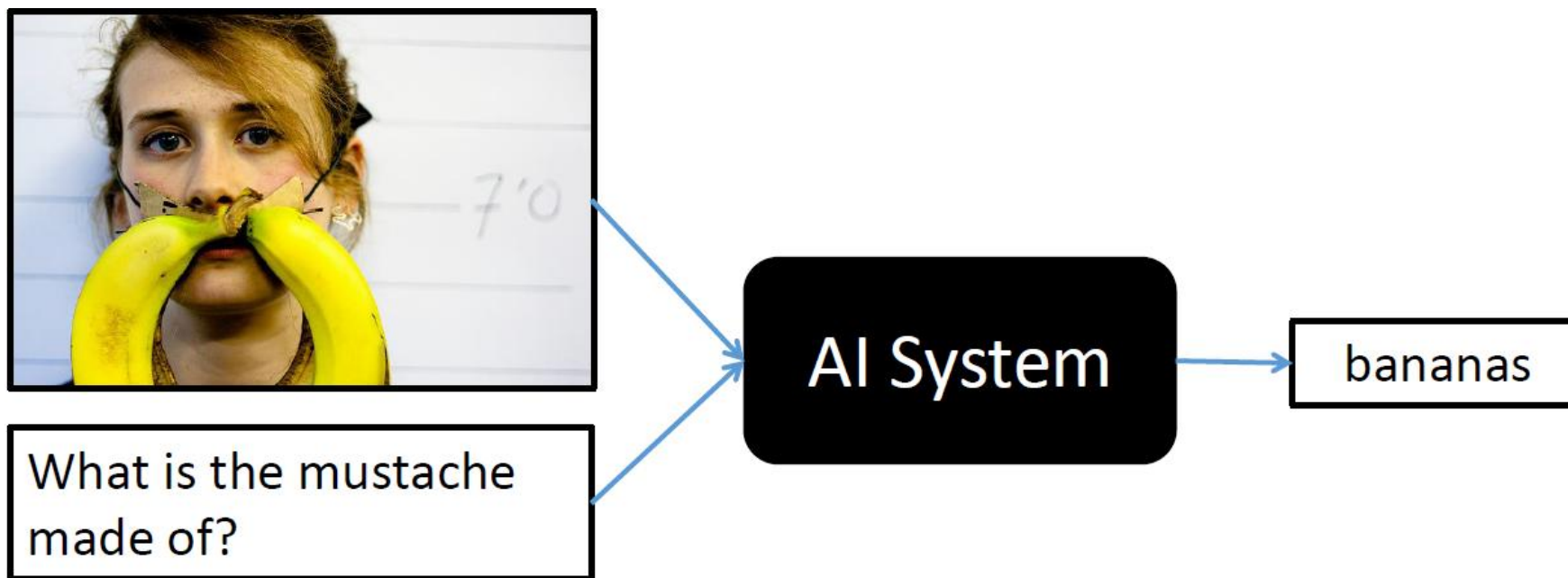
The position of reading head:

Story (16: basic induction)	Support	Hop 1	Hop 2	Hop 3
Brian is a frog.	yes	0.00	0.98	0.00
Lily is gray.		0.07	0.00	0.00
Brian is yellow.	yes	0.07	0.00	1.00
Julius is green.		0.06	0.00	0.00
Greg is a frog.	yes	0.76	0.02	0.00
What color is Greg? Answer: yellow Prediction: yellow				

Keras has example:

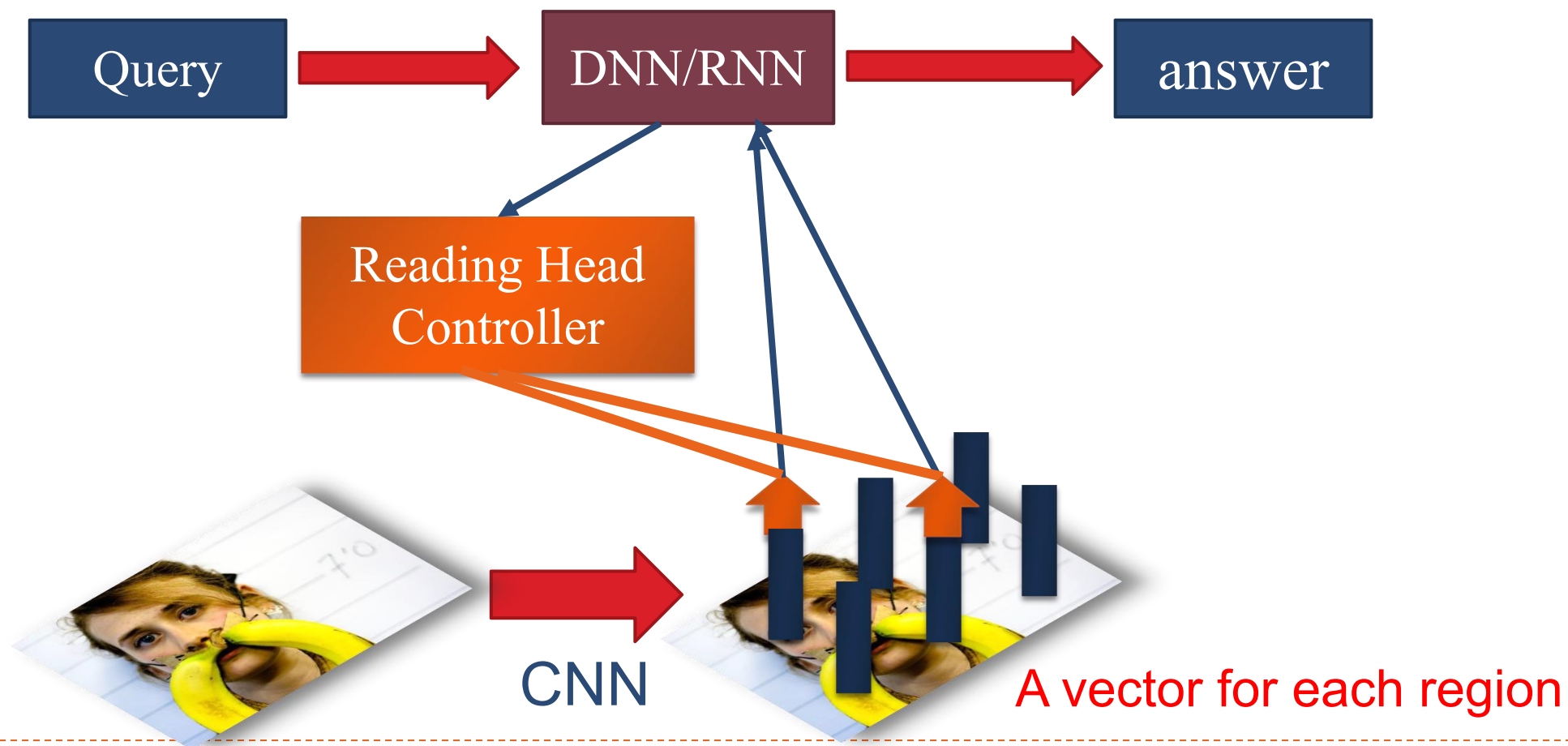
https://github.com/fchollet/keras/blob/master/examples/babi_memnn.py

Visual Question Answering



source: <http://visualqa.org/>

Visual Question Answering



Speech Question Answering

▶ TOEFL Listening Comprehension Test by Machine

▶ Example:

Audio Story:  (The original story is 5 min long.)

Question: “ What is a possible origin of Venus’ clouds? ”

Choices:

(A) gases released as a result of volcanic activity

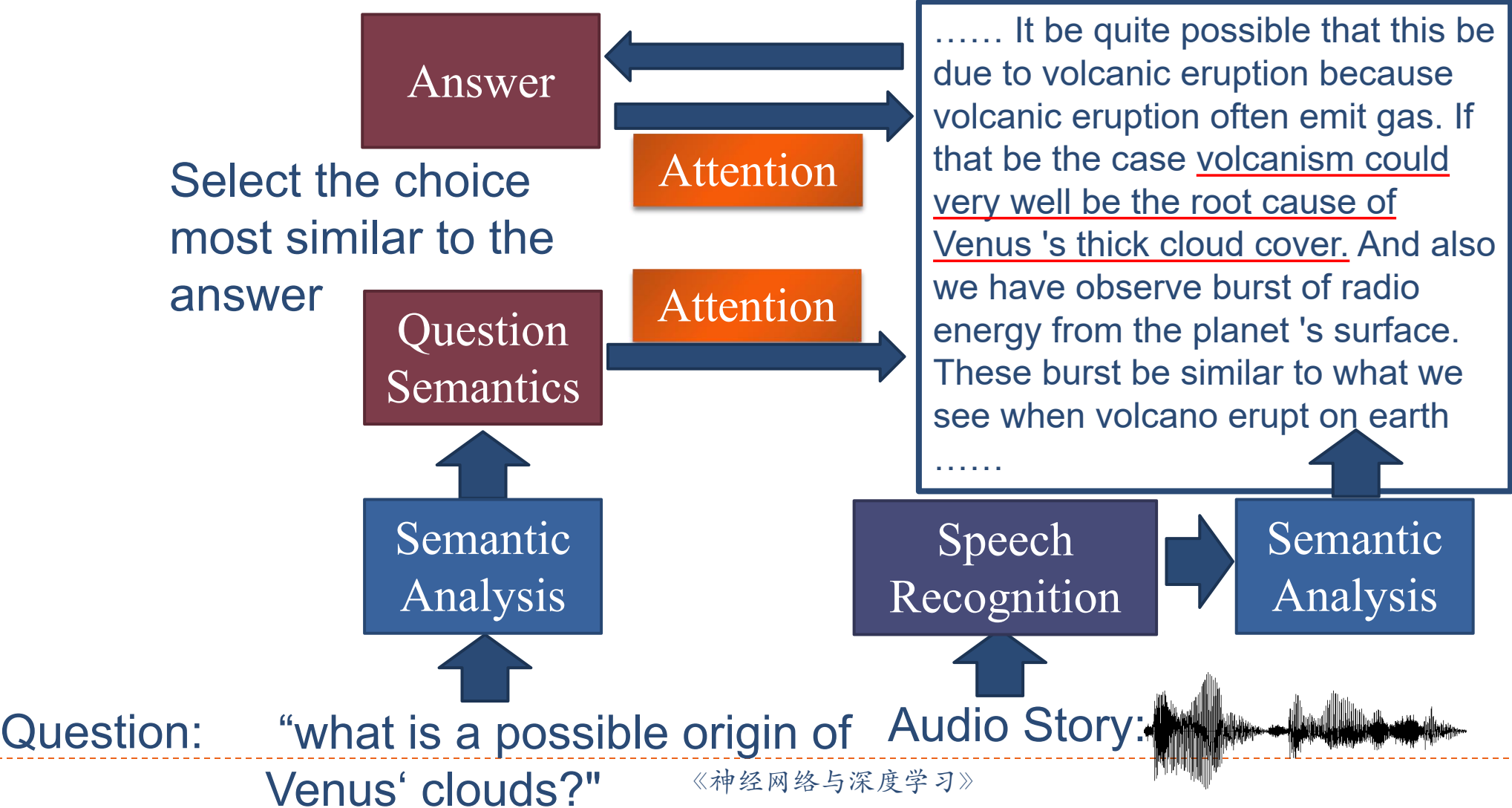
(B) chemical reactions caused by high surface temperatures

(C) bursts of radio energy from the plane's surface

(D) strong winds that blow dust into the atmosphere

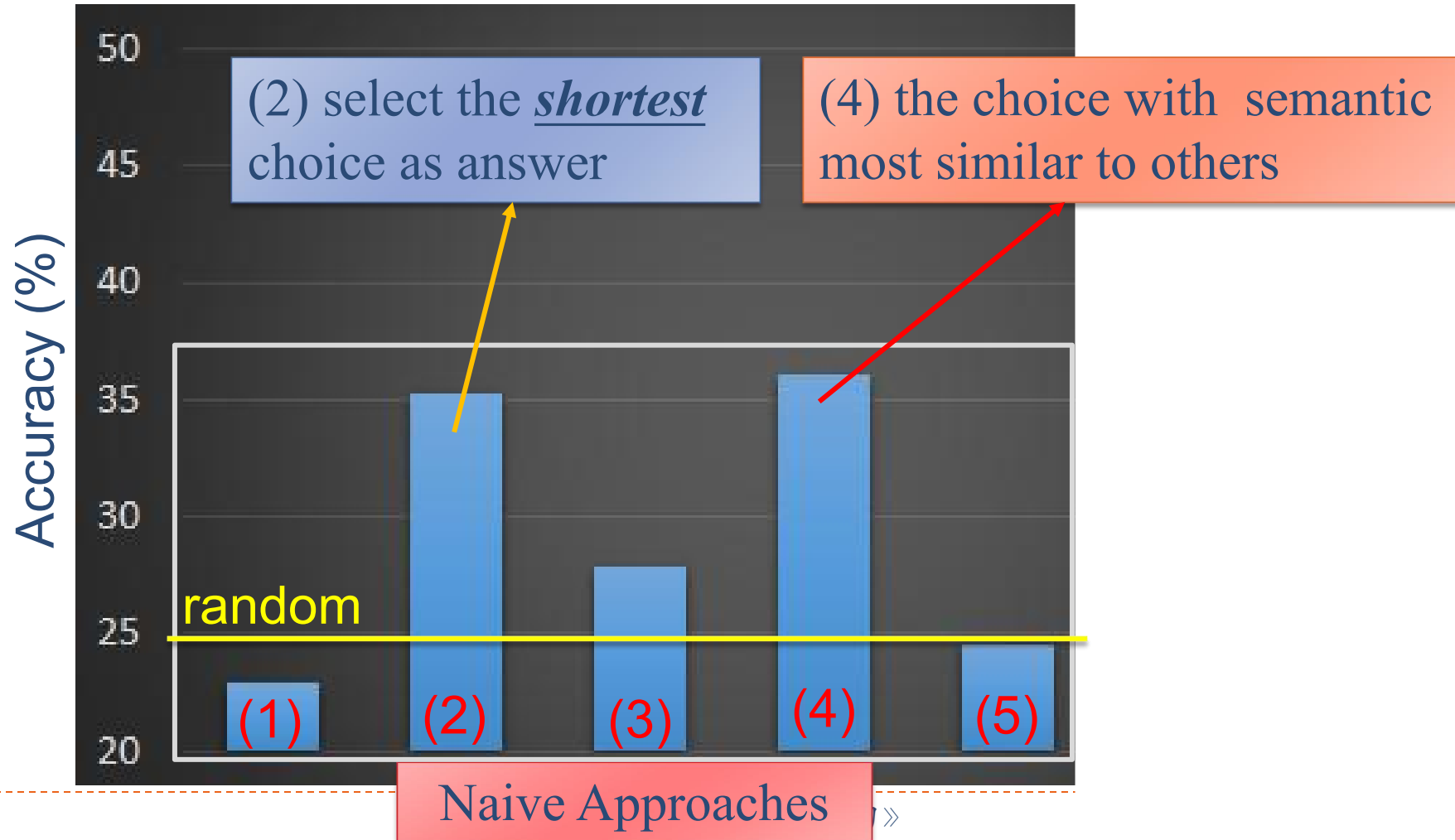
Everything is learned from training examples

Model Architecture

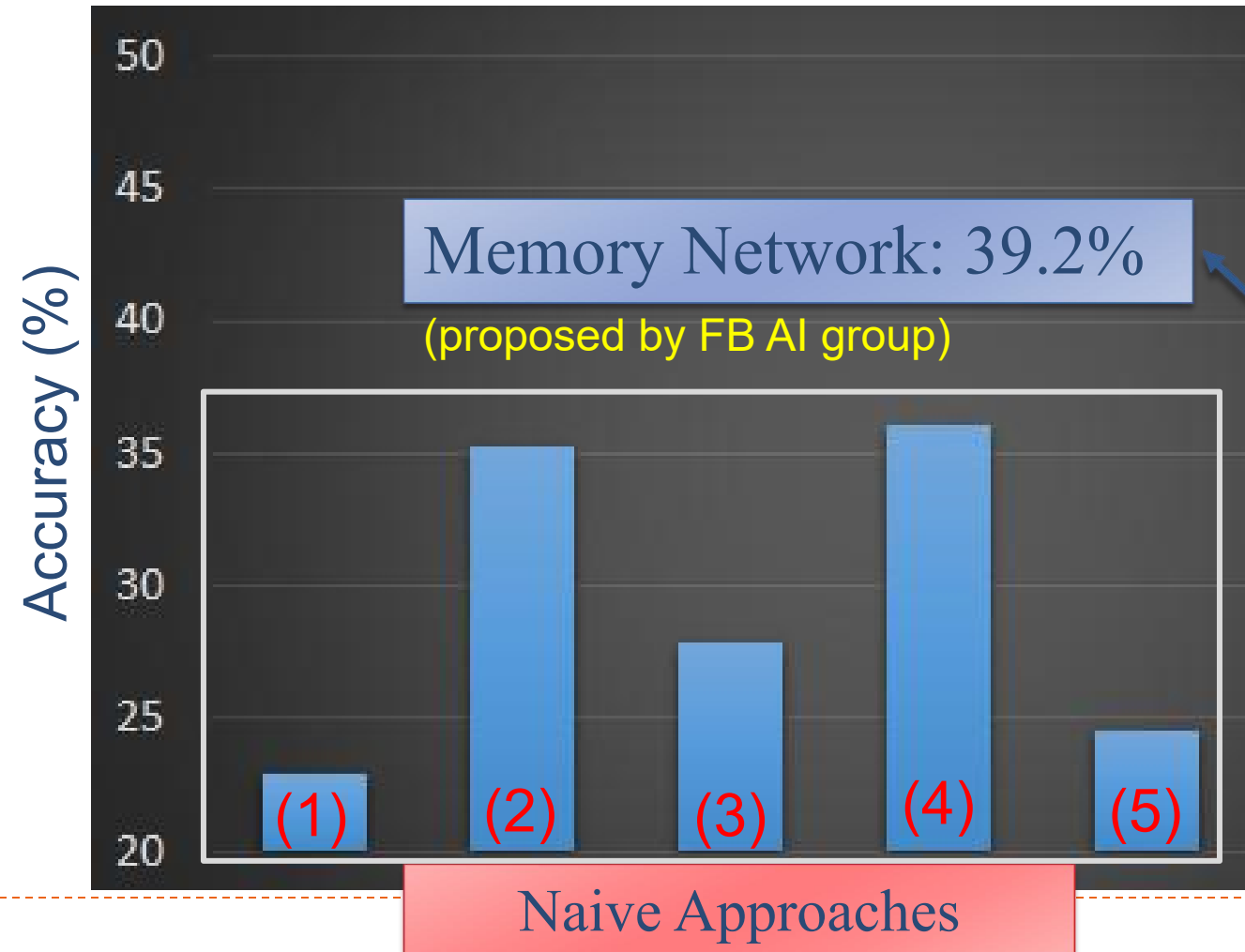


Simple Baselines

Experimental setup:
717 for training,
124 for validation, 122
for testing



Memory Network



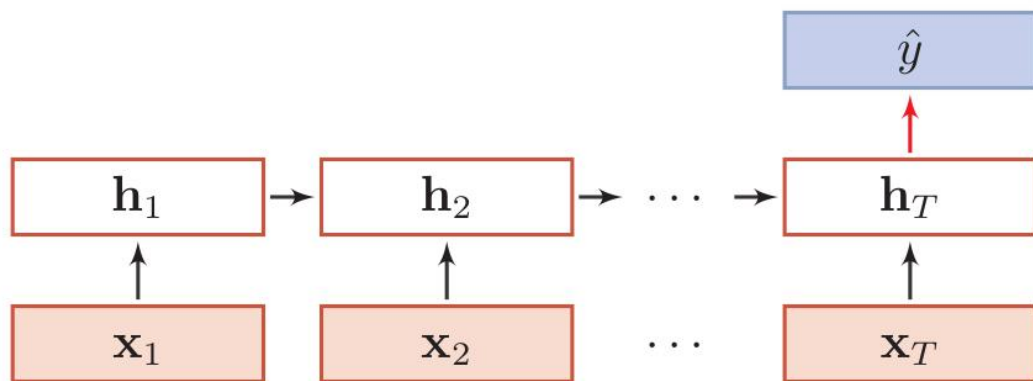


应用到机器学习

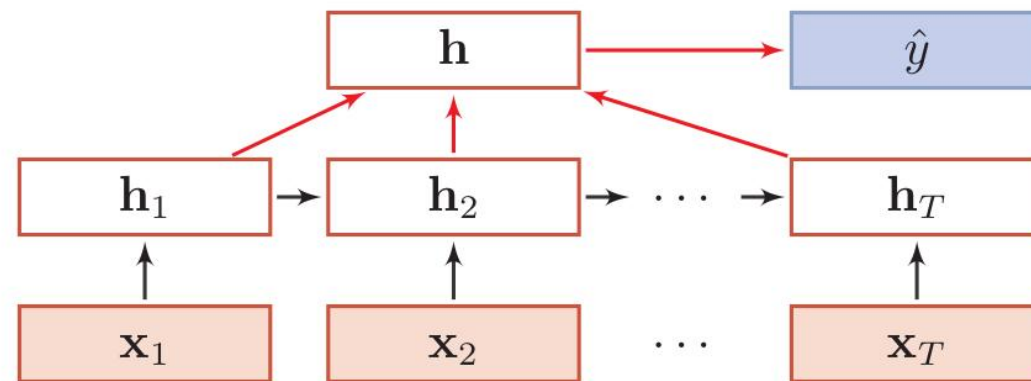
- ▶ 序列到类别
- ▶ 同步的序列到序列模式
- ▶ 异步的序列到序列模式

应用到机器学习

► 序列到类别



(a) 正常模式

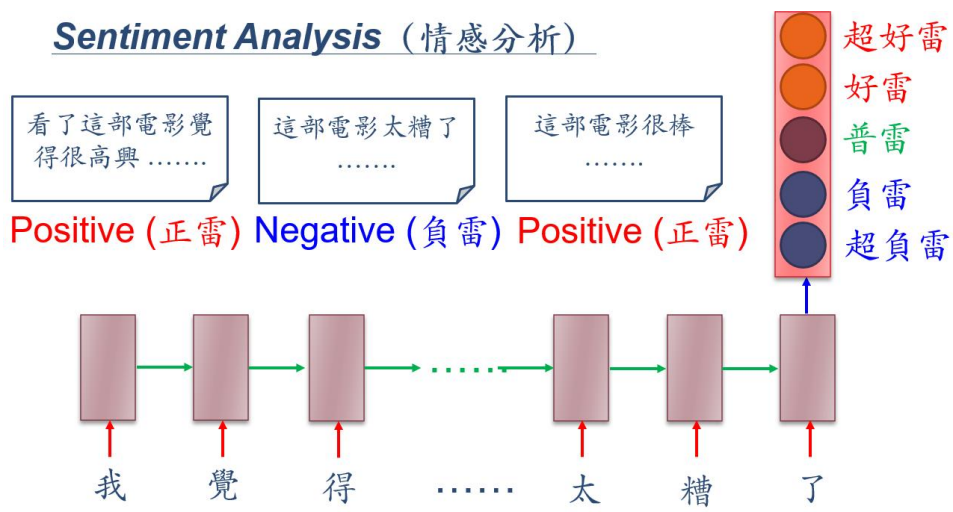


(b) 按时间进行平均采样模式

假设一个样本 $\mathbf{x}_{1:T} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ 为一个长度为 T 的序列，输出为一个类别 $y \in \{1, \dots, C\}$ 。我们可以将样本 $\mathbf{x}$ 按不同时刻输入到循环神经网络中，并得到不 同时刻的隐藏状态 $\mathbf{h}_1, \dots, \mathbf{h}_T$ 。然后可以将 $\mathbf{h}_T$ 看作整个序列的最终表示(或特 征)，并输入给分类器 $g(\cdot)$ 进行分类，即 $\hat{y} = g(\mathbf{h}_T)$ ，其中 $g(\cdot)$ 可以是简单的线性 分类器(比如 *Logistic* 回归)或复杂的分类器(比如 多层前馈神经网络)。

应用到机器学习

- “序列到类别”模式是指循环神经网络（RNN）接受一个完整的输入序列，然后输出一个类别标签，而不是为每个时间步生成一个输出。这种模式适用于需要对整个序列进行整体理解和分类的任务，例如情感分析、文本分类、语音识别中的说话人识别等。



输入层：每个评论分解成一个单词序列，例如 “这部电影很精彩” 可以转化为单词向量序列 $[v_1, v_2, v_3, v_4]$ 。

嵌入层：使用词嵌入将每个单词转换为固定维度的向量。

循环层：使用一个 RNN（例如 LSTM 或 GRU）逐步处理嵌入序列。

每个时间步更新隐藏状态，最终得到整个评论的隐藏状态 $h(T)$ 。

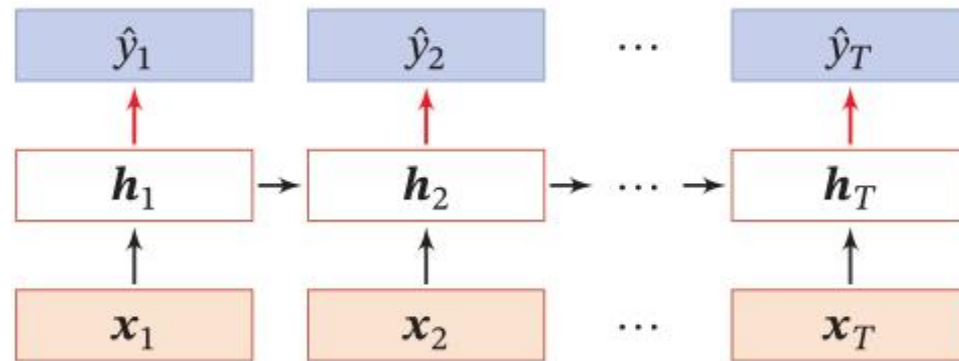
输出层：将最终隐藏状态 $h(T)$ 通过一个全连接层映射到 2 个神经元上，并用 softmax 输出类别概率。如果 softmax 输出的概率中，“正面” 概率较高，则评论分类为正面；反之，则为负面。

应用到机器学习

- ▶ 序列到类别：利用 RNN 对输入序列进行时间递归处理，将整句的信息整合在最终隐藏状态 $h(T)$ 中；通过全连接层将 $h(T)$ 映射到类别空间，生成类别打分；经过 softmax 处理，将打分转换为概率，从而输出最终的类别标签。
- ▶ 这种“序列到类别”的模式在文本分类、情感分析、语音命令识别等任务中非常常见，它的核心思想就是将整个序列压缩成一个全局的表示，再进行分类决策。

应用到机器学习

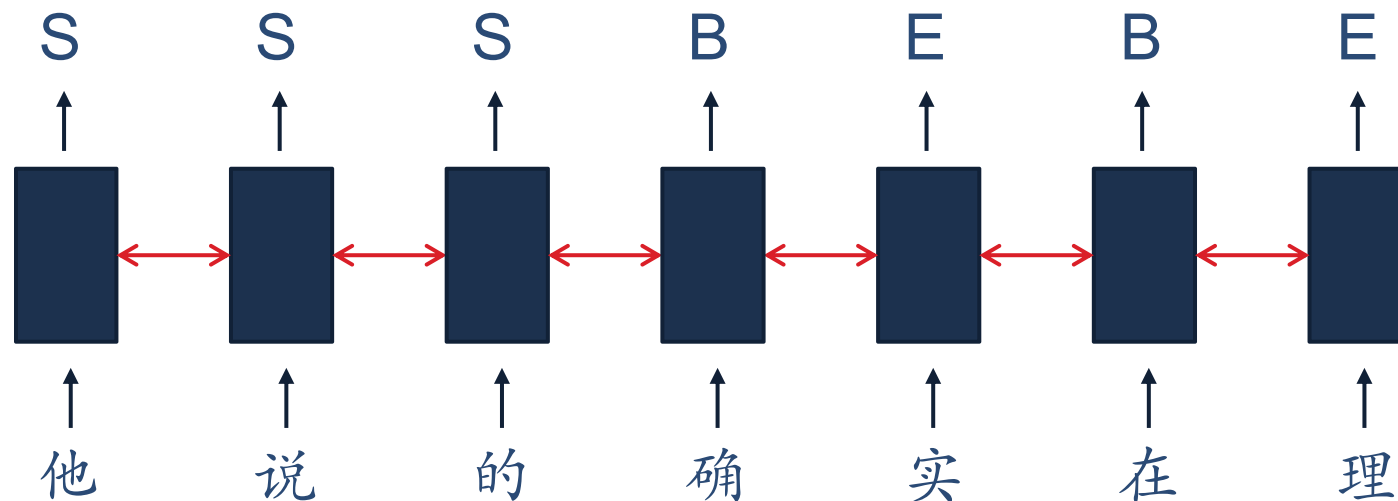
- ▶ 同步的序列到序列模式
- ▶ 举例：金融数据预测、传感器数据监控、音频信号处理（例如去噪、增强）等



该模式适用于输入和输出长度相同且时序对应的任务。在上述场景中，同步的模型能够捕捉局部时序变化，减少不必要的信息压缩和解码步骤，从而提高预测或恢复精度。同步的序列到序列模式主要用于序列标注 (Sequence Labeling) 任务，即每一时刻都有输入和输出，输入序列和输出序列的长度相同。

同步的序列到序列模式

► 中文分词



比如在词性标注(Part-of-Speech Tagging)中，每一个单词都需要标注其对应的词性标签。在同步的序列到序列模式中，输入为一个长度为 T 的序列 $\mathbf{x}_{1:T} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$ ，输出为序列 $\mathbf{y}_{1:T} = (\mathbf{y}_1, \dots, \mathbf{y}_T)$ 。样本 $\mathbf{x}$ 按不同时刻输入到循环神经网络中，并得到不同时刻的隐状态 $\mathbf{h}_1, \dots, \mathbf{h}_T$ 。每个时刻的隐状态 $\mathbf{h}_t$ 代表了当前时刻和历史信息，并输入给分类器 $g(\cdot)$ 得到当前时刻的标签 $\hat{y}_t$ ，即 $\hat{y}_t = g(\mathbf{h}_t)$, $\forall t \in [1, T]$ 。

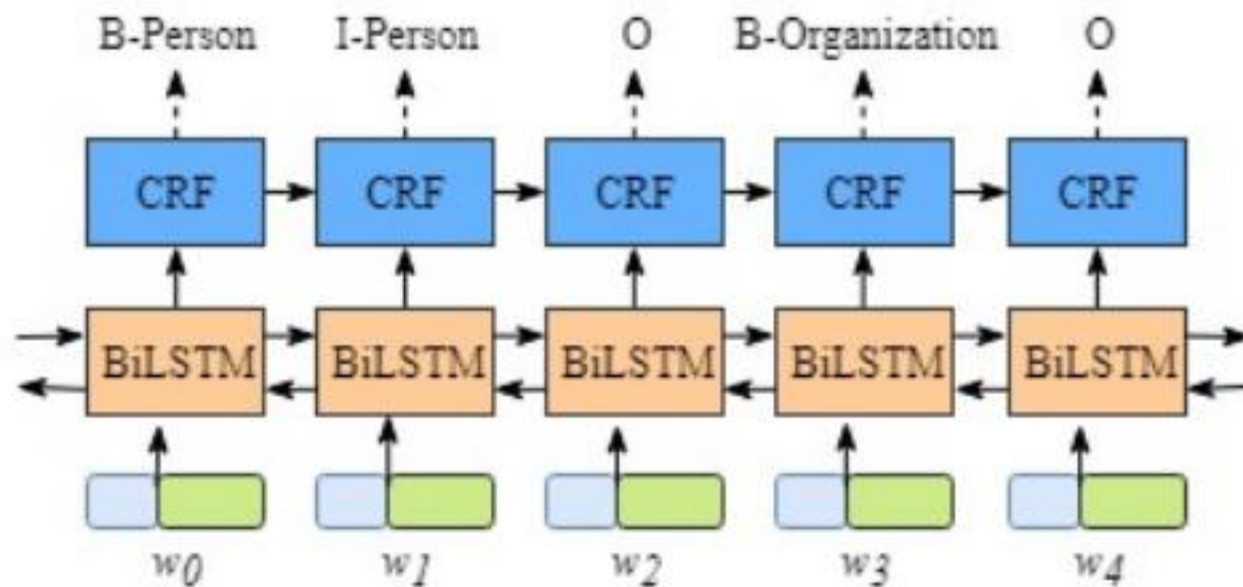
同步的序列到序列模式

► 信息抽取(Information Extraction, IE)

► 从无结构的文本中抽取结构化的信息，形成知识

小米创始人雷军表示，该公司2015年营收达到780亿元人民币，较2014年的743亿元人民币增长了5%。

RNN（或更常见的BiLSTM）用于建模词序列的上下文依赖，但实际输出标签（如 B-Person/I-Person）通常由其后接的条件随机场（CRF）或分类层生成。RNN 本身不定义标签体系，而是处理序列输入并提取特征，供下游任务使用。

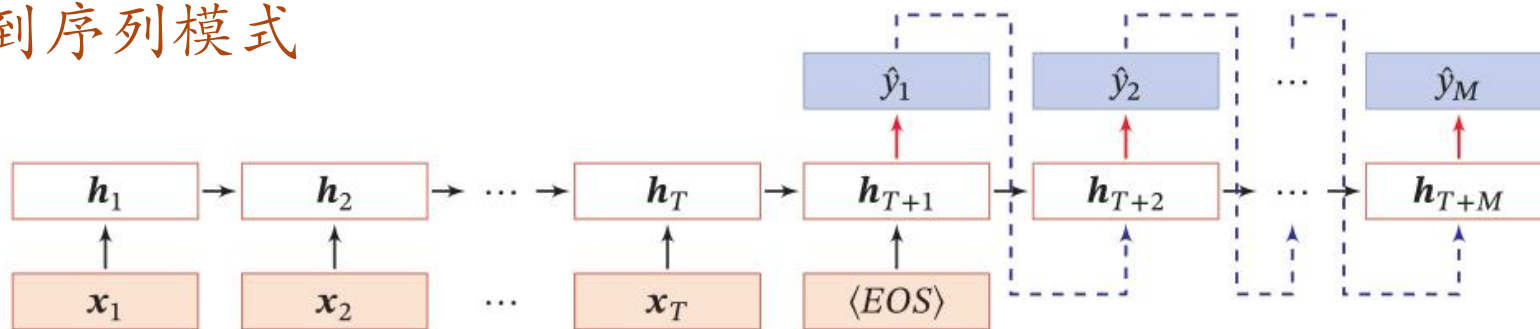


B-Person” 和 “I-Person” 是命名实体识别（NER）任务中的标签格式，常由 RNN（如 BiLSTM）等序列模型输出。

- B-Person: 表示一个人名实体的开始标记 (Begin)。
- I-Person: 表示一个人名实体的内部标记 (Inside)，即延续部分。

应用到机器学习

► 异步的序列到序列模式



异步的序列到序列模式也称为编码器-解码器(Encoder-Decoder)模型，即输入序列和输出序列不需要有严格的对应关系，也不需要保持相同的长度。比如在机器翻译中，输入为源语言的单词序列，输出为目标语言的单词序列。在异步的序列到序列模式中，输入为长度为 T 的序列 $\mathbf{x}_{1:T} = (x_1, \dots, x_T)$ ，输出为长度为 M 的序列 $\mathbf{y}_{1:M} = (y_1, \dots, y_M)$ 。

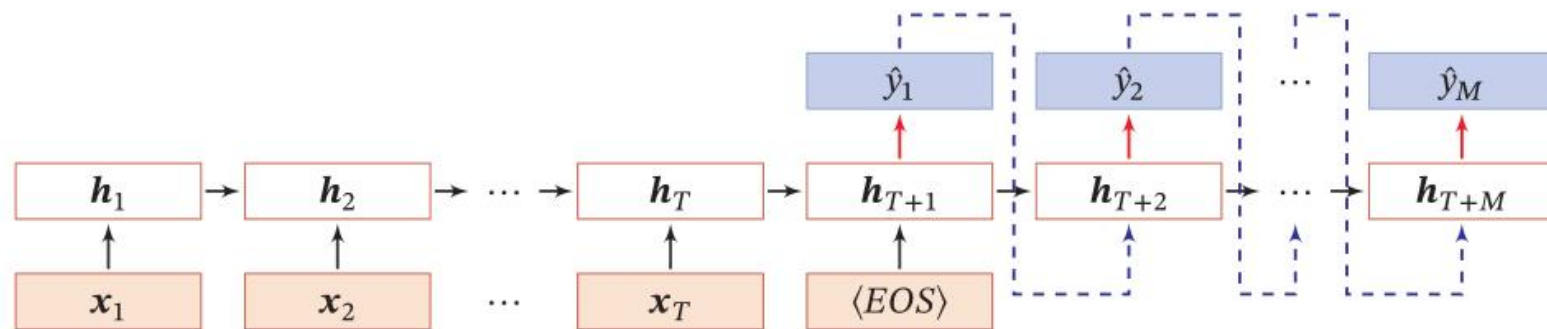
异步的序列到序列模式一般通过先编码后解码的方式来实现。先将样本 $\mathbf{x}$ 按不同时刻输入到一个循环神经网络(编码器)中，并得到其编码 $\mathbf{h}_T$ 。然后再使用另一个循环神经网络(解码器)，得到输出序列 $\mathbf{y}_{1:M}$ 。

应用到机器学习

► 异步的序列到序列模式

同声翻译：1. **输入语音帧序列**：[今，天，天，气，很，好]（逐帧输入）

2. **输出翻译序列**：[The, weather, is, nice, today]（可能在输入到"气"时开始输出"The weather"，后续调整）



场景描述输入：一段连续的中文语音（如"今天天气很好"），按时间分帧输入。输出：对应的英文翻译（"The weather is nice today"），输出可能比输入延迟几秒。

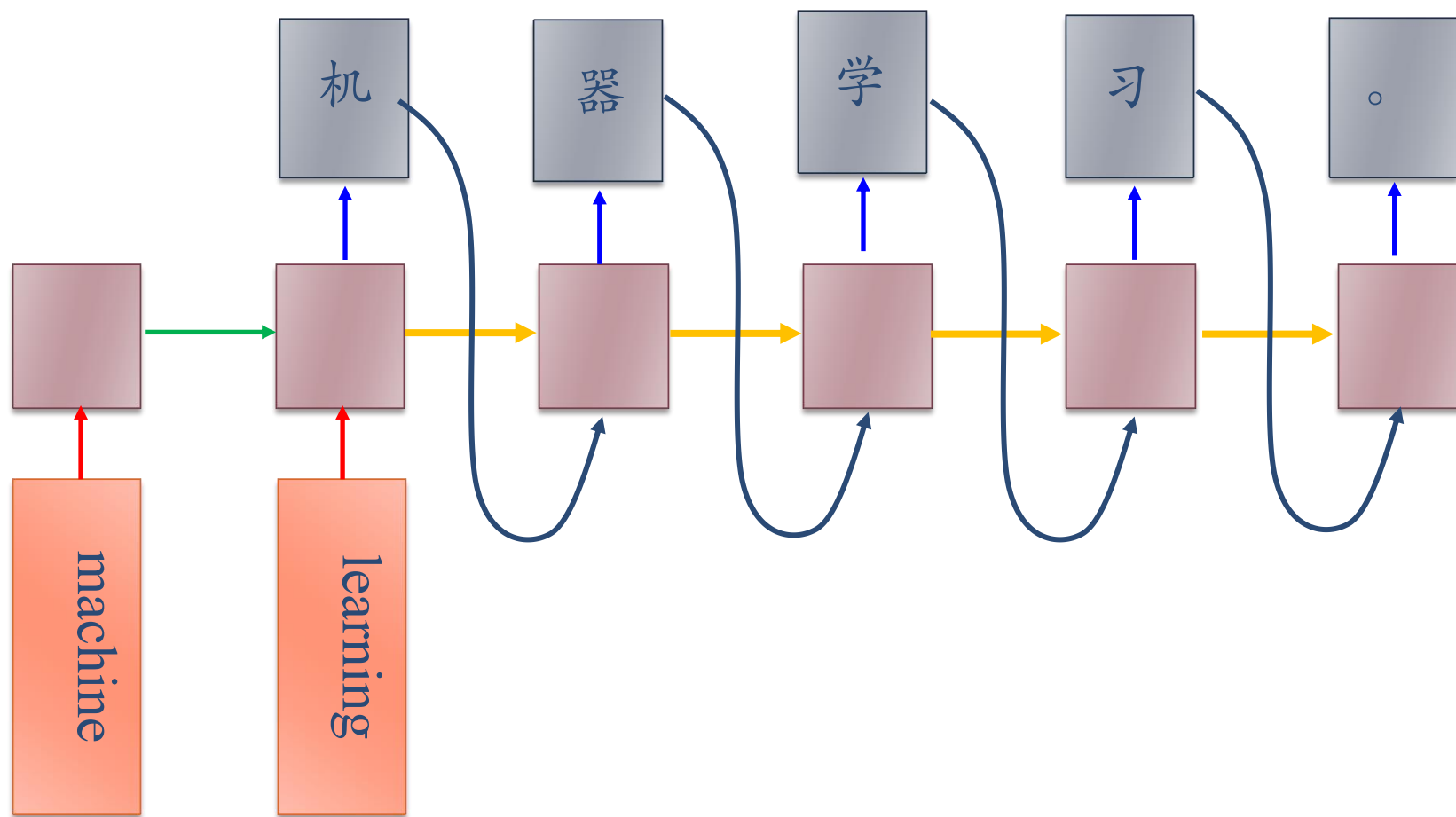
异步性体现输入（语音帧）是实时流入的，而输出（翻译结果）需要积累足够上下文后才能生成。输入序列长度（语音时长）与输出序列长度（英文单词数）不匹配。

RNN处理过程--编码器Encoder：使用RNN（如LSTM）逐步处理语音帧，生成隐藏状态。解码器Decoder：在输入未完全接收时，解码器已开始生成部分翻译，但需等待关键信息（如动词位置）后再输出完整句子。

注意力机制（Attention）：动态对齐输入语音帧与输出单词的对应关系。

异步的序列到序列模式

► 机器翻译



循环网络应用

语言模型

▶ 自然语言理解 → 一个句子的可能性/合理性

- ▶ ! 在报那猫告做只
- ▶ 那只猫在作报告!
- ▶ 那个人在作报告!

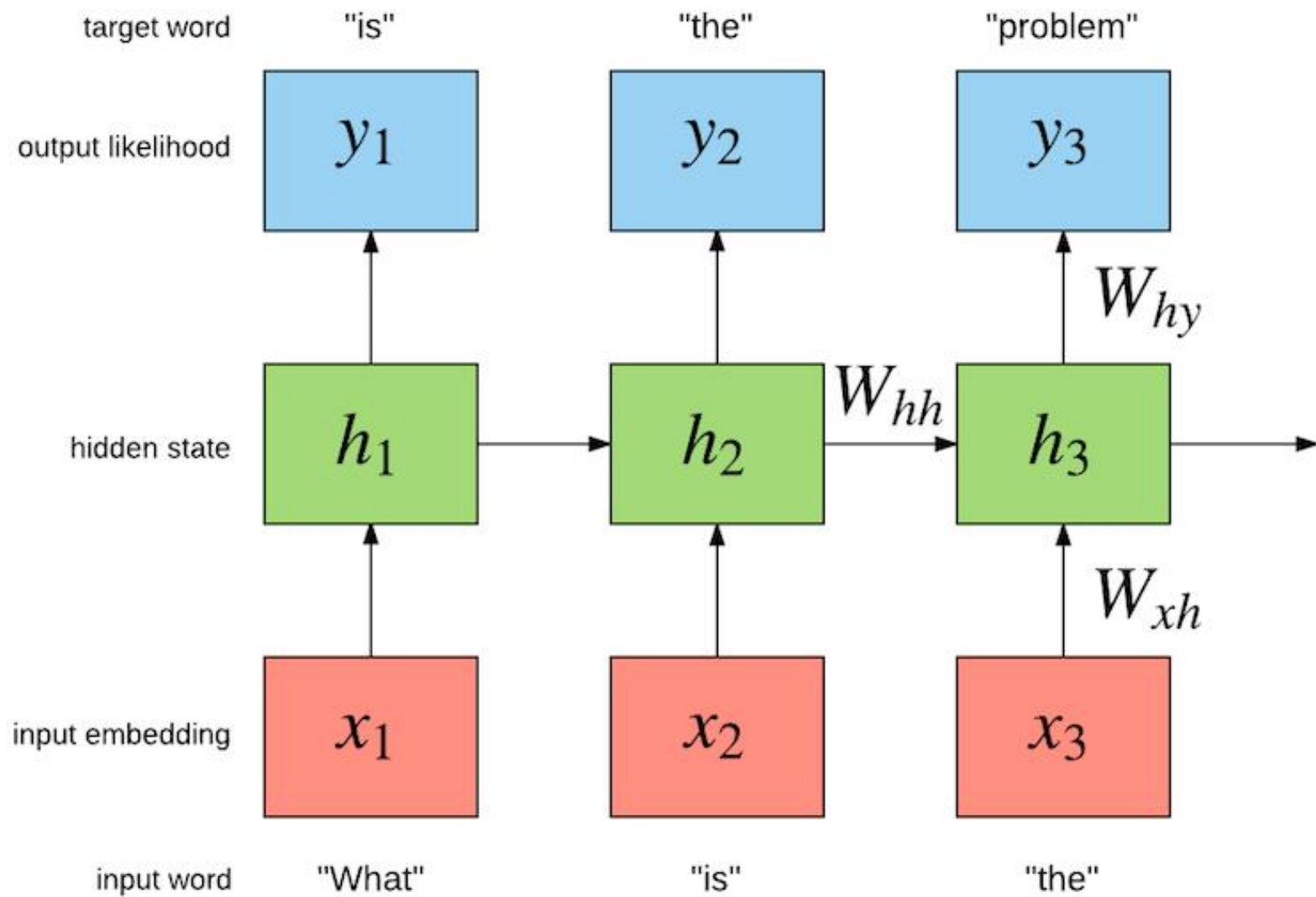


▶ 一切都是概率!

- ▶ $P(x_1, x_2, \dots, x_T)$
- ▶ $= \prod_i P(x_i | x_{i-1}, \dots, x_1)$
- ▶ $\approx \prod_i P(x_i | x_{i-1}, \dots, x_{i-n+1})$

N元语言模型

语言模型



生成Linux内核代码

```
/*
 * If this error is set, we will need anything right after that BSD.
 */
static void action_new_function(struct s_stat_info *wb)
{
    unsigned long flags;
    int lel_idx_bit = e->edd, *sys & ~((unsigned long) *FIRST_COMPAT);
    buf[0] = 0xFFFFFFFF & (bit << 4);
    min(inc, slist->bytes);
    printk(KERN_WARNING "Memory allocated %02x/%02x, "
        "original MLL instead\n"),
        min(min(multi_run - s->len, max) * num_data_in),
        frame_pos, sz + first_seg);
    div_u64_w(val, inb_p);
    spin_unlock(&disk->queue_lock);
    mutex_unlock(&s->sock->mutex);
    mutex_unlock(&func->mutex);
    return disassemble(info->pending_bh);
}

static void num_serial_settings(struct tty_struct *tty)
{
    if (tty == tty)
        disable_single_st_p(dev);
    pci_disable_spool(port);
}
```



作词机

► RNN在“学习”过汪峰全部作品后自动生成的歌词

► <https://github.com/phunterlau/wangfeng-rnn>

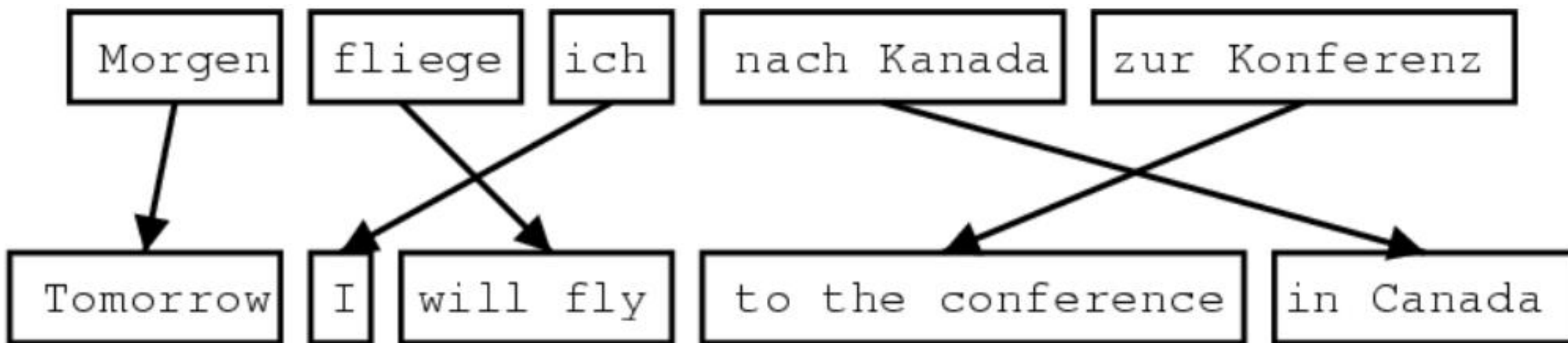
我在这里中的夜里
就像一场是一种生命的意叶
就像我的生活变得在我一样
可我们这是一个知道
我只是一天你会怎吗
可我们这是我们是不要为你
我们想这有一种生活的时候

作诗

白鹭窥鱼立， Egrets stood, peeping fishes. 青山照水开。 Water was still, reflecting mountains. 夜来风不动， The wind went down by nightfall, 明月见楼台。 as the moon came up by the tower.	满怀风月一枝春， Budding branches are full of romance. 未见梅花亦可人。 Plum blossoms are invisible but adorable. 不为东风无此客， With the east wind comes Spring. 世间何处是前身。 Where on earth do I come from?
--	--

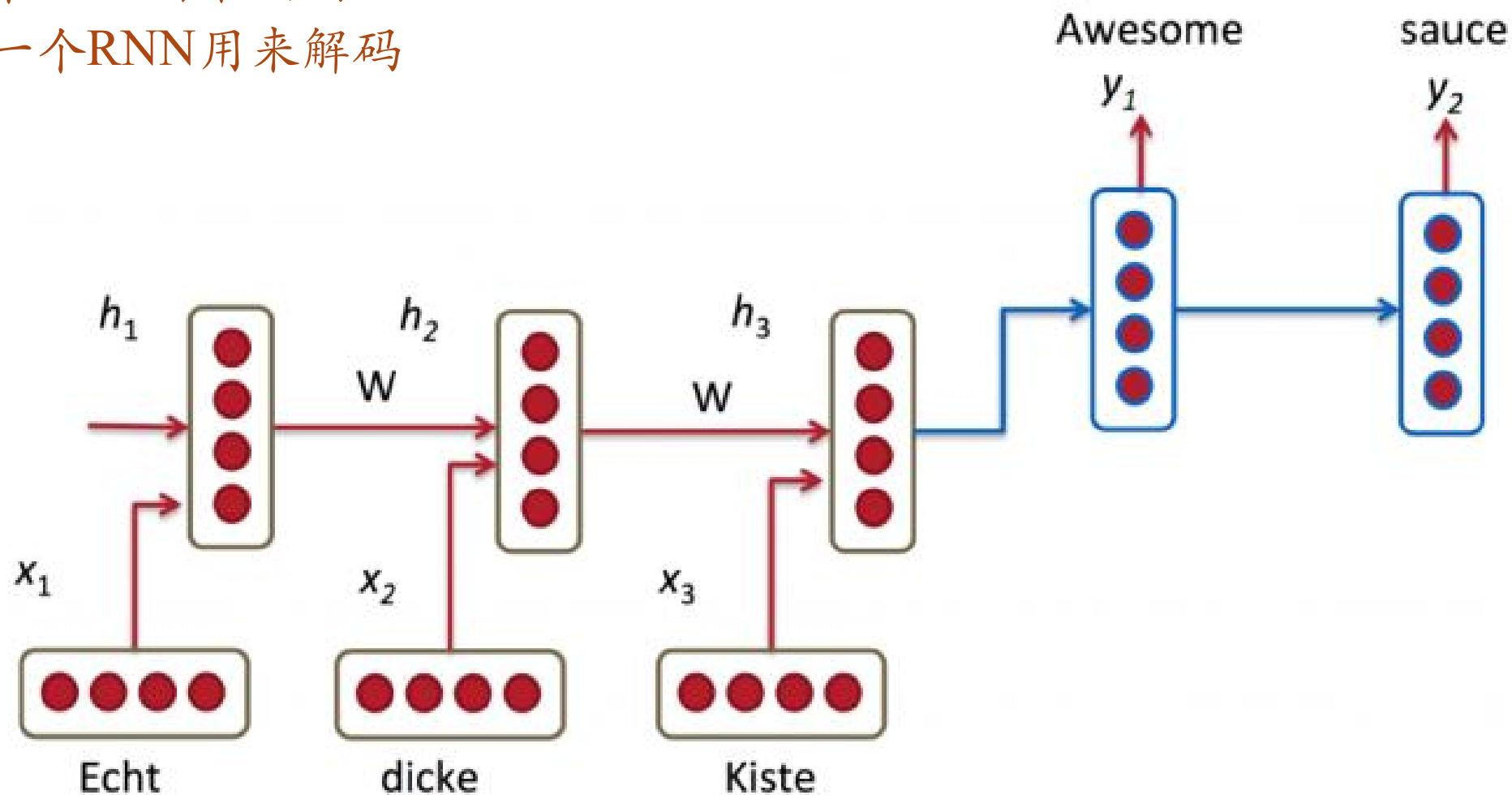
传统统计机器翻译

- ▶ 源语言: f
- ▶ 目标语言: e
 - ▶ 模型: $\hat{e} = \operatorname{argmax}_e p(e|f) = \operatorname{argmax}_e p(f|e)p(e)$
 - ▶ $p(f|e)$: 翻译模型
 - ▶ $p(e)$: 语言模型

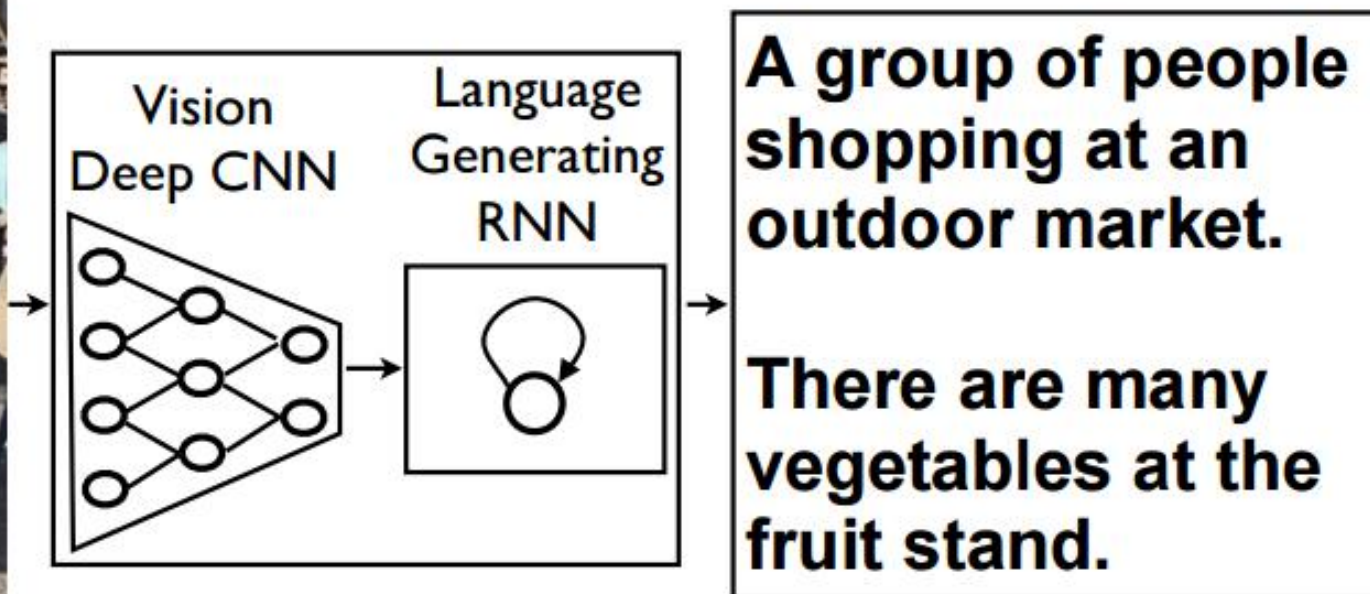


基于序列到序列的机器翻译

- ▶ 一个RNN用来编码
- ▶ 另一个RNN用来解码



看图说话



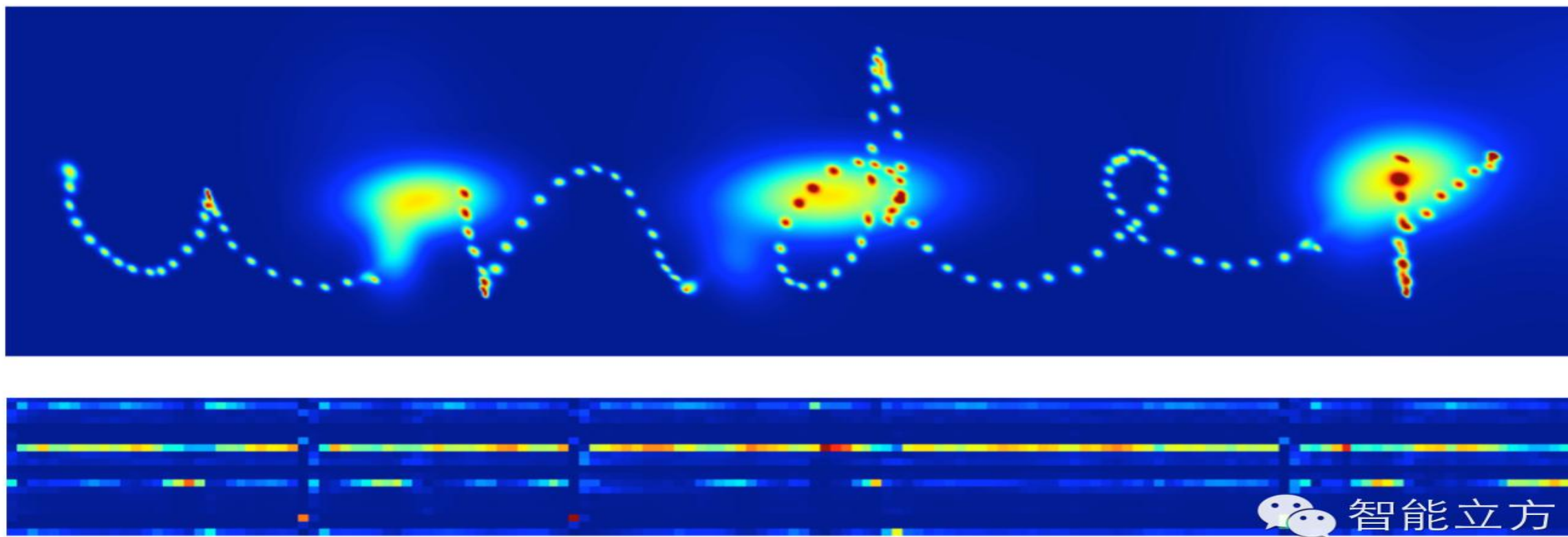
看图说话



Figure 5. A selection of evaluation results, grouped by human rating.

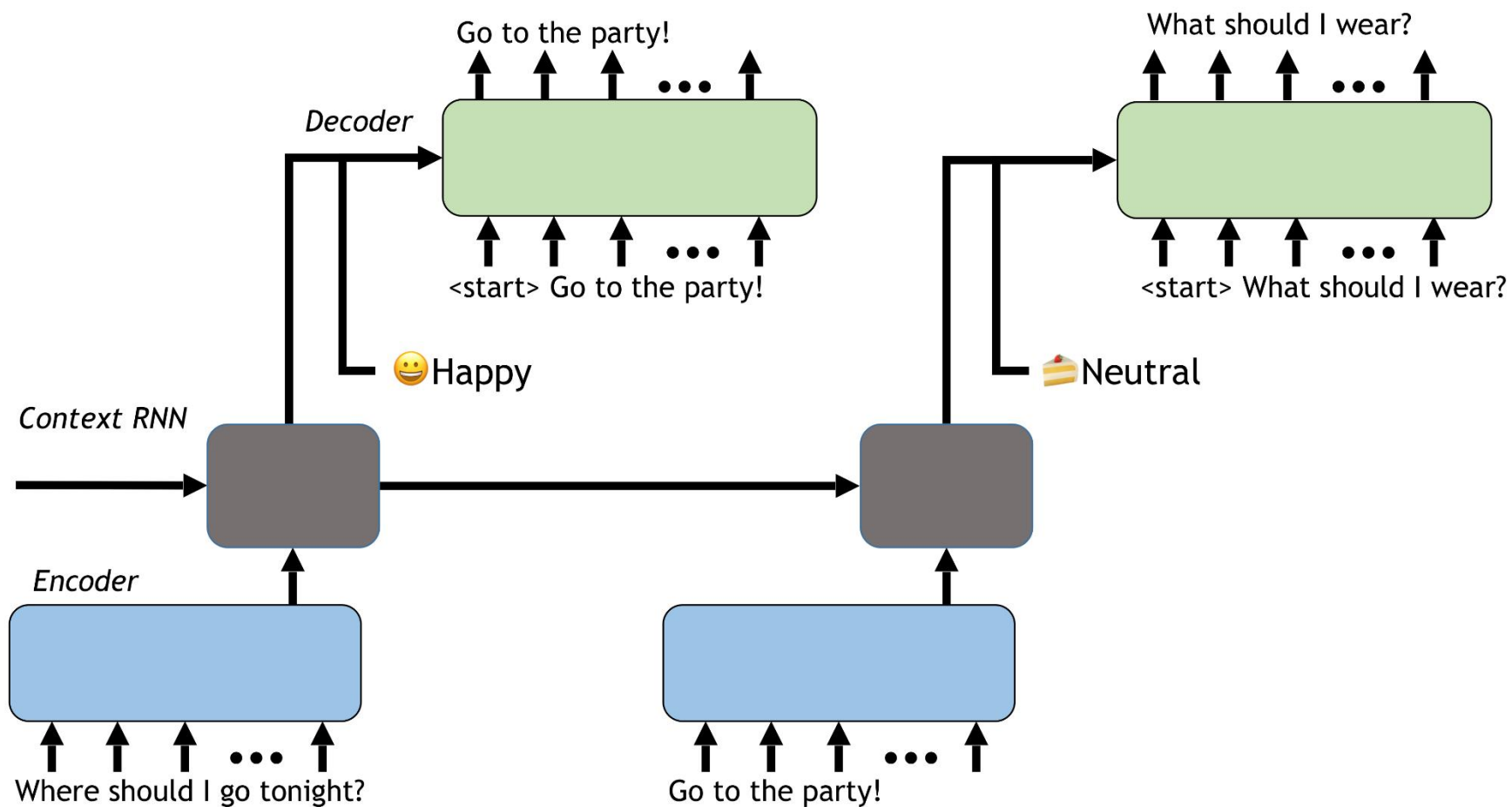
写字

- ▶ 把一个字母的书写轨迹看作是一连串的点。一个字母的“写法”其实是每一个点相对于前一个点的偏移量，记为(offset x, offset y)。再增加一维取值为0或1来记录是否应该“提笔”。



对话系统

<https://github.com/lukalabs/cakechat>



循环神经网络总结

▶ 优点:

- ▶ 引入记忆
- ▶ 图灵完备

▶ 缺点:

- ▶ 长程依赖问题
- ▶ 记忆容量问题
- ▶ 并行能力

课后作业

▶ 编程练习

▶ <https://github.com/nndl/exercise/>

▶ [chap6_RNN](#)

- ▶ 1) 利用循环神经网络来生成唐诗
- ▶ 2) 利用循环神经网络来进行加法运算

谢 谢

<https://nndl.github.io/>